The Cryptographic Protocol Shapes Analyzer: A Manual for CPSA 4[1]

Moses D. Liskov John D. Ramsdell
Joshua D. Guttman Paul D. Rowe

The MITRE Corporation

CPSA Version 4.4.7

February 5, 2026

[1] © 2016–2022 The MITRE Corporation. Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, this copyright notice and the title of the publication and its date appear, and notice in given that copying is by permission of The MITRE Corporation.

Contents

1 Introduction **1**
1.1 Recommended reading . . . 2
1.2 Tool components . . . 2

I Basic use of CPSA **4**

2 Setup and Installation **5**
2.1 Basic Installation . . . 5
2.1.1 Getting the Source . . . 5
2.2 Finding Documentation . . . 6
2.3 Running CPSA . . . 6
2.3.1 CPSA Projects . . . 7
2.3.2 Using the CPSA Makefile . . . 7
2.3.3 Using the Haskell Makefile . . . 7
2.3.4 Memory usage . . . 8
2.3.5 Parallelism . . . 8

3 Basic Protocol Modeling and Analysis with CPSA **9**
3.1 Basic CPSA modeling . . . 10
3.2 CPSA input . . . 11
3.3 CPSA output . . . 13
3.4 Interpreting shapes . . . 16
3.5 Blanchet's simple example protocol . . . 18

4 Algebra Features of CPSA **25**
4.1 Generic messages and long-term keys . . . 26
4.2 Modeling Diffie-Hellman . . . 31
4.2.1 Other examples . . . 34
4.3 Other Algebra Features . . . 35
4.3.1 Hashing . . . 35

4.3.2 Constants . . . 35
4.3.3 Bidirectional Long-Term Keys . . . 36
4.4 The Lang Field: Declaring new operators . . . 36

II Understanding and Guiding CPSA 39

5 The CPSA Search Algorithm 40
5.1 Solving tests . . . 41
5.2 Flawed Kerberos, revisited . . . 42
5.2.1 The operation field . . . 42

6 Constraining CPSA's search 46
6.1 Bundles: A Strand-Based Execution Model . . . 46
6.2 Secrecy assumptions . . . 47
6.3 Diffie-Hellman declarations . . . 48
6.4 Declaring skeletons . . . 49
6.5 A Caution about typos . . . 50

III Advanced features of CPSA 51

7 Logical Security Goals and Rules 52
7.1 Overview . . . 55
7.2 Semantics . . . 55
7.3 Examples . . . 57
7.3.1 Needham-Schroeder Responder . . . 58
7.3.2 A Needham-Schroeder Secrecy Goal . . . 58
7.4 The Rest of the Story . . . 58
7.5 Syntactic Abbreviations . . . 61
7.6 Shape Analysis Sentences . . . 62
7.7 Rules . . . 63
7.7.1 Facts . . . 63
7.7.2 DoorSEP . . . 64
7.7.3 Generated rules . . . 68
7.7.4 Rules generated for trace annotations . . . 69

8 Channels and State 76
8.1 Channels . . . 78
8.1.1 Yahalom with Channels . . . 78
8.1.2 Yahalom with Channels, 1: Role-level assumptions . . . 79
8.1.3 Yahalom with Channels, 2: Per-strand assumptions . . . 81

8.2 State . . . 82
8.2.1 The axioms of state . . . 84
8.2.2 Consequences of the axioms . . . 85
8.2.3 Syntax for state . . . 86
8.2.4 Channels and state: Summary . . . 89

IV Reference material **91**

9 Troubleshooting **92**
9.1 Non-termination . . . 92
9.1.1 Tweaking the search . . . 93
9.2 Error messages . . . 94

10 CPSA Program **100**
10.1 CPSA pre-processing . . . 100
10.2 CPSA input syntax . . . 101
10.3 Algebra reference . . . 104
10.3.1 Basic crypto algebra . . . 104
10.3.2 The Diffie-Hellman crypto algebra . . . 104
10.4 Declaration syntax . . . 105
10.5 Command-line options . . . 105
10.5.1 Heralds . . . 109

11 CPSA Tools **110**

List of Figures

3.1 Needham-Schroeder roles 10
3.2 Needham-Schroeder `defprotocol` 11
3.3 Initiator point of view 12
3.4 Responder point of view 12
3.5 Needham-Schroeder search tree 14
3.6 Needham-Schroeder skeleton 15
3.7 NS shape, initiator point of view 17
3.8 NS shape for responder point of view 19
3.9 The Blanchet simple example protocol 20
3.10 Blanchet points of view 21
3.11 Blanchet privacy search tree 22
3.12 Blanchet shape for responder's point of view 23

4.1 A flawed version of Kerberos 27
4.2 Flawed Kerberos point of view 27
4.3 Flawed Kerberos dead skeleton 28
4.4 Flawed Kerberos with a generic ticket 29
4.5 Flawed Kerberos shape 30
4.6 Diffie-Hellman defprotocol 33
4.7 Deletion of strand 34

5.1 Flawed Kerberos key moment in analysis 44

7.1 Needham-Schroeder Initiator Point of View 54
7.2 Needham-Schroeder Responder Point of View 59
7.3 Needham-Schroeder Secrecy Goal 60
7.4 Two Initiator Authentication Goals 60
7.5 Initiator Shape Analysis Sentence 62
7.6 DoorSEP Protocol 64
7.7 DoorSEP Weakness 65
7.8 Door Simple Example Protocol 66
7.9 Rule with a flag to control its use 67

7.10 Minipay rely/guarantee formulas 72
7.11 Goal statements, 1 73
7.12 Goal statements 74

8.1 Yahalom (role decls.) 80
8.2 The responder's point of view 81
8.3 Responder's first guarantee 82
8.4 Leads-to axioms 84
8.5 Three derived rules for state: Scissors, no interruption, and cake 85
8.6 Device-open role causing door to open 87
8.7 State segments are critical sections 89
8.8 Critical section: Generated rules 90

11.1 `defprot` Example 112

List of Tables

4.1 Lang Field Syntax 37

7.1 Goal syntax 53

10.1 CPSA Input Syntax 102
10.2 The Basic Cryptoalgebra 103
10.3 CPSA Basic Algebra Syntax; sorts, operators ID from LANG field 104
10.4 The Diffie-Hellman Cryptoalgebra 106
10.5 CPSA Diffie-Hellman Algebra Syntax 107
10.6 Declaration syntax 107

11.1 `defprot` Grammar 111
11.2 `defprot` commands 111

Chapter 1

Introduction

CPSA, the Cryptographic Protocol Shapes Analyzer, is a software tool designed to assist in the design and analysis of cryptographic protocols. A cryptographic protocol is a specific pattern of interaction between principals. TLS and IKE are some examples of well-known cryptographic protocols.

The tool takes as input a protocol definition and a partial description of an execution, each built within a particular formal model by the user. It attempts to produce descriptions of *all* possible executions of the protocol that complete the partial description, consistent with the presence of a powerful network adversary capable of diverting, altering, replaying, or dropping network messages. Such an adversary may be able to manipulate honest participants into an unexpected execution, breaking a secrecy or authentication property that the protocol was intended to achieve.

Naturally, there are infinitely many possible executions of a useful protocol, since different participants can run it with varying parameters, and the participants can run it repeatedly. However, for many naturally occurring protocols, there are only finitely many of these runs that are *essentially* different. Indeed, there are frequently very few, often just one or two, even in cases where the protocol is flawed. We call these minimal, essentially different executions the *shapes* of the protocol. Authentication and secrecy properties are easy to "read off" from the shapes, as are attacks and anomalies.

The analysis performed by CPSA is done within a pure Dolev-Yao model [3]; as such, the analysis reveals only structural flaws in the protocols it analyzes. It cannot detect flaws in underlying cryptographic algorithms or in actual implementations of protocol participants.

The purpose of this document is to provide the background required to make effective use of the CPSA software distribution.

1.1 Recommended reading

If you are new to CPSA, it is recommended that you first read Part I, which is introductory in nature and presented as a tutorial. Chapter 2 discusses how to download, install, and run the tool. Chapter 3 begins the tutorial describing how to use the tool, and Chapter 4 describes some of the more important additional features.

Readers, especially those without direct access to an experienced user of the tool, are encouraged to attempt the "explorations" present from Chapter 3 onwards. The purpose of the explorations is to give the reader experience in the use of the tool and a chance to test his or her understanding of its features. After reading through the first part, you should be ready to attempt to use the tool to analyze a protocol that interests you, whether it be an existing protocol or one you need to design.

When you are ready for a deeper base of knowledge, read Part II. The chapters in Part II will be helpful as you cross from trying to understand how to use the tool to trying to impact your work on protocol design through use of the tool. Chapter 5 will be of general use as you try to understand the analyses CPSA conducts, and Chapter 6 will help you narrow the tool's focus to what interests you.

Part III deals with special-purpose features of the tool. Chapter 8 deals with stateful protocols and Chapter 7 deals with logic-based protocol goals. You should read these chapters if those features seem important to you.

Part IV is reference material. Chapter 9 contains reference material about dealing with errors of various sorts that arise during the use of CPSA, while Chapter 10 documents the complete syntax of the tool.

1.2 Tool components

The distribution includes a number of separate executable command-line tools. Of these, three are key components of the core expected workflow: `cpsa4`, `cpsa4graph`, and `cpsa4shape`. The other tools are auxiliary utilities most general users will not need to use, although `cpsa4diff` is of some use to a general user when updating the tool.

The `cpsa4` program takes as input one or more analysis problems (at least, a protocol and a partial description of an execution), and analyzes them one at a time. It outputs the full, step-by-step analysis for each problem, ultimately describing all possible full executions that complete the partial execution.

The `cpsa4shape` program takes an analysis and reduces it to an analysis that skips directly from analysis input to the set of shapes associated with that input.

The `cpsa4graph` program takes an analysis (from either `cpsa4` or `cpsa4shape`) and formats it into an XHTML file and includes SVG(scalable vector graphics) diagrams of each partial execution as well as the overall branching pattern of the analysis.

The expected work flow follows. An analysis problem is entered using an ordinary text editor, preferably one with support for Lisp syntax. The `cpsa4` program uses an S-

expression-based syntax for both input and output. S-expression is an abbreviation for a Symbolic Expression (as in the Lisp programming language).

The body of the input consists of two forms: `defprotocol` statements that describe a protocol, and `defskeleton` statements that describe a partial execution of a protocol. The exact details of both forms depend on the message algebra specified by the protocol.

Assuming there are no errors in the input, the analyzer will produce output as a text document. The text document contains each step used to derive a shape from a problem statement. It is common to filter the output using the `cpsa4shapes` program, and look only at the computed shapes associated with each problem statement.

The `cpsa4graph` program is applied to the output to produce a more readable, hyperlinked XHTML document that can be displayed in a standards-compliant web browser. See Chapter 11 for more information on the programs that come with a CPSA distribution.

Part I

Basic use of CPSA

Chapter 2

Setup and Installation

2.1 Basic Installation

The use of CPSA requires Haskell, the programming language in which CPSA is coded. Our recommendation is to use the Haskell Platform, which is available for Windows, Mac, and Linux. The "Core" platform is sufficient, but the "Full" platform is also fine. On Linux, install the `haskell-platform` package. Otherwise, follow the download and installation instructions at:

`http://www.haskell.org/platform/`

To install CPSA, run in a terminal or command prompt:
`$ cabal update` (to get the latest package list; this may take a while.)
`$ cabal install cpsa`

Note that if you are behind a proxy, you may have to set the http proxy for your shell if you haven't already. For example, on a mac:
`$ export http_proxy=http://proxy.myorg:port`
On Windows:
`$ set HTTP_PROXY=http://proxy.myorg:port`

Cabal will install CPSA in a directory specified in its config file (usually in /.cabal/config, unless you've installed Haskell in a different directory). Instructions for changing your path are included in the config file. The final step in the cabal install process should print the location that CPSA has been installed in.

2.1.1 Getting the Source

If you have trouble with `cabal`, or if you'd like extra features such as the CPSA test suite of example protocols, you can download the current source distribution directly at:

`http://github.com/mitre/cpsa`

There is a directory named `cpsa4` at the top-level of the repository. It contains a copy of the CPSA sources downloaded and compiled using `cabal`. On all platforms, to install from this source, change into the `cpsa4` directory and type:

```
$ cabal update
$ cabal install parallel
$ cabal configure
$ cabal build
$ cabal install
```

Alternatively, there are other install options described in `README.txt` in the `cpsa4` directory.

2.2 Finding Documentation

CPSA comes with documentation, but it can be difficult to locate by hand when the tool has been installed through cabal. Run

```
$ cpsa4 -h
```

to see the program's help message, including the documentation directory, where this manual should be found.

2.3 Running cpsa

To analyze a protocol you have put in prob.scm type:

```
$ cpsa4 -o prob.txt prob.scm
$ cpsa4shapes -o prob_shapes.txt prob.txt
$ cpsa4graph -o prob_shapes.xhtml prob_shapes.txt
```

See Section 10.5 for command-line options.

The `cpsa4shapes` command is optional, but recommended; it cuts down CPSA's output to only show final results. Unless you're doing detailed debugging, using it will make the output much easier to read.

To analyze a protocol without using the `cpsa4shapes` command, type:

```
$ cpsa4 -o prob.txt prob.scm
$ cpsa4graph -o prob.xhtml prob.txt
```

The .xhtml results can be opened in a web browser.

2.3.1 cpsa Projects

A CPSA project is a directory containing some CPSA input files and items that orchestrate the use of the various CPSA tools. To create a project, in a fresh directory, run `cpsa4init`.

```
$ mkdir project
$ cd project
$ cpsainit
Created Makefile
Created cpsa.mk
Created Make.hs
Created cpsa4prolog.pl
```

The `cpsa4init` will not overwrite a file if it is already present in the current directory. The distribution provides two ways to relieve users of the tedium of issuing individual commands.

2.3.2 Using the cpsa Makefile

The easiest way to orchestrate CPSA programs is to use GNU make. A CPSA project contains the files `Makefile` and `cpsa4.mk`. To analyze the problems in your project, type `make`. Edit the file `Makefile` to adjust program flags to suit your needs. For example, you many want to enable the use of multiple processors during protocol analysis as described in Section 2.3.5.

The CPSA program is Emacs friendly. If you run the above makefile via `M-x compile`, the results will be displayed in a buffer in Compilation Mode. The command `C-x ‘` will visit the locus of the next error message or match (next-error) in your CPSA input file.

2.3.3 Using the Haskell Makefile

This approach is designed to be easy for Windows users, who do not want to bother installing Cygwin or MSYS.

Locate `Make.hs` in your project directory. If using Windows, double-click on the file and it will open up a new window with a prompt. On a Mac or Linux machine, run `$ ghci Make.hs`

From the `Make.hs` prompt, you can use the following commands:

- `build`: Run CPSA on all protocols (.scm files) in the directory, and produce .xhtml output files displaying the results.
- `clean`: Remove all CPSA output files, to ensure that any changes to protocol files are reflected in the output.

  - Because intermediate files are used for behind-the-scenes processing, it can be possible to have the results in the .xhtml output files not reflect the most up-to-date protocol file contents. If you make changes and don't see them reflected in the output, try running `clean`. Getting into the habit of running `clean` before `build` is a good idea.

- `cpsa4 "`*`protocolname`*`"` Run CPSA on just the protocol provided. Note that the file extension (.scm) should *not* be included in the name; if your protocol is in `foo.scm`, you would run `cpsa4 "foo"`. Most useful if you have an exceptionally large number of protocols in a single directory.

- `:q` Quit.

2.3.4 Memory usage

On large problems, CPSA can consume large amounts of memory. To protect against memory exhaustion, run CPSA with the command-line options `+RTS -M512m -RTS`. The initial makefile includes these options by default.

2.3.5 Parallelism

CPSA is built so it can make use of multiple processors. To make use of more than one processor, start CPSA with a runtime flag that specifies the number of processors to be used, such as `+RTS -N4 -RTS`. The GHC documentation describes the `-N` option in detail.

Chapter 3

Basic Protocol Modeling and Analysis with CPSA

This chapter is designed to be a tutorial for a new user with access to the tool, but totally unfamiliar with the ideas behind it. We will explain the basics of the tool while stepping through an example input and its output. Input *and output* for the examples in this chapter and in other chapters are included in the distribution. Explorations are included for readers to build their understanding of the tool through experience. If you are a new user but do not intend to work through the explorations, we recommend that you at least copy the input files and run the tool yourself to check that you can produce outputs mirroring those in the distribution.

The first protocol we discuss is the Needham-Schroeder protocol for establishing key transport over insecure networks. The protocol has two participants: an *initiator* a and a *responder* b. The intention is for the following interaction to take place:

1. a picks a fresh, random nonce n_1 and encrypts a message containing n_1 and a's name under b's public encryption key, and sends the result to b.

2. b, on receiving such a message, picks a fresh random nonce n_2 and encrypts a message containing n_1 and n_2 under a's public encryption key, and sends the result to a.

3. a, on receiving this reply, encrypts n_2 under b's public encryption key and sends the result to b.

The intention is that a and b should have authenticated each other (that is, that a is communicating with b and vice versa) and that the pair of nonces establish a unique session of such authentication. The nonces should also be unreadable by the network adversary, so that they can be used to create a session key between a and b.

This protocol has a well known but non-obvious flaw discovered by Lowe [10] that CPSA can discover automatically.

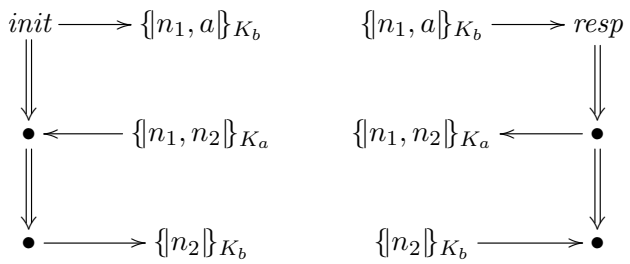


Figure 3.1: Needham-Schroeder initiator and responder roles

3.1 Basic CPSA modeling

In order to use CPSA on this protocol, we must first understand some basics about how CPSA models protocols and messages.

Since CPSA's aim is to analyze protocols in the presence of a powerful network attacker, we equate the network with the attacker, and do not model the notion of addressing of messages. The description provided above for Needham-Schroeder describes messages that (for instance) a sends to b. In CPSA, we ignore the intended recipient because the attacker is free to ignore it.

The protocol can be thought of as made up of the roles that entities can play during the protocol. In the case of Needham-Schroeder, there are two: the initiator and the responder. These roles describe the sequence of message-related events each party observes during the protocol. The events are described by giving a formula for the format of each message, along with an indiciation whether each event is a reception of a message or a transmission of one.

Messages in CPSA are represented as formally structured objects, specifically as terms in an order-sorted algebra [4]. Terms are either variables or functional outputs of simpler terms. Variables have types called *sorts*, and function symbols have specific signatures that specifies the sorts of each input and the sort of the output. The roles of Needham-Schroeder are given in Figure 3.1.

The messages in these roles are built from variables (n_1, n_2, a, b) and function symbols; the three function symbols used in this protocol are encryption, pairing, and the "key of" function.

$\{|m|\}_k$ denotes the encryption of message m under encryption key k. In cpsa, encryption operators are always regarded as offering authenticated encryption, meaning that a participant without access to an encryption key is overwhelmingly unlikely to generate a message that can be successfully descrypted. This property is primarily relevant in symmetric (secret-key) kinds of encryption.

Terms in a pair are represented in comma-separated lists. And K_a denotes the result

```
(defprotocol ns basic
  (defrole init
    (vars (a b name) (n1 n2 text))
    (trace (send (enc n1 a (pubk b)))
           (recv (enc n1 n2 (pubk a)))
           (send (enc n2 (pubk b))))
  (defrole resp ...))
```

Figure 3.2: Needham-Schroeder `defprotocol`

of the "key of" function symbol on input a. This represents the public key (either an encryption key or a signature verification key) of a. The values n_1 and n_2 are of a different sort than a and b: the latter are names to which the $K_{(\cdot)}$ function can be applied, while n_1 and n_2 are simple values.

In addition to a description of the protocol, CPSA expects the description of a what we call a *skeleton*—a partial protocol execution. A skeleton is made up of *instances* of the roles, that is, viewpoints of honest parties, along with what values are associated with the variables in those viewpoints. These viewpoints may be partial, but they always represent a prefix of a full role.

3.2 CPSA input

The `ns.scm` file in the examples directory contains a protocol description for Needham-Schroeder and two skeletons: one representing the viewpoint of a completed initiator, and one representing the viewpoint of a completed responder. The `.scm` extension used for CPSA input files refers to the Scheme programming language, which is a language derived from Lisp. This allows the user to make use of an IDE or text editor that knows about Scheme syntax, for ease of editing input files. The CPSA tool itself does not require any particular extension, but auxiliary tools may, including the `Make.hs` program described in Section 2.3.

The input file for Needham-Schroeder contains comments found on lines beginning with ';', and four top-level S-Expressions: a `herald`, a `defprotocol`, and two `defskeleton`s. We will describe heralds in Section 10.5.1; ignore them for now.

The `defprotocol` S-expression describes and names a protocol, while the `defskeleton` describes a skeleton, referencing a particular protocol. A portion of the Needham-Schroeder `defprotocol` is reproduced in Figure 3.2 for ease of reference.

A `defprotocol` S-expression starts with a protocol name, `ns` in this case, followed by the name of a message algebra. The `basic` algebra contains enough elements to describe Needham-Schroeder and most simple examples; the only other algebra contained in the

```
(defskeleton ns
  (vars (b name) (n1 text))
  (defstrand init 3 (b b) (n1 n1))
  (non-orig (privk b))
  (uniq-orig n1))
```

Figure 3.3: Initiator point of view

```
(defskeleton ns
  (vars (a name) (n2 text))
  (defstrand resp 3 (a a) (n2 n2))
  (non-orig (privk a))
  (uniq-orig n2))
```

Figure 3.4: Responder point of view

CPSA distribution is `diffie-hellman`; see Chapter 4, and specifically Section 4.2, for details of the Diffie-Hellman algbera.

The rest of the `defprotocol` S-expression is a sequence of roles, each defined by a `defrole` S-expression. In our example, there are two roles, and thus two `defrole`s, the first defining the initiator role (`init`) and the latter describing the responder role (`resp`).

The first input to a `defrole` is a role name; the second should be a set of variable declarations (`vars`), and the third should be a `trace` declaration which describes the event sequence of the role. The variable declarations define and give types to the variables used in the role's trace. The `trace` S-expression defines the list of events: a `recv` S-expression describes a reception and `send` describes a transmission.

Function symbols in the CPSA message algebra have specific S-expressions associated with them. `enc` denotes an encryption, `cat` denotes a pair, and `pubk` denotes the "key of" function. You may notice that `cat` does not occur in the figure: this is because its use is hidden by "syntactic sugar"—a convenient shortcut in the syntax. The message `(enc n1 a (pubk b))` is more properly the encryption of the pair (n_1, a) under the key K_b, but when an `enc` S-expression is given more than two inputs, it is assumed that all but the last are concatenated together using pairs.

In Figures 3.3 and 3.4, we reproduce the skeletons described in our example input file. A `defskeleton` S-expression includes first of all a protocol name, then variable declarations, and then a list of instances, most of which are defined by the `defstrand` S-expression. A defstrand includes an input specifying the name of the role the strand is an instance of, as well as a height, that is, a number of the events (`send` / `recv`) in the role that are to be reflected in the instance, starting from the first event. In our example input, each `defskeleton` includes one `defstrand`, which defines an instance of height 3 since that refers to a full execution of either role in the protocol. A `defstrand` S-expression may

optionally include *maplets* that specify values to be used to instantiate variables in the role specification. A maplet is a parentheses-delimited pair where the first element is the name of the role variable to be instantiated and the second is the value, which can be any term formed over the variables declared in the `vars` portion of the `defskeleton`.

A `defskeleton` will usually have one or more *declarations* in it that restrict the class of executions the tool is to explore. Here, each example includes two declarations: one `non-orig` declaration and one `uniq-orig` declaration. Declarations must be made about expressions that can be parsed given the variables in the `defskeleton`; it is because of these declarations that we specify an instantiation of certain variables in a `defstrand`.

A `non-orig` declaration specifies a value (usually a symmetric or private key) as secret and never sent by honest parties in any potentially decryptable form. A `uniq-orig` declaration specifies a value as being randomly and freshly chosen where it first occurs in a transmission. Here, the initiator point of view specifies two assumptions: that the initiator picks her own nonce properly (i.e. randomly), and that the initiator's intended communication partner has an uncompromised private key. Similarly, the responder's point of view assumes that the the responder picks his own nonce properly and that his intended partner has an uncompromised private key.

3.3 CPSA output

When we run the CPSA tool on the Needham-Schroeder input file, and then run the `cpsa4graph` graphing tool on the result, we obtain a .xhtml file that can be viewed in a web browser. The `ns.xhtml` file in the examples directory contains these results.

The graphing output contains some top matter that includes the herald from the input file. Below this is a list of trees, each of which represents the analysis of one of the input `defskeleton`s; in the case of our example, there are two trees.

The rest of the graph output consists of the search results. The numbers in the list of trees link to the start of each tree. Each search result starts with an identification ("Tree 0" in the example), followed by a graph of the search, then the `defprotocol` used in that search, and then the skeletons considered by CPSA during its analysis. Figure 3.5 illustrates the list of trees, the tree identification, and that tree's search graph.

The search tree diagrams the steps in the search. Each skeleton in the entire graph file has a label, a number starting from 0. The number associated with a tree is the label number of the input skeleton. The left edge of the search graph is the root of the tree: in the case of Figure 3.5, the graph does not look very tree-like because the analysis doesn't branch. The numbers in the graph are the labels of the skeletons considered by CPSA during its analysis, and clicking on a number will direct the browser to display the corresponding skeleton.

The process by which CPSA analyzes a skeleton is the repeated use of an operation called the *cohort*, which takes an input skeleton and produces a set of more refined skeletons that

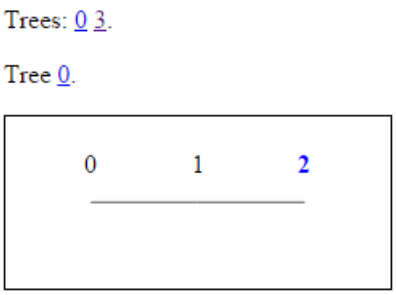


Figure 3.5: Search tree diagram for the initiator point of view in `ns.xhtml`

cover all the possible executions the input skeleton covered. The relationship between a parent skeleton and a child is that a child is included in the cohort calculated with the parent as an input.

Numbers are normally displayed in black, but may also be displayed in other colors. Blue numbers represent *realized* skeletons, that is, skeletons that may represent an actual execution.[1] Red numbers represent *dead* skeletons, that is, skeletons that represent partial executions that are not part of any actual execution – in other words, impossible scenarios.

Numbers may occur in the tree more than once, because it is possible that CPSA will discover a particular skeleton through more than one branch of the analysis. Green, italicized numbers represent skeletons present in more than one branch that are not dead skeletons, while orange italicized numbers represent dead skeletons present in more than one branch.

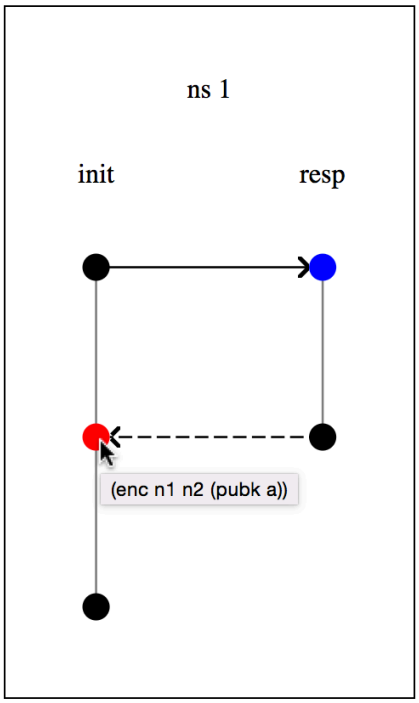
Each skeleton starts with a line that indicates its label (`item`) and the labels of its parent ((`parent`), if any) and its children ((`child`), if any) See Fig. 3.6) The parent and child numbers link to those skeletons, while the "item" number links back to the tree this skeleton is part of.

Below the diagram is a `defskeleton` that fully describes the skeleton. This text is fully compatible with CPSA input and can be used as a skeleton input for analysis with this protocol, although some of the fields in it are added by CPSA and would be ignored during input, for instance, the `label` and `parent` fields.

The diagram shows the skeleton as a graph. Strands are columns, ordered from top to bottom. The nodes in the graph are events, normally transmissions or receptions of messages. Nodes may be blue, red, or black; a black node represents a transmission, while blue and red nodes represent receptions. A blue node represents an explainable reception while a red node represents an unexplainable one. The left-most strands in a skeleton are normally the strands from the input `defskeleton`.

[1]Note that while realized skeletons already represent complete executions, CPSAdoes further analysis once a realized skeleton is reached in order to *generalize* that skeleton as much as possible. A skeleton that is both realized and cannot be further generalized is a *shape*. See page 33 for more detail on generalization.

Item 1, Parent: 0, Child: 2.




```
(defskeleton ns
  (vars (n1 n2 n2-0 text) (a b name))
  (defstrand init 3 (n1 n1) (n2 n2) (a a) (b b))
  (defstrand resp 2 (n2 n2-0) (n1 n1) (b b) (a a))
  (precedes ((0 0) (1 0)) ((1 1) (0 1)))
  (non-orig (privk a) (privk b))
  (uniq-orig n1)
  (operation nonce-test (added-strand resp 2) n1 (0 1)
    (enc n1 a (pubk b)))
  (label 1)
  (parent 0)
  (unrealized (0 1))
  (comment "1 in cohort - 1 not yet seen"))
```

Figure 3.6: Skeleton from the initiator point of view in `ns.xhtml`

The user may hover their mouse cursor over any node and will see a display of the S-expression describing the message at that event (see Figure 3.6). Here, if we hover over the red node (as shown) we will see that this is an event where the initiator receives the message $\{|n1, n2|\}_{K_a}$. This occurs after two transmissions: the first event in the init strand and the second in the resp strand. Those two transmissions are $\{|n1, a|\}_{K_b}$ and $\{|n1, n2_0|\}_{K_a}$. Neither transmission is the expected message, but sometimes a reception can be explained even if no regular node sends the exact message. Here, it is a question of what the adversary can build given the messages available. In this skeleton there are `non-orig` or `uniq-orig` assumptions about $n1, SK_a$, and SK_b, so since both messages are encrypted under keys for which we have a secrecy assumption on the decryption key, the adversary is unable to decrypt them. The adversary is also unable to build the required message: although the adversary is allowed access to $n2$ and K_a (since there are no restrictions on those), the adversary does not have access to $n1$. Hence, this node is unexplainable.

Arrows in the diagram represent basic orderings in the skeleton; arrows go from earlier events to later events. An arrow is solid when it goes from an event transmitting a message to one receiving the same message. So the blue node in this example is obviously explained because the exact message was transmitted by the initiator, specifically, at the first node in the initiator strand. The arrow ending at the red node is dashed because the messages do not agree, but it still represents that in this skeleton the second node of the responder strand occurs before the second node of the initiator strand.

3.4 Interpreting shapes

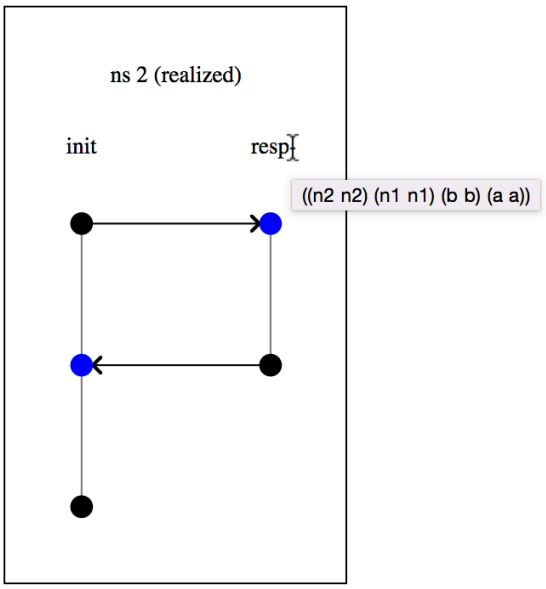
Next, we turn our attention to Figure 3.7, which is the one shape found by CPSA during the search on the initiator point of view.

Before we get into detail on what is contained in this skeleton, note the graph. All the arrows are solid, and there are arrows everywhere we expect them to be. This describes a message being sent by an initiator and received, unaltered, by a responder, who then sends a message that is received, again unaltered, by the same initiator, who then sends a message.

The user can hover their mouse cursor over the name of the role at the top of a strand in the skeleton diagram to see the variable assignment used in that instance. Here, hovering over both the instances indicates that they are in agreement about the values of $n1, n2, a$, and b: that is, the initiator's internal value for each variable is the same as the responder's internal value for the variable of the same name.

It may seem slightly odd that the initiator sends a message in its third node that is not received by anyone, but we know that in general it need not be received. The adversary completely controls the network, so it does not have to deliver that message.

Item 2, Parent: 1.




```
(defskeleton ns
  (vars (n1 n2 text) (a b name))
  (defstrand init 3 (n1 n1) (n2 n2) (a a) (b b))
  (defstrand resp 2 (n2 n2) (n1 n1) (b b) (a a))
  (precedes ((0 0) (1 0)) ((1 1) (0 1)))
  (non-orig (privk a) (privk b))
  (uniq-orig n1)
  (operation nonce-test (contracted (n2-0 n2)) n1 (0 1)
    (enc n1 n2 (pubk a)) (enc n1 a (pubk b)))
  (label 2)
  (parent 1)
  (unrealized)
  (shape)
  (maps ((0) ((a a) (b b) (n1 n1) (n2 n2))))
  (origs (n1 (0 0))))
```

Figure 3.7: The only shape from the initiator point of view (`ns.xhtml`)

The responder's point of view. The shape found during the search on the *responder's* point of view, however, includes something unusual. See Figure 3.8.

The graph of this shape should look less like expected behavior. Two things look odd, even at first glance. Most noticeable is the dashed arrow from the third initiator node to the third responder node. Also, there's the fact that there is a blue node (the one in the top left) that does not have any arrow coming in.

Inspection of the instances in this shape reveals that the initiator and responder agree on all the values except for b. This explains the dashed arrow: the initiator sends $\{|n2|\}_{K_{b_0}}$ but the responder receives $\{|n2|\}_{K_b}$; these messages are not the same, which is why the arrow is not solid. As for how the responder could receive the proper value, note that we only assumed SK_a and SK_b are secure, but we did not assume SK_{b_0} was secure. It would have been hard to do so, since $b0$ is a value we know nothing about from the initiator's point of view. The initiator's transmission of $\{|n2|\}_{K_{b_0}}$ thus does not protect $n2$ from decryption, so an attacker could have created the responder's received message by encrypting $n2$ under K_b.

The lack of an incoming arrow for the responder's first node can be explained because of the lack of any assumption about the value $n1$. The value $n1$ is the initiator's nonce, but this analysis does not assume that an initiator, if present, will choose their nonce properly. So $n1$ could actually be a value already chosen by the adversary, and the message $\{|n1, a|\}_{K_b}$ can be constructed and delivered to the responder before the initiator even starts.

The fact that the initiator and responder do not agree on b is an interesting feature of this protocol. We know from the initiator's point of view that the initiator a, when they have completed their execution, can infer that b has taken part in the responder role with a, and that they agree on both nonces.

The responder's point of view leads to less information. The responder b knows that a has taken part in the initiator role, but does not know that a intended to initate communication with b. This lets us conclude that the Needham-Schroeder protocol provides less than an ideal level of authentication.

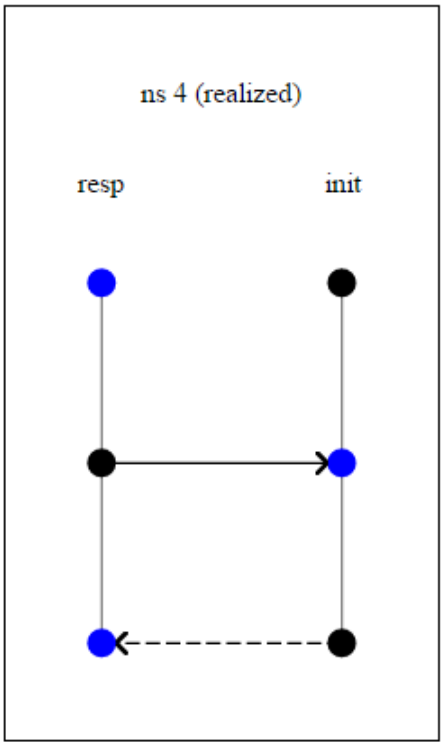
Exploration 3.1. The attack described above was first identified by Gavin Lowe [10], who also proposed a fix, namely, to have the second message in the Needham-Schroeder protocol include the name b of the responder.

Make a copy of the Needham-Schroeder input file and modify the protocol so that the second message (in both roles) includes a b. Run the analysis again and graph it. You should observe that the disagreement on b from the responder's point of view is no longer possible, and that the initiator's point of view is still good.

3.5 Blanchet's simple example protocol

Next we turn our attention to a second protocol, which will help build the reader's experience with CPSA and also introduce some additional features. This protocol is due to

Item 4. Parent: 3.




```
(defskeleton ns
  (vars (n2 n1 text) (a b b-0 name))
  (defstrand resp 3 (n2 n2) (n1 n1) (b b) (a a))
  (defstrand init 3 (n1 n1) (n2 n2) (a a) (b b-0))
  (precedes ((0 1) (1 1)) ((1 2) (0 2)))
  (non-orig (privk a) (privk b))
  (uniq-orig n2)
  (operation nonce-test (added-strand init 3) n2 (0 2)
    (enc n1 n2 (pubk a)))
  (label 4)
  (parent 3)
  (unrealized)
  (shape)
  (maps ((0) ((a a) (b b) (n2 n2) (n1 n1))))
  (origs (n2 (0 1))))
```

Figure 3.8: The only shape in the responder point of view search in `ns.xhtml`

```
(defprotocol blanchet basic
  (defrole init
    (vars (a b akey) (s skey) (d data))
    (trace (send (enc (enc s (invk a)) b))
           (recv (enc d s)))
    (uniq-orig s))
  (defrole resp
    (vars (a b akey) (s skey) (d data))
    (trace (recv (enc (enc s (invk a)) b))
           (send (enc d s)))
    (uniq-orig d)))
```

Figure 3.9: The Blanchet simple example protocol

Bruno Blanchet, and has a flaw introduced by design for the purpose of discussing protocol analysis. In this protocol there are again two participants: an initiator and a responder. However, in this protocol, we do not use names, just public keys. Specifically, one party has a public signing key (a), while the other has a public encryption key (b).

The protocol is as follows, informally:

- The initiator chooses a fresh, random session key s, signs it with their private signing key (corresponding to the public key a), and encrypts it with the responder's public key b and sends the result to the responder.
- The responder receives and decrypts such a message, confirms the signature, and then encrypts a piece of data d under s and sends this back to the initiator.

The file `blanchet.scm` in the examples directory contains Blanchet's simple example protocol described above. See Figure 3.9 for the protocol declaration.

There are several elements of this protocol input that are new. First of all, the Needham-Schroeder protocol used only two types: `name` and `text`, while this protocol uses three new types. The `data` type is for simple values, much like `text`. In fact, the two types are interchangeable, but both are available for cases where an analyst may wish to describe a protocol in which two types of simple values exist that cannot be confused for each other.

The `akey` and `skey` types are for keys, specifically, asymmetric and symmetric keys, respectively. The `invk` function symbol maps an asymmetric key to its inverse.

Note that we use `(enc s (invk a))` to represent the digital signature. A digital signature in the CPSA message algebra is represented as an encryption under the signature key.

A third feature is the presence of a delcaration such as `(uniq-orig s)` within a `defrole`. Like the `uniq-orig` declaration that can appear in a skeleton, this declaration indicates that the value contained inside is freshly chosen. When this declaration

```
(defskeleton blanchet
  (vars (a b akey) (s skey) (d data))
  (defstrand init 2 (a a) (b b) (s s) (d d))
  (deflistener d)
  (non-orig (invk b)))

(defskeleton blanchet
  (vars (a b akey) (s skey) (d data))
  (defstrand resp 2 (a a) (b b) (s s) (d d))
  (deflistener d)
  (non-orig (invk a) (invk b)))
```

Figure 3.10: Blanchet points of view

appears in the role, however, the assumption is that the value is freshly chosen by *every* honest participant in the protocol who plays that role. Declarations present in a role are inherited by every skeleton with an instance of that role. See Chapter 6 for more on the declarations supported by CPSA.

Exploration 3.2. Make a variant of the Needham-Schroeder protocol in which the freshness of each party's nonce is assumed via a `uniq-orig` declaration in the protocol role. Run the analysis from each participant's point of view. What differs in the shapes, and why? (There should be one difference that's noticeable in the graph of one of the shapes.)

Exploration 3.3. Make a variant of the Needham-Schroeder protocol in which the secrecy of each party's partner's private key is assumed via a `non-orig` declaration in the protocol role. Run the analysis from each participant's point of view.

You should observe that the authentication failure is no longer present in the shape from the analysis of the responder's viewpoint. What conclusion can you draw about this declaration? Did we fix the protocol? If not, why does the analysis seem to contain no flaws?

The `blanchet.scm` file contains four inputs to the analysis. The first and second are just the points of view of each participant, under typical assumptions. But the third and fourth contain another new element. See Figure 3.10 for these two inputs.

These two inputs are prepared to ask a confidentiality question: specifically, is the value d exposed to the adversary? A *listener* is a pseudo-role that is considered part of all protocols by CPSA. That role consists of receiving some arbitrary message and then sending that same message. Listeners can show up in CPSA analyses, in order to handle a case where a certain value is learnable, so that the rest of the case breakdown can assume that value is not learnable.

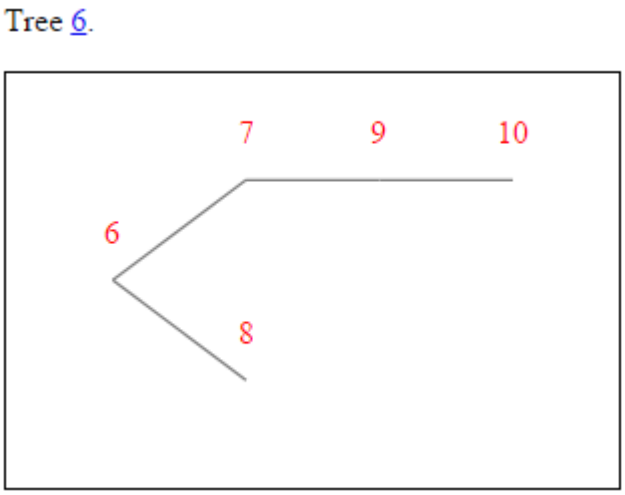


Figure 3.11: The search tree for the privacy of d in the initiator's point of view in `blanchet.xhtml`

In this protocol, the secret data d remains private in the initiator's point of view. Figure 3.11 shows the search tree for the point of view in which the initiator completes the protocol and in which there is a listener for the same d value the initiator hears. The fact that all the numbers are red here indicates that all the skeletons in the search are "dead", meaning that they are inconsistent with any actual executions. In other words, there are no executions in which d is revealed given our assumption that the private decryption key of b is not compromised.

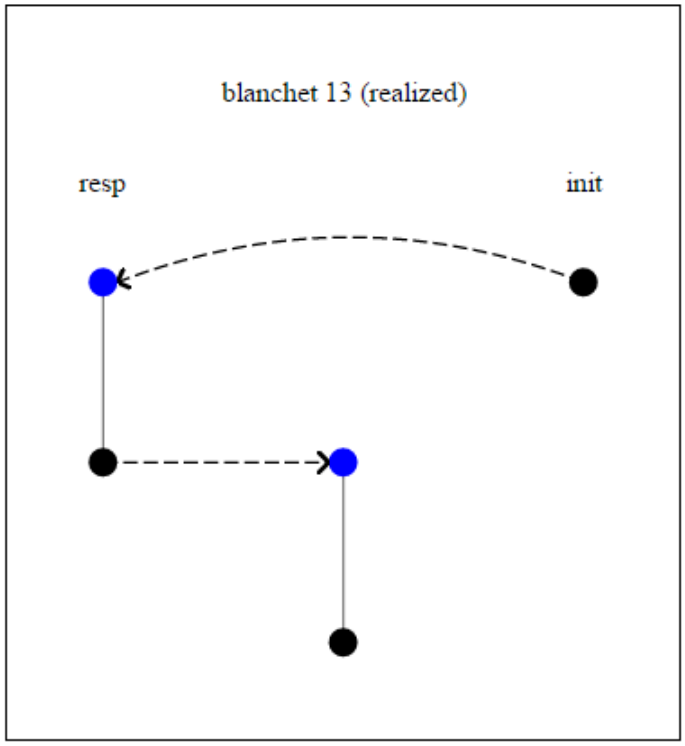
However, d does not remain private in the responder's point of view. Figure 3.12 shows the graph of a shape for the point of view in which the responder completes the protocol and in which there is a listener for the d value the responder sends. Note that although d is encrypted under s, and s is freshly chosen by an initiator, the shape shows that s can leak. We are not guaranteed that the initiator and responder agree on b. Therefore, the initiator may have sent s encrypted with b_0, and since the private key corresponding to b_0 is not necessarily secret, s may leak.

The `blanchet.scm` file also contains a second protocol with the name `blanchet-corrected` in which the flaw that allows d to be learned in the responder's point of view is eliminated.

Exploration 3.4. Modify the Blanchet protocol to add a role that is identical to the initiator role, except that s is not declared `uniq-orig`. What impact does this have on the analyses?

Exploration 3.5. If you modify the Blanchet protocol to instead add a role that is identical to the responder role, except that d is not declared `uniq-orig`, what impact do you believe this will have on the analyses? Make a prediction, then check your prediction.

Item 13, Parent: 12.




```
(defskeleton blanchet
  (vars (d data) (s skey) (a b b-0 akey))
  (defstrand resp 2 (d d) (s s) (a a) (b b))
  (deflistener d)
  (defstrand init 1 (s s) (a a) (b b-0))
  (precedes ((0 1) (1 0)) ((2 0) (0 0)))
  (non-orig (invk a) (invk b))
  (uniq-orig d s)
  (operation encryption-test (added-strand init 1) (enc s (invk a))
    (0 0))
  (label 13)
  (parent 12)
  (unrealized)
  (shape)
  (maps ((0 1) ((a a) (b b) (s s) (d d))))
  (origs (s (2 0)) (d (0 1))))
```

Figure 3.12: The only shape in the analysis for the responder's point of view with a listener for d in `blanchet.xhtml`. The second strand, with no role name, is the listener.

Exploration 3.6. Try modifying the Blanchet protocol, removing the `uniq-orig` declarations from the two roles. Replace the `defskeleton`s in the input file with the point of view skeletons for the un-corrected version of the Blanchet protocol from `blanchet.xhtml`; these skeletons will explicitly include declarations that would be inherited but were lost due to the removal of the role declaration.

Chapter 4

Algebra Features of CPSA

The CPSA distribution comes equipped with two cryptographic alegbras, the `basic` cryptoalgebra and the `diffie-hellman` cryptoalgebra. The Diffie-Hellman algebra is a pure extension of the basic algebra, so a user may always use the Diffie-Hellman algebra to access all algebraic features. However, the performance of the tool is superior when using the basic algebra, so users are advised to choose the basic algebra whenever they are not making use of Diffie-Hellman features.

In Chapter 3, we introduced the basic cryptoalgebra, along with the `data, text, name, skey,` and `akey` sorts, and the `pubk, privk, invk, enc,` and `cat` function symbols. In addition to these, the basic cryptoalgebra contains the sorts `tag` and `mesg`, and the `ltk` and `hash` function symbols, and string constants.

The Diffie-Hellman cryptoalgebra introduces two further sorts, `rndx` and `expt`, and six new function symbols, `bltk, exp, rec, mul, gen,` and `one`.

At one time variables of sort `base` were also permitted, but a variable `b` of sort `base` has the same meaning as a term `(exp (gen) alpha)` where `alpha` is of sort `expt`. Eliminating variables of sort `base` allows us to eliminate some tricky but conceptually unnecessary, even practically unnecessary, code. Thus, replace these variables with `(exp (gen) alpha)`, where `alpha` is a variable of sort `expt`. We still refer to these *terms* as having sort `base`, since they may be the bases for further exponentiation, but there are no variables of this sort.

In this chapter we will explain these additional features with examples. In Section 4.1, we will discuss the `ltk` function symbol and the use of the `mesg` sort, worked with an example based on the Kerberos protocol. In Section 4.2, we will discuss the tool's Diffie-Hellman features. In Section 4.3, we will discuss the remaining features, and note examples that demonstrate their use. For a more complete reference about the two algebras, see Section 10.3.

4.1 Generic messages and long-term keys

Securing communications purely with symmetric keys faces an inherent scaling problem: when there are n parties that may wish to communicate, there must be $O(n^2)$ keys shared between parties, which gets to be too many in any system with a large number of users.

The Kerberos protocol is a well-known protocol for the distribution of symmetric keys. Instead of having the n parties share keys with each other party, the users share a key only with a central key server (also known as a key distribution center). The key server controls key distribution within a *realm* that can be thought of as the set of users that share keys with the key server.

Suppose a user wishes to communicate securely with another user in the same realm. The first user would contact the key server and request a session key for communication with the other user. The key server could then encrypt the session key twice, once under each user's shared key with the server. One encrypted key is sent back to the user that requested the channel, and the other (called the "ticket") is also sent to the requesting user, to be forwarded on to their communication partner.

We will focus on a flawed protocol similar to the initialization protocol used by Kerberos. The protocol is described informally as follows. There are three parties, the initiator a, the responder b, and the key server s.

- a sends a message (a, b, n) to the key server, where n is a freshly chosen nonce.
- The key server, on a message (a, b, n), picks a fresh, random session key k, and sends two encryptions to a: $\{|k, n|\}_{SK(a,s)}$ and $\{|k, a, b|\}_{SK(b,s)}$. Here, $SK(x, s)$ refers to the shared secret between the key server and x.
- a receives these two messages, decrypts the first to learn k and to check that the proper nonce n was included, then sends a message m on to the responder, encrypted under k, along with the second message (the ticket).
- b receives two encrypted messages, decrypts the second to learn k, and decrypts the first with k to learn the message.

The `ltk` function symbol is used in CPSA to represent long-term shared keys between two specific parties, when communication of that key is assumed out of scope of the analysis. See Figure 4.1 for a `defprotocol` that attempts to describe this flawed version of the Kerberos initialization protocol. See `kerb.scm` in the examples directory for the input file discussed here.

The actual Kerberos protocol contains several additional elements that we omit for simplicity, but the key difference is that the message encrypted under $SK(a, s)$ should include b. The message from the initiator that requests a session key is transmitted in the clear and can be tampered with, so the initiator needs assurance that the session key k is

```
(defprotocol kerb-flawed basic
  (defrole init
    (vars (a b s name) (m n text) (k skey))
    (trace (send (cat a b n))
           (recv (cat (enc k n (ltk a s)) (enc k a b (ltk b s))))
           (send (cat (enc m k) (enc k a b (ltk b s)))))
    (uniq-orig n))
  (defrole keyserv
    (vars (a b s name) (n text) (k skey))
    (trace (recv (cat a b n))
           (send (cat (enc k n (ltk a s)) (enc k a b (ltk b s)))))
    (uniq-orig k))
  (defrole resp
    (vars (a b s name) (m n text) (k skey))
    (trace (recv (cat (enc m k) (enc k a b (ltk b s)))))))
```

Figure 4.1: A flawed version of Kerberos

```
(defskeleton kerb-flawed
  (vars (a b s name) (m text))
  (defstrandmax init (a a) (b b) (s s) (m m))
  (deflistener m)
  (non-orig (ltk a s) (ltk b s))
  (uniq-orig m))
```

Figure 4.2: Flawed Kerberos point of view

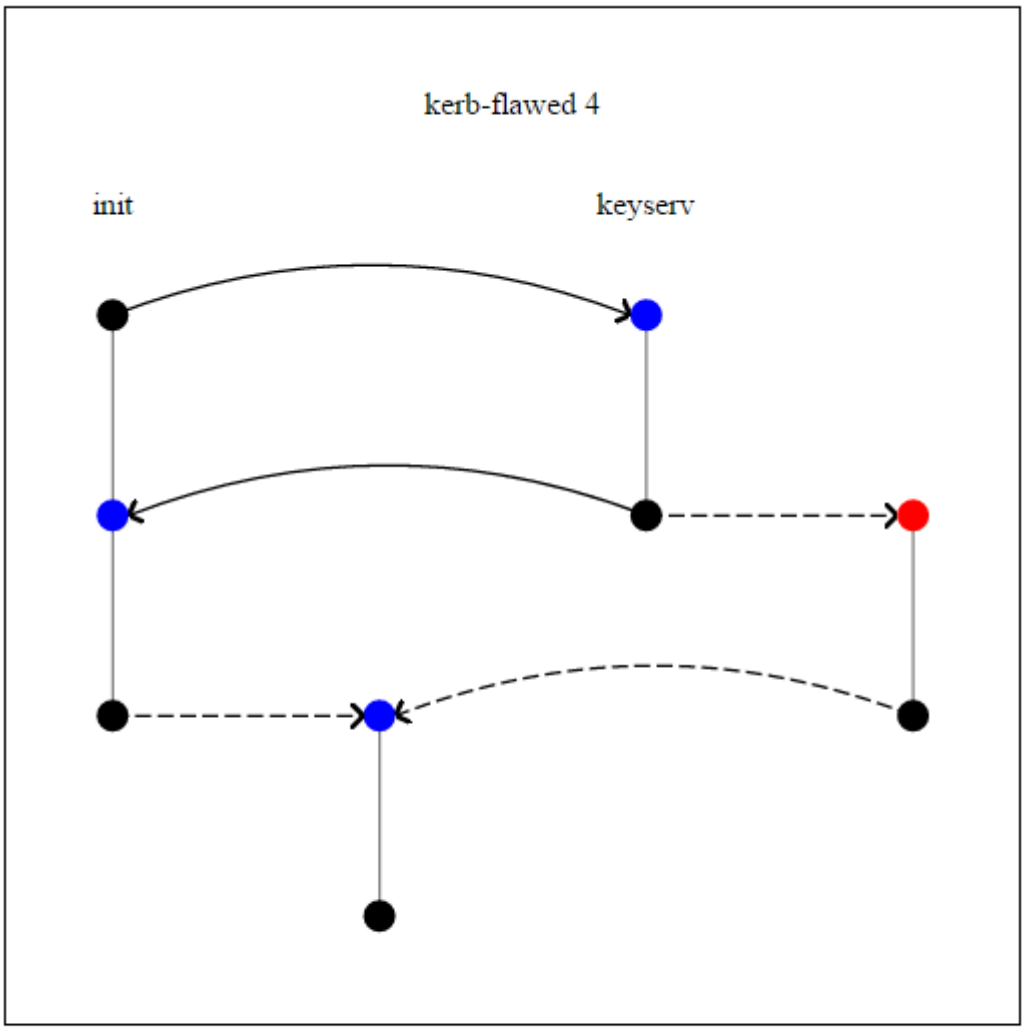
being exposed only to the initiator and b. Otherwise, even assuming the long-term keys of both a and b with the key server are secure, a third party may observe the message m: all the adversary has to do is block the initial message and substitute b with another name c.

Will CPSA find this attack? See `kerb.xhtml` for the results of the analysis for the point-of-view skeleton described in Figure 4.2. Note the use of `defstrandmax`; this is a shortcut you can use in CPSA to specify a full-height instance of a role.

Interestingly, the result of the analysis does not indicate any attack is possible! The analysis gets stuck at a skeleton (see Figure 4.3) where the key server does in fact generate a session key for a and b, and where that session key is exposed, but it cannot be exposed if it was generated for a and b and if we are assuming both of their long-term keys are private.

The reason CPSA gets the wrong result here is that we have inadvertently modeled our protocol question incorrectly. Specifically, we have described a version of the protocol where the initiator is able to check the validity of the ticket. The attack we have in mind

Item 4, Parent: 3.




```
(defskeleton kerb-flawed
  (vars (m n text) (a s b name) (k skey))
  (defstrand init 3 (m m) (n n) (a a) (b b) (s s) (k k))
  (deflistener m)
  (defstrand keyserv 2 (n n) (a a) (b b) (s s) (k k))
  (deflistener k)
```

Figure 4.3: Flawed Kerberos dead skeleton. The use of the `mesg` sort provides a fix to this badly modeled version of the protocol.

```
(defprotocol kerb-flawed2 basic
  (defrole init
    (vars (a b s name) (m n text) (ticket mesg) (k skey))
    (trace (send (cat a b n))
           (recv (cat (enc k n (ltk a s)) ticket))
           (send (cat (enc m k) ticket)))
    (uniq-orig n))
  (defrole keyserv
    (vars (a b s name) (n text) (k skey))
    (trace (recv (cat a b n))
           (send (cat (enc k n (ltk a s)) (enc k a b (ltk b s)))))
    (uniq-orig k))
  (defrole resp
    (vars (a b s name) (m n text) (k skey))
    (trace (recv (cat (enc m k) (enc k a b (ltk b s)))))))
```

Figure 4.4: Using a variable of sort `mesg` in a flawed version of Kerberos

is one where the two encrypted keys received by the initiator are prepared by a key server instance, but one which does not agree with the initiator on b. However, the ticket would not match what the initiator expects in our expression of the initiator role.[1]

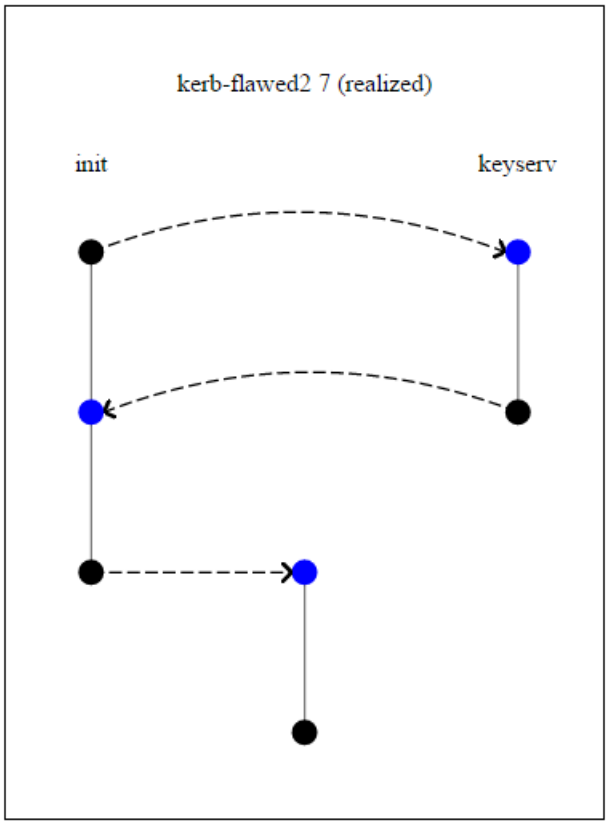
We described an initiator that will only proceed onto the third step (sending m) if the ticket is of the form we specified. In fact, the initiator cannot do this. The initiator cannot even verify that the ticket is encrypted under the correct key! The solution is to use a generic variable, one that can stand for any message, even one that a particular participant cannot parse or understand. The `mesg` sort is the sort of all possible messages, so a variable of the `mesg` sort can stand for any potential value at all. See Figure 4.4 for a version that models the initiator's reception properly. Note the `ticket` variable in the initiator role, which stands for the ticket value the initiator cannot inspect.

This version of the flawed Kerberos protocol may also be found in the file `kerb.scm`, and its analysis may be found in `kerb.xhtml`. This time, there is a shape found. See Figure 4.5.

Notes on generic variables. Variables of the `mesg` sort are constrained in CPSA, and may only be used when they are received before they are transmitted. So for instance the m variable in the initiator role of the flawed Kerberos protocol cannot be of the `mesg` sort, because it appears first in a transmission. However, if the responder is willing to accept an

[1] An alert reader may wonder how they could detect such an error of their own when using the tool. We will return to this example in Chapter 5, when we discuss how the CPSA search process works.

Item 7, Parent: 6.




```
(defskeleton kerb-flawed2
  (vars (ticket mesg) (m n text) (a b s b-0 name) (k skey))
  (defstrand init 3 (ticket ticket) (m m) (n n) (a a) (b b) (s s) (k k))
  (deflistener m)
  (defstrand keyserv 2 (n n) (a a) (b b-0) (s s) (k k))
  (precedes ((0 0) (2 0)) ((0 2) (1 0)) ((2 1) (0 1)))
  (non-orig (ltk a s) (ltk b s))
  (uniq-orig m n k)
  (operation encryption-test (added-strand keyserv 2)
    (enc k n (ltk a s)) (0 1))
  (label 7)
  (parent 6)
  (unrealized)
  (shape)
  (maps ((0 1) ((a a) (b b) (s s) (m m) (ticket ticket) (n n) (k k))))
  (origs (k (2 1)) (n (0 0)) (m (0 2))))
```

Figure 4.5: Flawed Kerberos shape. The use of the `mesg` variable allows us to find the attack illustrated here.

encryption of any message m, then m may declared as a variable of the `mesg` sort within that role.

Also, variables of the `mesg` sort should never be used as a key in any encryption. This is because CPSA uses a single function symbol to represent both symmetric and asymmetric encryption, and when the key is a variable of sort `mesg`, it is ambiguous which is meant.

Notes on long-term keys. Long-term keys are uni-directional: `(ltk a b)` and `(ltk b a)` are distinct values. In fact, CPSA believes that they cannot be the same unless $a = b$. This is fine for modeling protocols like Kerberos where there is a clear distinction between client and server behavior. Note that in our protocol example, all long-term keys were described in all roles with a client name in the first position and a server name in the second position. See Section 4.3.3 for discussion of the `bltk` function symbol, which models *bi-directional* long-term symmetric keys.

For another example of the use of long-term symmetric keys in a protocol, see `yahalom.scm` in the examples directory.

Exploration 4.1. Fix the flawed version of Kerberos so that the initiator can believe their transmission is private to them and b. Continue to use the ticket variable of sort `mesg`.

Construct a `defskeleton` for the responder's point of view for your fixed protocol, analyze it, and explain in English what authentication property this analysis implies.

Exploration 4.2. Construct a `defskeleton` for the initiator's point of view, without the listener for m, in the fixed version you created in Exercise 4.1. Run the tool on your `defskeleton`. Why are there dashed arrows in the result? Does this represent an insecurity in the protocol?

Whether or not you think this is an insecurity, think about how you would alter the protocol to avoid the dashed arrows, and try out your ideas.

The Otway-Rees protocol is another example of very similar modeling; see `or.scm` and `or.xhtml` if you wish to explore these issues further on your own.

4.2 Modeling Diffie-Hellman

In their seminal 1976 paper, "New Directions in Cryptography", Diffie and Hellman proposed the notion of public-key cryptography [1]. They did not have a method for public-key direct encryption, but they did have a key exchange protocol that has become a crucial building block in cryptographic protocols.

The Diffie-Hellman protocol works as follows. A large prime number p is chosen and agreed upon as a parameter, and g is chosen to be some integer modulo p. In order to enable secure communication between arbitrary parties, Diffie and Hellman imagined a directory of public values, like a phone book. Each person who wishes to be able to communicate securely with others will generate for themselves a private value x, and publicize $g^x \bmod p$

as their public value. If Alice's private value is a, and Bob's private value is b, then the shared secret between Alice and Bob would be $g^{ab} \bmod p$, which each party can calcuate from their own private value and their partner's public value. For instance, Alice can calculate $g^{ab} \equiv (g^b)^a \bmod p$.

Version 3 of CPSA introduces the Diffie-Hellman algebra, which allows for analysis of protocols that incorporate Diffie-Hellman techniques. The Diffie-Hellman algebra includes all the function symbols and sorts available in the basic algebra, plus two additional sorts of variable, namely the two sorts `rndx` and `expt` for exponents such as x. The Diffie-Hellman specific functions symbols are as follows:

- `exp` represents exponentation. For example, h^x is encoded as `(exp h x)`.
- `mul` represents multiplication of exponents. So if x is an exponent, `(mul x x)` would represent the square of x.
- `gen` represents the standard generator g. It is probably best to think of `gen` as a constant, i.e. a function symbol with arity 0.
- `one` represents the multiplicative identity for the group of exponents. Like `gen`, `one` is a 0-ary function.

  Important: `gen` and `one` are functions, so they must be enclosed in parentheses. So `(exp (gen) x)` represents g^x, while `(exp gen x)` would represent gen^x where gen is expected to be a variable. Similarly, `(mul (one) x)` represents $1 \cdot x$ while `(mul one x)` would represent $one \cdot x$ where the tool expects one to be an exponent variable.
- `rec` represents the multiplicative inverse in the group of exponents. So for instance `(exp (exp (gen) x) (rec x)) = (exp (gen) (one)) = (gen)`.

At this time, CPSA does not model addition of exponents, although there are many examples of protocols that add or subtract exponents. There is no way to take a product of exponentiated values either (e.g. $g^x \cdot g^y$) since this would be equivalent to including addition of exponents.

Several examples of Diffie-Hellman protocols are available in the examples directory of the distribution. The `plaindh.scm` example models a simple, unauthenticated Diffie-Hellman exchange between two parties.

The initiator and responder perform a Diffie-Hellman exchange, followed by the initiator choosing a random nonce n and sending it, encrypted with the Diffie-Hellman key, to the responder, who decrypts n and sends it back.

The analysis result can be found in `plaindh.xhtml`. You can see there that a shape is found where an initiator exists but no responder; the n can be decrypted because g^{xx_0} can be calculated by the adversary when x_0 is not assumed secret.

```
(defprotocol plaindh diffie-hellman
  (defrole init
    (vars (x rndx) (y expt) (n text))
    (trace (send (exp (gen) x))
           (recv (exp (gen) y))
           (send (enc n (exp (gen) (mul y x))))
           (recv n))
    (uniq-orig n)
    (uniq-gen x))
 ...)
```

Figure 4.6: Diffie-Hellman defprotocol

Two features of the CPSA model of Diffie-Hellman in this protocol are worth drawing attention to. See Figure 4.6

Note that the initiator uses a variable x of the `rndx` sort to represent its own random variable, and a variable y of the `expt` sort to represent the exponent present in the base value `(exp (gen) y)` it receives from the initiator. Distinct values of the `rndx` sort model distinct independent random choices of exponents, while `expt` values merely represent arbitrary exponents which may or may not be calculated as some product of other known values. Here, since the initiator chooses their own exponent, we model x as an `rndx` value. But since the initiator cannot know how the base value g^y was calculated, we model y as an `expt` value.

Note that unlike the example in Section 4.1, where receiving a specifically formatted encryption in a protocol role implied the ability to decrypt and check that structure is present, the use of a value like g^y in a reception does not imply that y is *known*, only that y is defined to be the value such that g^y is the base value received.

Second, you may notice that n is declared `uniq-orig` while x is declared `uniq-gen`. The difference between these declarations is rather technical: see Section 6.2 for details. For the moment, it is sufficient to say that one should use `uniq-gen` for exponents that first occur within an exponentiation, rather than `uniq-orig`.

It is also instructive to examine the two final skeletons in the analysis that leads to the shape; see Figure 4.7. The first realized skeleton reached in the branch is skeleton 4, on the left, which includes an instance of the initiator role and a listener. But this realized skeleton has a child: skeleton 5. The difference is that skeleton 5 does not include the listener. This is an example of a step that CPSA takes called *generalization*: when CPSA recognizes a realized skeleton (that is, one that has no unexplainable receptions), it attempts to identify ways to make that skeleton more general without losing coverage. Here, CPSA has deleted the listener, because that actual explicit reception and re-transmission of g^{xx_0} is not strictly necessary. When a realized skeleton cannot be further generalized,

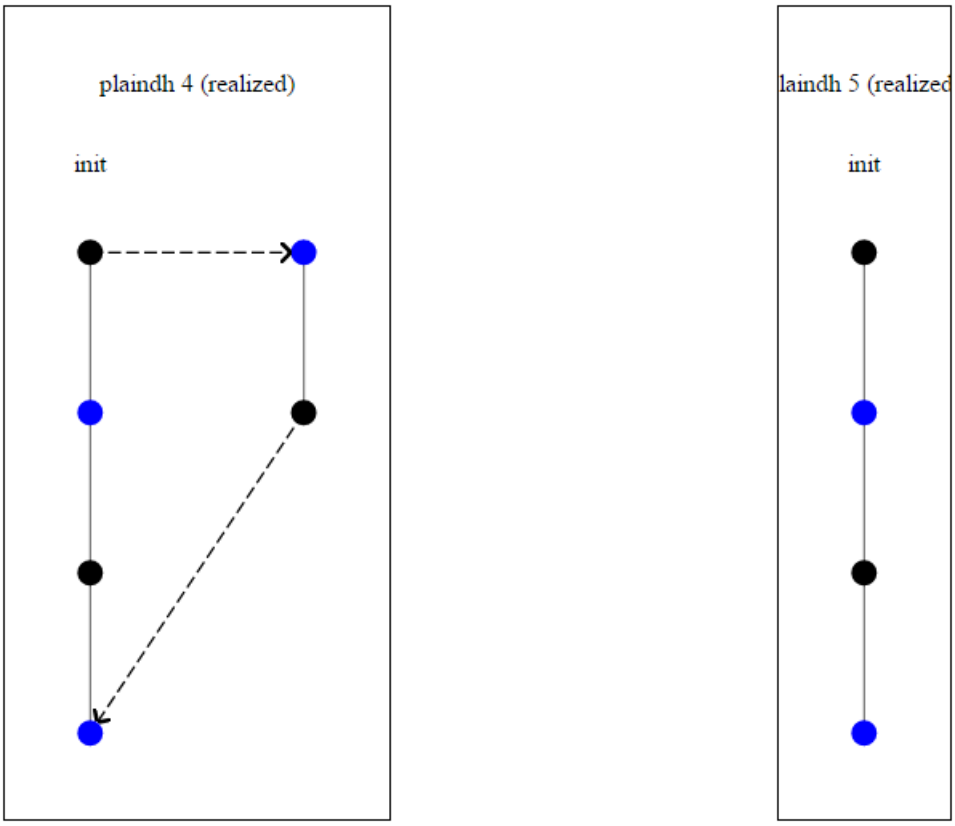


Figure 4.7: Deletion of strand

CPSA declares it a "shape" and stops working in that branch of the analysis.

You may notice there is another shape in `plaindh.xhtml`, in which a responder is present. As it happens, the other shape is strictly less general than the shape shown as skeleton 5. CPSA does not promise that the set of shapes it outputs is strictly minimal, and this is one example where the output of CPSA is not minimal.

4.2.1 Other examples

The examples directory also contains the `station.scm` and `station.xhtml` examples that model the station-to-station protocol. This protocol uses digital signatures in order to authenticate a Diffie-Hellman exchange so that the key established represents a secure and authenticated channel. Two versions of the protocol are provided; in one, the Diffie-Hellman exponents are assumed fresh in the roles, while in the other, they are not. The two analyses produce different results.

Another example provided can be found in `iadh-um.scm` and `iadh-um.xhtml`. These

inputs concern a method of determining Diffie-Hellman session keys using both long-term and "ephemeral" exponents called the *unified method*. The "ia" in the example name stands for *implicitly authenticated*, because this method of session key creation allows the parties to be sure that no other party knows the key. One interesting feature of this input is that it contains an example of an exponent being transmitted outside of an exponentiation. Specifically, there is a role in which a party generates and signs their long-term Diffie-Hellman public value, and then compromises it by releasing the key. The explicit compromise allows us to test the "forward secrecy" property of the unified method.

4.3 Other Algebra Features

4.3.1 Hashing

The CPSA basic cryptoalgebra includes a `hash` function symbol that can be used to represent the use of a hash function. The function takes a single input, but `(hash t1 t2 ... tn)` is interpreted as shorthand for `(hash (cat t1 (cat t2 ( ...tn) ...)))`.

Exploration 4.3. The Needham-Schroeder protocol discussed in Chapter 3 was described as for key agreement, but no session key is apparent in the protocol. The intention is to use the hash of the two nonces as the key. Make a copy of the `ns.scm` example in which this key is explicitly used to encrypt and transmit a separate fresh value, and make a point of view testing the confidentiality of the plaintext.

4.3.2 Constants

Numerous cryptographic protocols make use of magic numbers or string constants to disambiguate the purpose of various messages that occur during the protocol. CPSA includes constant strings in the basic cryptoalgebra. Such constants always appear as quoted strings, e.g. `"foo"`. These strings do not need to be declared in a `vars` statement because they are not variables. Tags are messages, so that a variable of `mesg` sort may be instantiated by a tag.

Exploration 4.4. Note that in the Needham-Schroeder protocol, there seems to be no way for an initiator to accidentally talk to another initiator, even though the initiator both sends and receives an encrypted message with two components in it. Make a copy of the Needham-Schroeder protocol in which the nonces are declared to be of sort `name` instead of `text`, so that $\{|n_1, a|\}_{K_b}$ and $\{|n_1, n_2|\}_{K_a}$ are modeled as similar enough in format that confusing the two is possible. What changes?

Then try adding distinct tag constants to each encrypted message, to avoid the ambiguity we just created. Run the analysis to see the effect.

Constants may also be used to modify the `pubk` and `privk` function symbols, to describe distinct keys associated with a particular name. For instance, one might use `(pubk`

`"encrypt" a)` to describe the public encryption key of a, so as to distinguish it from the public signature verification key of a. Or, one might wish to describe a certifying authority's certificate-signing key but also a key used to sign certificate revocation statements that might be different.

4.3.3 Bidirectional Long-Term Keys

The `ltk` function symbol is used to describe the long-term secret key shared between two parties. However, the names of the two parties are, in the function symbol, presented in a distinguished order.

In other words, CPSA regards `(ltk a b)` and `(ltk b a)` as distinct from each other; in fact, they are only considered equal when $a = b$.

The `bltk` function symbol is available in CPSA in *the Diffie-Hellman algebra only*,[2] and regards the two names as equivalent in order. In the Kerberos example discussed in Section 4.1, we noted that the server's name s always appears second in our `ltk` expressions. This is fine if participants never can act as both client and server, but if a participant can act as both, the use of `ltk` implies that the participant maintains a strict separation between the keys they share with other servers when acting as a client (in which their own name appears first), and keys they share with clients when acting as a server (in which their own name appears second).

The use of `bltk` implies that participants can act as both servers and clients, and that they only share one key with other entities, and use that key both when acting as a server and when acting as a client.

See `bltk_or.scm` and `bltk_or.xhtml` for modeling of the Otway-Rees protocol with bi-directional long-term keys rather than uni-directional ones.

Exploration 4.5. Make a version of the flawed Kerberos input file `kerb.scm` that uses bi-directional long-term keys instead. Remember to switch the algebra to Diffie-Hellman! What differences do you observe?

Exploration 4.6. Repeat the Exercise 4.5 but with the Yahalom protocol (`yahalom.scm`) instead.

4.4 The Lang Field: Declaring new operators

The signature of the message algebra used by CPSA is extensible. There are two forms of extensibility provided by CPSA. Users can add operations for encryption, tupling, and hashing; and add sorts for atomic data.

[2] This choice may seem odd; it was made for performance reasons. The presence of `bltk` or of Diffie-Hellman elements complicates some basic algebraic operations. The `basic` cryptoalgebra is provided for optimized performance when analyzing protocols that do not include these features.

LANG	←	(`lang` LDECL∗)
LDECL	←	(SYMBOL+ KIND)
KIND	←	`atom` | `akey` | `hash` | (`tuple` INTEGER)
	|	`enc` | `senc` | `aenc` | `sign`

Table 4.1: Lang Field Syntax

A common extension is to add tupling operations to message algebras. Previously, complex protocols often made use of tagged concatenation to encode a tagged sequence of messages. In CPSA, concatenation is implemented as sequences of pairing operations. Thus, the expression for a certificate body of the form

```
(cat "cert-body" dn serial-no pub-key)
```

is syntax for

```
(cat "cert-body" (cat dn (cat serial-no pub-key)))
```

Algebra extensions are declared using the `lang` key in the protocol's association list, at the end of a protocol. For the certificate body example,

```
(defprotocol cert basic
  ...
  (lang (cert-body (tuple 3))))
```

adds one tupling operation so that the example above can be written as

```
(cert-body dn serial-no pub-key)
```

CPSA represents this form internally as a sequence of messages with a distinguished mark. This representation is more efficient as compared with the tagged concatenation representation.

The syntax of a `lang` declaration is given in Table 4.1. There are eight kinds of ways that CPSA algebras can be extended.

- An atomic sort is added to the algebra when a symbol is declared to be of kind `atom`. If `dollar` is declared to be an atomic sort, then `(price dollar)` in a `var` form declares `price` to be of sort `dollar`.
- An atomic sort is also added to the algebra when a symbol is declared to be of kind `akey`. However, in addition to adding the sort, CPSA adds the equation $(x^{-1})^{-1} = x$ for variables of the new sort.

- A unary operation is added to the algebra when a symbol is declared to be of kind `hash`. As with the default hash operation, the adversary cannot extract the contents of the hash.
- An n-ary operation is added to the algebra when a symbol is declared to be of kind (`tuple` n). As with concatenation, the adversary can construct and extract contents of tuples. Constraint: $n > 0$.
- A binary operation is added to the algebra when a symbol is declared to be of kind `enc`. As with the default encryption operation, the adversary cannot extract contents of the encryption without access to the inverse of the key.
- A symbol of kind `senc` is just like one of kind `enc` except that the key supplied when it is applied must be symmetric, that is, the sort of the key must not be asymmetric.
- A symbol of kind `aenc` is just like one of kind `enc` except that the key supplied when it is applied must be asymmetric, but not the inverse applied to an asymmetric key.
- A symbol of kind `sign` is just like one of kind `enc` except that the key supplied when it is applied must be the inverse applied to an asymmetric key.

Part II

Understanding and Guiding CPSA

Chapter 5

The CPSA Search Algorithm

The result of running the `cpsa4shape` tool is a file that contains only "shapes", which include all essential structures possible in executions of the protocol under the conditions input to the analysis. Although this output contains the most important elements of the results of an analysis, it can be useful to an analyst to examine the full result.

Consider analyzing a protocol that has a secrecy property you wish to guarantee. After modeling the protocol, you model a set of conditions under which you expect the secrecy property to hold, but you include a listener that would invalidate it. See Section 3.5 for an instance of the use of this technique.

Suppose the analysis confirms the secrecy property. This would mean that there are no shapes at all, because any shape would represent a possible way in which the conditions were present but the secret is revealed. If you look only at the shapes file, you will see nothing other than the fact that no shapes are present. The full analysis, however, can be used to understand the space of attacks that were explored, which gives a much clearer sense of *why* the analysis resulted in no shapes.

It is a complicated and somewhat unnatural process modeling a protocol and setting up the conditions for an analysis. Human error is possible, and when a human error occurs, it is difficult for the analyst—most likely, the human that made the error in the first place—to distinguish a human error from a correct analysis result.

By examining the full analysis carefully, the analyst may discover errors of two types: segments of the analysis that seem inconsistent with the intended scenario, or the *lack* of analysis that the user expected to be present. The first circumstance indicates an analysis that was *under-constrained*, while the latter indicates one that was *over-constrained*. In order to detect errors of the second type, however, the analyst needs to understand the search algorithm.

Another reason to understand the search algorithm is when the user has obtained an analysis that describes a genuine attack against a protocol. By stepping through the path in the analysis that led to the shape in question, a user may gain an understanding of what

features of the protocol allowed this attack to take place. This often leads to insights about how to repair the protocol and eliminate the attack.

5.1 Solving tests

A *realized* skeleton is one in which every reception is explainable given the assumed restrictions on the attacker. The attacker is capable of producing every basic value (that is, every possible value of the basic sorts DATA, TEXT, NAME, AKEY, SKEY, and RNDX), except for those specifically withheld from the adversary. The values withheld are the ones for which there is one of the following declarations present: `uniq-orig`, `uniq-gen`, `non-orig`, or `pen-non-orig` (see Section 6.2 for detail).

In addition, the attacker observes all of the transmissions made in a skeleton. The attacker is also capable of manipulating messages in certain specific ways – these are the derivations present in Table 10.2.

When a reception message can be constructed by the attacker from the basic values accessible to the attacker and the transmissions that occur prior to that reception, we say it is *realized*, meaning that this reception can be explained.

When a skeleton to be analyzed contains at least one unrealized node, one of these nodes is selected as the "test node", and the subsequent skeletons are determined based on four distinct strategies for resolving (or at least making progress at resolving) the problem at the test node. The strategies focus on a *critical term*, a sub-value of the reception value that is a specific problem for derivability. The critical term may be a freshly generated value, which was generated by a regular participant but is subsequently found in a different message context than any in which it is known to have been transmitted. It may also be an encrypted (or digitally signed) unit, which must be generated on some regular strand unless the generating key is compromised. This inference builds in the assumption that cryptographic operators have an authentication value, as a symmetric encryption may be an authenticated encryption. CPSA always interprets an encryption operator as guaranteeing authenticity, unless the key is compromised.

Only certain kinds of sub-values may be of interest as critical terms, specifically, those which are *carried* . The notion of a carried sub-term can be defined recursively: a term carries itself, a pair carries either of its components, and an encryption carries its plaintext.

The four strategies are:

- A *regular augmentation*, which assumes the presence of an additional transmission carrying the critical term, in an entirely new strand,
- A *displacement*, which assumes the presence of an additional transmission carrying the critical term, but in an already existing strand,
- A *contraction*, which assumes that the critical issue can be resolved with some more refined version of the current transmissions that carry the critical value, and

- A *listener augmentation*, which assumes that some key not known to be derivable is in fact derivable.

When examining a skeleton and one of its children in full CPSA analysis it is often simple to determine which sort of strategy was used. If a child has one more strand than its parent then it was produced by either augmentation (if the extra strand is a role instance) or listener augmentation (if the extra strand is a listener). If a child has the same number of strands but additional nodes, it is the result of a displacement. When a child has the same number of nodes and strands, it may be the result of a displacement or a contraction, but either way the only thing that has changed is a map of terms.

5.2 Flawed Kerberos, revisited

In Section 4.1, we worked through an example – a flawed version of the Kerberos protocol. The first attempt at modeling the protocol was flawed in a way that would prevent the tool from discovering the attack against the protocol.

See Figure 4.1 for the flawed model of the protocol. Recall that the initiator is a, the server is s, and the responder is b. See Figure 4.4 for the corrected model; the difference is that in the corrected version, the initiator uses a variable of sort MESG to represent the ticket the initiator receives, since the initiator is not able to check the internal format of the ticket.

5.2.1 The operation field

Examining the full CPSA analysis is essential to detecting the mistake. Each skeleton has an `operation` field describing how it was derived from its parent, with one exception: when the input skeleton does not meet all the requirements for a skeleton, the first child will be the minimal skeleton incorporating the input.

In `kerb.xhtml`, tree 0 is the flawed model. Item 0 is the input point of view and Item 1 is the completion of that input into a skeleton; notice that the skeleton below Item 1 has no S-expression of the form `(operation ...)` in it.

Item 2 is a child of Item 1 that adds a key server instance. This was a regular augmentation, because a new protocol role instance was added. The operation field will include `added-strand` whenever the skeleton was produced by regular augmentation. The full operation field in Item 2 is:

```
(operation encryption-test (added-strand keyserv 2)
                (enc k n (ltk a s)) (0 1))
```

The first argument to the operation field is the general type of operation performed. Typically this will be one of the following three possibilities:

- `encryption-test` indicates that the critical term at the test node was an encryption.
- `nonce-test` indicates that the critical term at the test node was an atom, not an encryption.
- `channel-test` indicates that the critical term at the test node was an authenticated channel message.
- `generalization` indicates that the skeleton was realized and the tool is trying to make it more general without making it unrealized.

The second argument describes the operation; here, the operation is `added-strand` (regular augmentation), where the new instance is an instance of `keyserv` and is of length 2. The third argument is the criticial term; here it was `(enc k n (ltk a s))`, the portion of the server's message intended for the initiator. The last argument is a node, normally the test node, so in this case, the `(0 1)` refers to strand 0, node 1 – the second node of the initiator strand in Item 1.

Returning to our example, Item 2's critical term is the encryption $\{|k, n|\}_{SK(a,s)}$. The regular augmentation creates a new transmission of that encryption in some carried form; in this case, it is a transmission by the key server. Note that the key server instance in Item 2 agrees with the initiator on a but not on b.

See Figure 5.1 for Item 3 in the analysis. It is this step at which the analyst should notice there was a mistake of some sort. If the analyst is trying to understand why it should be impossible for the secret m to leak, thus far the reasoning is that if the initiator proceeds to its third event, there is a key server that agreed with the initiator on a, s, and n. But since the second initiator node is red, this indicates there is still something unexplained in the execution.

The operation field of Item 3 is telling:

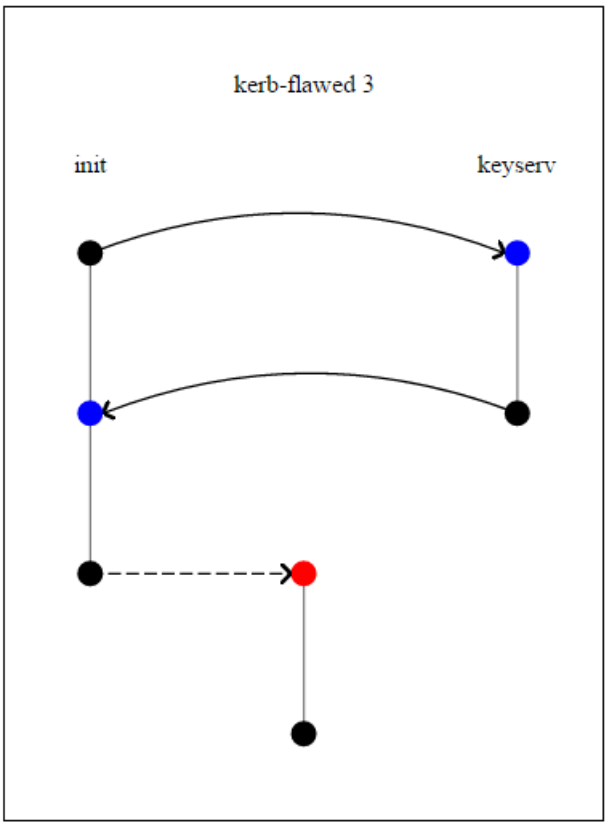
```
(operation encryption-test (displaced 3 2 keyserv 2)
              (enc k a b-0 (ltk b-0 s)) (0 1))
```

The critical term here is the encryption $\{|k, a, b0|\}_{K(b0,s)}$. Thus, the ticket portion of the initiator's reception is driving this step in the analysis.[1] After the displacement (which, you may notice, adds no new nodes), the key server agrees with the initiator on b.

In other words, the tool reasons first that the presence of the "authenticator" portion of the reception guarantees there is a key server instance that believes it is responding to a request initiated by a, but may or may not believe the request initiated by a was for communicating with b. The ticket portion makes the guarantee, but that's strange because

[1] Note that the specific encryption $\{|k, a, b0|\}_{K(b0,s)}$ is not actually present in Item 2 or in Item 3, it is a side effect of the internal way CPSA represents variables and determines how to display them. Still, only the ticket portion of the node `(0 1)` is of this kind of format, so the conclusion in the text is correct.

Item 3, Parent: 2, Child: 4.




```
(defskeleton kerb-flawed
  (vars (m n text) (a s b name) (k skey))
  (defstrand init 3 (m m) (n n) (a a) (b b) (s s) (k k))
  (deflistener m)
  (defstrand keyserv 2 (n n) (a a) (b b) (s s) (k k))
  (precedes ((0 0) (2 0)) ((0 2) (1 0)) ((2 1) (0 1)))
  (non-orig (ltk a s) (ltk b s))
  (uniq-orig m n k)
  (operation encryption-test (displaced 3 2 keyserv 2)
    (enc k a b-0 (ltk b-0 s)) (0 1))
  (label 3)
  (parent 2)
  (unrealized (1 0))
  (comment "1 in cohort - 1 not yet seen"))
```

Figure 5.1: Flawed Kerberos key moment in analysis

the ticket is for b and not a to examine. To believe this reasoning implies that the initiator really is checking the format of the ticket, which the analyst should realize is not the way the protocol is intended to work.

If you next examine the properly modeled protocol, you will find that Items 5, 6, and 7 follow a very similar path. Item 6 makes Item 5 into a skeleton, while Item 7 adds a key server instance in which the key server agrees with the initiator on a but not on b. But Item 7 is a realized skeleton, while Item 2 is not realized.

Chapter 6

Constraining CPSA's search

If you have read Chapters 3 and 4, you have seen the `uniq-orig` and `non-orig` features, which are the first representatives of a large class of features called *declarations*. These "declare" certain assumed properties about an execution in order to constrain the tool's search. In this chapter, we will describe more precisely the notion of execution that motivates the CPSA analysis, and precisely define these and other declarations available for use in the tool.

6.1 Bundles: A Strand-Based Execution Model

The CPSA tool is based on strand space theory [2]. A strand is simply a sequence of *events*, which are transmissions or receptions of messages.

A bundle is a set of strands, along with a satisfaction relation $\rightarrow$ from transmissions to receptions, where for every reception event e_1 of a message m, there is a unique transmission event e_0 of m such that $e_0 \rightarrow e_1$, and such that the graph formed on the events of the strands with edges defined by $\rightarrow$ and strand succession is acyclic.

Bundles express an explanatory framework in which a set of sequential viewpoints of transmissions and receptions is self-contained. Although bundles do not totally order the events, they express the orderings that are essential due to causality.

The most basic notion of a protocol is simply a set of roles which are each themselves strands representing a template of behavior. A bundle is a bundle *of a protocol* P if every strand in the bundle is either an instance of a role in P or an instance of a penetrator role. There are two types of penetrator roles: derivation roles and the "create" role. The derivation roles are determined by the message model; one matches each derivability rule in the algebra used. The create role consists of a one-event strand in which certain basic values may be transmitted. This always includes all constants (such as tag constants or the generator g in the Diffie-Hellman algebra), and also includes all values of basic sorts: the sorts other than MESG in the basic algebra, and those sorts plus the RNDX sort in the

Diffie-Hellman algebra.

6.2 Secrecy assumptions

A priori, the analysis of (say) the initiator's point of view in Needham-Schroeder is tasked with exploring all bundles of the Needham-Schroeder protocol in which an initiator instance of sufficient length is included. As it turns out, there is a most general shape, namely, the full-length initiator instance, alone. However, the bundle this shape describes is not of great interest, because the nonce chosen by the initiator appears in a "create" instance.

An assumption of secrecy is, in essence, a statement that the analyst is interested only in certain bundles. In the case of this nonce, for instance, we assume the nonce is secret and are thus uninterested in bundles in which that nonce appears in a "create" instance.

There are four declarations that may be used in CPSA to represent a secrecy assumption, but these assumptions are distinct from each other and each have a particular semantic and syntactic meaning.

To describe these assumptions, we first need some terminology. We use "carried" to refer to subterms of a term that can potentially be obtained from that term via decomposition. A pair carries each of its elements, and an encryption carries its plaintext. We extend this notion transitively, so for instance $\{\!|n1, n2|\!\}_{K(a)}$ carries $n1$ since it carries the plaintext $(n1, n2)$ which in turn carries $n1$.

We say that a value *originates* on a strand when its first *carried* occurrence is in a transmission. We say that a value *generates* on a strand when its first occurrence (whether carried or not) in that strand is in a transmission.

- A value *uniquely originates* in a bundle if it originates on exactly one strand. Note that if the unique origination point of a value is on a regular strand, then it cannot be produced in a "create" instance because that would constitute a second point of origination.

- A value is *non-originating* in a bundle if it does not originate on any strands. A simple lemma can show that a value is non-originating only if it is not carried in any message in the bundle.

- A value is *penetrator non-originating* in a bundle of a protocol if it does not originate on any penetrator strands. Clearly, such a value cannot be produced in a "create" instance.

The presence of a `uniq-orig`, `non-orig`, or `pen-non-orig` declaration of a value is a statement by the user that they are only interested in bundles in which the declared value uniquely originates, is non-originating, or is penetrator non-originating, respectively.

The fourth declaration that imposes a secrecy assumption is the `uniq-gen` declaration. The presence of a `uniq-gen` declaration of a value equates to a restriction to bundles in

which the declared value generates on only one strand. Note that if a value that generates uniquely generates on a regular strand, it cannot be produced in a "create" instance.

The `uniq-orig` (resp. `uniq-gen`) declaration can only be used in CPSA input for values that do originate (resp. generate) on the role or in the skeleton in which the declaration appears. This forces the use of such declarations to create a secrecy assumption.

Because "generation" covers a wider set of occurrences than "origination," a value that originates uniquely on one strand may still generate on a different strand. For instance, it may be used to encrypt a message on that strand. This means that one can model a key (for instance) that is distributed from a point of unique origination, and subsequently used to encrypt messages on other strands, without modeling a particular mechanism for storing it in the recipient's device. This property of origination and generation can, however, cause surprises if the key could surface unexpectedly in other strands. See Chapter 8 to understand how to store values explicitly, when that is desired.

In fact, because "generation" covers a wider set of occurrences than "origination," when a value v originates on a strand, declaring v `uniq-gen` is actually stronger than declaring it `uniq-orig`. The `uniq-gen` declaration prohibits unexplained key usage, as opposed to the `uniq-orig` declaration that permits it.

Exploration 6.1. Experiment with variants of your favorite simple protocol in which you try replacing `uniq-orig` with `uniq-gen` and/or `non-orig` with `pen-non-orig`. What do you find?

Exploration 6.2. Suppose a protocol you wish to model involves key management, and includes keys being encrypted and transmitted and received in encrypted form. What declaration should you use to model a long-term (not recently chosen) key as secret, if you are concerned about the actual possibility of that key leaking?

Note: these four declarations (`uniq-orig, non-orig, pen-non-orig,` and `uniq-gen`) may only be applied to basic values. One of the main purposes of such declarations is to restrict the use of the "create" role in bundles, so such declarations on values that cannot be produced in the "create" role is undesirable.

6.3 Diffie-Hellman declarations

A declaration arises as a part of the algorithm CPSA uses for Diffie-Hellman protocols. Like other declarations, these may be provided as part of a protocol or skeleton input.

- The `absent` declaration specifies that a particular RNDX variable does not occur in an exponent expression. This declaration is automatically added to specify x is absent within y whenever y is an exponent expression that occurs in a strand in which x is declared `uniq-gen` before the point where x is generated. This declaration is also added by the algorithm in rare circumstances.

6.4 Declaring skeletons

To ask CPSA a query, one uses a `defskeleton` form. The detailed syntax definition is in Table 10.1. We give here a summary of the core elements and how to use them. A `defskeleton` form such as those in Figs. 3.3–3.4 has the form

```
(defskeleton prot-name
  (vars ...)
  strand-decls ...
  assumptions ...
  comments)
```

`prot-name` is the name of the protocol. The `(vars ...)` give the names and sorts of the values appearing in the resulting skeleton, for instance values that may be parameters of one or more role instances in the skeleton, and may appear in the assumptions. The assumptions include unique origination and non-origination claims, as well as channel properties and state annotations (Chapter 8) and facts (Section 7.7.1). Comments are for the convenience of the specifier. The `strand-decls` consist of one or more forms

- `(defstrand role height bindings)` for an instance of `role` of the given height,
- `(defstrandmax role bindings)` to indicate an instance of `role` of its full maximum height, or
- `(deflistener t)` to indicate that the term `t` is heard transmitted publicly.

Generally, a `defskeleton` q containing a `(deflistener t)` declaration is used to see whether the term `t` remains confidential in the situation described by the remainder q' of the `defskeleton`. If CPSA determines that there are no shapes for this query, then there is no extension of q' in which the participants emit messages allowing the adversary to extract `t`. Typically, this is what the specifier intends to find: These are non-disclosure queries.

A `defskeleton` form most frequently contains a single `defstrand` or `defstrandmax`, in which case CPSA will determine what other behaviors this participant can rely on having occurred when a run of this role takes this number of steps. Thus, it determines what is learnt *from the point of view* of this participant. These are authentication queries from that participant's point of view.

When a single-strand authentication query gives some information about other local sessions, but we would like also to know whether additional conclusions follow if we assume unique origination or non-origination for values in those local sessions, then we can would use multiple `defstrand`s.

The bindings (aka "maplets") in a strand declaration are pairs `(p v)` where `p` is a parameter of the role being declared and `v` is a value defined from the variables declared

for this skeleton. Most often `v` just is one of these variables. These bindings can be confusing, since the role parameter and the variable in the skeleton may have the same name, so we have a binding `(b b)` or `(n1 n1)`, as in Fig. 3.3. However, the two `b`s are different: The first is the parameter declared in the `init` role in Fig. 3.2, while the second is the skeleton variable declared in the variable form `(vars (b name) (n1 text))`. A binding must appear in the strand declaration only if we need to assert something about it. In Fig. 3.3, those two bindings appear because the skeleton variable `n1` is assumed uniquely originating, and `b` appears because `(privk b)` is assumed non-originating. Any other parameters may be omitted, in which case CPSA supplies an unconstrained value for it.

6.5 A Caution about typos

Declarations are typically found syntactically in "association lists" which include declarations and may also include other fields, whether in protocol declarations, role declarations within them, or in skeleton query declarations. For instance, the CPSA tool creates associations in its `defskeleton` outputs specifying things like the algorithmic method used to produce this skeleton, or the label of the parent, etc. CPSA handles comments in the same way. Any association list entry `(header` ... *stuff* ... `)` in which the header is unrecognized is treated as a comment, and preserved in the output. This is convenient not just for documenting your specifications, but also for defining material to be used not by CPSA itself but by downstream tools.

Be careful when using declarations that you do not mis-spell the keywords! If you do, they will typically be treated just like a comment and ignored, so they will not have the effect you intended.

Part III

Advanced features of CPSA

Chapter 7

Logical Security Goals and Rules

How easy is it to read off authentication and secrecy properties? What precisely is it that an expert sees? This chapter describes CPSA's support for reasoning about security goals using first-order logic. With security goals expressed in first-order logic, intuition is replaced by determining if a formula is true in all executions of the protocol.

This treatment of security goals relies heavily on a branch of first-order logic called model theory. It deals with the relationship between descriptions in first-order languages and the structures that satisfy these descriptions. In our case, the structures are skeletons that denote a collection of executions of a protocol. This chapter describes the language of security goals and its structures without requiring the reader to have studied model theory.

The model theoretical foundation of this approach to security goals appears in [6]. A practical use of security goals in protocol standardization is described in [8, 14]. The precise semantics of the goal language is in [13]. The syntax of security goals appears in Table 7.1.

The CPSA distribution contains, in its examples directory, the input file `goals.scm`. The reader is encouraged to run the examples and examine the output while reading this chapter.

In this chapter we return to the Needham-Schroeder protocol discussed in Chapter 3. The roles are displayed in Figure 3.2.

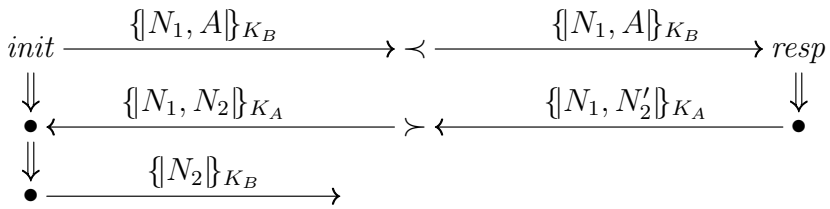
The protocol is analyzed from the point of view of a complete run of one instance of the initiator role. The input security goal, followed by the point of view skeleton it generates and the shape produced by CPSA, are shown in Figure 7.1. The syntax and semantics of the goal will be explained later. Roughly speaking, it asserts that if a realized skeleton contains a full length initiator strand, its private key is uncompromised, and it uniquely generates `n1`, then the skeleton will contain a responder strand that agrees with the initiator on the name `b`. The shape produced by CPSA contains the annotation `(satisfies yes)`. This indicates that its structure satisfies the description given by the security goal. It will be explained later why the properties of CPSA allows us to conclude that this goal is true in all executions of the protocol, and therefore conclude that the Needham-Schroeder protocol

GOAL	←	(**defgoal** ID SENTENCE+ ALIST)
SENTENCE	←	(**forall** (GVDECL∗) IMPLICATION)
GVDECL	←	(ID+ SORT) | (ID+ **strd**) | (ID+ **indx**)
VMAP	←	(ID ID)
IMPLICATION	←	(**implies** CONJUNCTION CONCLUSION)
CONJUNCTION	←	CONJUNCT | (**and** CONJUNCT+)
CONJUNCT	←	ATOMIC | (**strand** ID STRDVAR INTEGER (VMAP∗))
	|	(**listener** STRDVAR ID)
CONCLUSION	←	(**false**) | EXISTENTIAL
	|	(**or** EXISTENTIAL+)
EXISTENTIAL	←	CONJUNCTION | (**exists** (GVDECL∗) CONJUNCTION)
ATOMIC	←	(**p** STRING STRDVAR INTEGER)
	|	(**p** STRING STRING STRDVAR TERM)
	|	(**prec** STRDVAR INDEX STRDVAR INDEX)
	|	(**leads-to** STRDVAR INDEX STRDVAR INDEX)
	|	(**non** TERM) | (**pnon** TERM)
	|	(**uniq** TERM) | (**uniq-at** TERM STRDVAR INDEX)
	|	(**ugen** TERM) | (**ugen-at** TERM STRDVAR INDEX)
	|	(**conf** TERM) | (**auth** TERM)
	|	(**=** STRDVAR STRDVAR) | (**=** TERM TERM)
	|	(**gen-st** TERM) | (**trans** STRDVAR INDEX)
	|	(**comm-pr** STRDVAR INDEX STRDVAR INDEX)
	|	(**state-node** STRDVAR INDEX)
	|	(**same-locn** STRDVAR INDEX STRDVAR INDEX)
	|	(**component** TERM TERM) | (**fact** ID TERM*)
STRDVAR	←	SYMBOL
INDEX	←	SYMBOL | INTEGER

Table 7.1: Goal syntax. See Tables 10.3 and 10.5 for the algebra syntax, which defines the TERM and SORT symbols.

```
(defgoal ns                ; Goal
  (forall ((b name) (n1 text) (z0 strd))
    (implies
     (and (p "init" z0 3)
      (p "init" "n1" z0 n1) (p "init" "b" z0 b)
      (non (privk b)) (uniq n1))
     (exists ((z1 strd))
      (and (p "resp" z1 2) (p "resp" "b" z1 b))))))

(defskeleton ns            ; Point of view skeleton
  (vars (a b name) (n1 n2 text))
  (defstrand init 3 (a a) (b b) (n1 n1) (n2 n2))
  (non-orig (privk b))
  (uniq-orig n1))
```




```
(defskeleton ns            ; Shape
  (vars (n1 n2 text) (a b name))
  (defstrand init 3 (n1 n1) (n2 n2) (a a) (b b))
  (defstrand resp 2 (n2 n2-0) (n1 n1) (b b) (a a))
  (precedes ((0 0) (1 0)) ((1 1) (0 1)))
  (non-orig (privk b))
  (uniq-orig n1)
  (satisfies yes))
```

Figure 7.1: Needham-Schroeder Initiator Point of View

achieves this authentication goal.

7.1 Overview

In addition to `defskeleton` S-expressions, a CPSA input file may contain `defgoal` S-expressions. Like a `defskeleton`, a `defgoal` takes as its first parameter the name of a protocol which the tool expects has been previously defined in the input file. The second parameter to a `defgoal` is a *geometric* sentence in first-order logic. A geometric sentence contains one implication. The antecedent is a conjunction of atomic formulas. The free variables that occur in the antecedent are universally quantified. The conclusion is a disjunction of existentially quantified conjunctions of atomic formulas.

Alternately, a `defskeleton` can be augmentend with a `goal`, which may specify one or more geometric formulas to check.

When the tool is run, a `defgoal` is converted to a `defskeleton` that represents as narrowly as possible the left-hand side of the implication and has a `goal` recorded for the right-hand side. The tool then analyzes `defskeleton`s as usual, but realized skeletons found during the course of analysis of a `defskeleton` with a `goal` are augmented with a `satisfies` S-expression indicating whether the goal is met or not.

The tool `cpsa4goalsat` will examine the output from a CPSA run, and, for each goal formula, will determine whether the analysis ensures that the goal is met. If not, it identifies the labels of all the skeletons that furnish counterexamples to the goal. See Section 11.

The remainder of this chapter is devoted to explaining the semantics of this feature in greater detail.

7.2 Semantics

In a `defgoal` sentence, the antecedent specifies the point of view skeleton. We focus on the antecedent. In the example,

```
(defstrand init 3 (a a) (b b) (n1 n1) (n2 n2))
```

is extracted from

```
(and (p "init" z0 3)
  (p "init" "n1" z0 n1) (p "init" "b" z0 b)).
```

Notice that CPSA adds a binding for `a` and `n2` just as it does had

```
(defstrand init 3 (b b) (n1 n1))
```

been given as input.

Our aim now is to specify how to decide if a security goal is true in all possible executions of a protocol. A skeleton defines a set of executions that contain the skeleton's structure. We say a skeleton *satisfies* a formula if the skeleton contains all of the structure specified by the formula. To decide if a skeleton satisfies a formula, we decide if it satisfies each of its atomic formulas, and combine the results using the rules of first-order logic.

Atomic formula `(p "init" z0 3)` is called a role strand length formula. A skeleton k satisfies the formula if `z0` maps to a strand s in k such that

1. the trace of strand s in k has a length greater than 2, and
2. the trace when truncated to length 3 is an instance of the init role.

Consider the shape in Figure 7.1. It satisfies `(p "init" z0 3)` when `z0` maps to 0.

Atomic formula `(p "init" "n1" z0 n1)` is called a role parameter formula. A skeleton k satisfies the formula if `z0` maps to strand s in k, `n1` first occurs in at position i in the trace of the init role, and `n1` maps to a message algebra term t in k such that

1. the trace of strand s in k has a length greater than i,
2. the trace truncated to length $i + 1$ is an instance of the init role, and
3. the truncated trace is compatible with mapping the init role's `"n1"` role variable to t.

The shape in Figure 7.1 satisfies `(p "init" "n1" z0 n1)` when `z0` maps to 0 and `n1` maps to the message algebra term `n1`.

Collectively, a skeleton satisfies the formula

```
(and (p "init" z0 3)
     (p "init" "a" z0 a) (p "init" "b" z0 b)
     (p "init" "n1" z0 n1) (p "init" "n2" z0 n2))
```

if the skeleton contains the structure specified by

```
(defstrand init 3 (a a) (b b) (n1 n1) (n2 n2)).
```

The antecedent in Figure 7.1 contains two origination assertions. The formula `(non (privk b))` is extracted as `(privk b)`. A skeleton k satisfies the formula if `b` maps to a message algebra term t in k such that k assumes that t is non-originating. The unique origination formula for `n1` is similarly extracted.

Putting it all together, the mapping

$$\{\mathtt{n1} \mapsto \mathtt{n1}, \mathtt{n2} \mapsto \mathtt{n2}, \mathtt{a} \mapsto \mathtt{a}, \mathtt{b} \mapsto \mathtt{b}, \mathtt{z0} \mapsto 0\}$$

shows that the shape in Figure 7.1 satisfies the antecedent of the security goal.

The `prec` predicate is used to assert a node precedes another node. The conclusion in Figure 7.1 with `(prec z1 1 z0 2)` added is satisfied by the shape using the mapping `z0` $\mapsto 0$ and `z1` $\mapsto 1$.

The `uniq-at` predicate is used to assert not only that an atom uniquely originates, but also the node at which it originates. In the Figure 7.1 goal, the `(uniq n1)` formula could have been replaced by `(uniq-at n1 z0 0)`. The extracted point of view skeleton is the same.

The `ugen` and `ugen-at` predicates are similar, except that they refer to an earliest outbound occurrence, whether carried or not. If an atom is an asymmetric key, it and its inverse are generated together at the first outbound occurrence of either one.

Predicates `conf` and `auth` hold true of *channels*, when those channels are assumed to be confidential or authenticated channels as described in Chapter 8. The `gen-st` predicate holds true of a message value that cannot be an unexplained value in a state location, but can only be the result of a transition caused by a regular role (Section 8.2). The `leads-to` predicate relates a pair of nodes when the former creates a state value in a location, and the latter reads it from that location with no intervening transitions (Section 8.2). The `comm-pr`, `comm-pr`, and `state-node` predicates are used mainly internally to CPSA.

Recall that our aim in analyzing a protocol is to find out what security goals are true in all of its possible executions. We are interested in runs of CPSA that show that when every shape satisfies a goal, it is true in every execution.

When CPSA performs a shape analysis, every shape it generates refines the input skeleton. Skeleton refinement is defined in Chapter 5. The definition makes precise the notion of structure containment, as skeleton A refines skeleton B if and only if A contains the structure of skeleton B.

The skeleton k_0 extracted from the antecedent of a security goal has the property that a skeleton refines k_0 if and only if it satisfies the antecedent. A skeleton with this property is called the *characteristic skeleton* of the antecedent.

Given a goal Φ, consider a shape analysis starting from the characteristic skeleton k_0 of its antecedent. Assume CPSA finds shapes $k_1, \ldots, k_n$ and that CPSA determines that each k_i satisfies Φ. Consider the executions that contain the structure in k_0. CPSA tells us that each execution is in the executions that contain the structure of some k_i. Furthermore, because k_0 is a characteristic skeleton, each k_i satisfies the antecedent of Φ. Executions that do not contain the structure in k_0 do not satisfy the antecedent. Therefore, Φ is true in all executions of the protocol and maximally informative.

7.3 Examples

This section contains examples of both authentication and secrecy goals. The first example shows the feedback the user receives when a shape does not satisfy a security goal. The second example shows how to use a listener to state a secrecy goal.

7.3.1 Needham-Schroeder Responder

Figure 7.2 contains an analysis of Needham-Schroeder from the point of view of a complete run of the responder under the assumption that the responder's private key is uncompromised and the nonce it generates uniquely originates.

The conclusion of the goal asserts that in an execution compatible with the point of view, there must be an initiator strand that agrees with the responder strand on the name `b`. The shape produced by CPSA is a counterexample to this assertion. Because of this, CPSA annotates the shape with

```
(satisfies (no (a a) (b b) (n2 n2) (z0 0))).
```

The annotation includes a variable mapping for the shape that satisfies the antecedent of the goal but does not satisfy its conclusion. The reason the shape does not satisfy the goal is because the mapping `(b b)` maps the initiator's `b` parameter to `b`, not `b-0` as is required to model the shape.

Galvin Lowe identified this authentication failure in Needham-Schroeder and provided a fix. In the Needham-Schroeder-Lowe Protocol, the name `b` is included within the encryption in second message of both roles. With this modification, the shape found by CPSA satisfies the security goal in Figure 7.2, so Needham-Schroeder-Lowe achieves this authentication goal.

7.3.2 A Needham-Schroeder Secrecy Goal

Figure 7.3 contains an analysis of Needham-Schroeder from the point of view of a complete run of the initiator under the assumption that the responder's and its peer's private keys are uncompromised and the nonce `n1` it generates uniquely originates. Futhermore, the point of view asserts that the nonce is leaked using a listener.

```
(p "" z1 1) (p "" "x" z1 n1)    ; Listener
```

CPSA finds no shapes, so Needham-Schroeder achieves this secrecy goal and does not leak `n1`.

7.4 The Rest of the Story

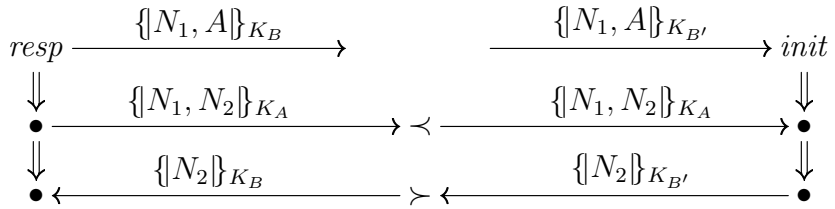
The examples in the previous section demonstrate the typical way security goals are analyzed with CPSA. However, there are more features that may be useful.

A `defgoal` form may contain more than one sentence. See Figure 7.4 for an example. When presented with more than one sentence, CPSA extracts the point of view skeleton from the first sentence.

It is wise to ensure that the antecedent in every sentence is identical. When CPSA performs satisfaction-checking on sentence Φ, it only determines if each shape it finds is

```
(defgoal ns                ; Goal
  (forall ((a b name) (n2 text) (z0 strd))
    (implies
      (and (p "resp" z0 3) (p "resp" "n2" z0 n2)
        (p "resp" "a" z0 a) (p "resp" "b" z0 b)
        (non (privk a)) (uniq n2))
      (exists ((z1 strd))
        (and (p "init" z1 2) (p "init" "b" z1 b))))))

(defskeleton ns            ; Point of view skeleton
  (vars (a b name) (n1 n2 text))
  (defstrand resp 3 (a a) (b b) (n1 n1) (n2 n2))
  (non-orig (privk a))
  (uniq-orig n2))
```




```
(defskeleton ns            ; Shape
  (vars (n1 n2 text) (a b b-0 name))
  (defstrand resp 3 (n2 n2) (n1 n1) (b b) (a a))
  (defstrand init 3 (n1 n1) (n2 n2) (a a) (b b-0))
  (precedes ((0 1) (1 1)) ((1 2) (0 2)))
  (non-orig (privk a))
  (uniq-orig n2)
  (satisfies (no (a a) (b b) (n2 n2) (z0 0))))
```

Figure 7.2: Needham-Schroeder Responder Point of View

```
(defgoal ns
  (forall ((a b name) (n1 text) (z0 z1 strd))
    (implies
      (and (p "init" z0 3) (p "init" "n1" z0 n1)
        (p "init" "a" z0 a) (p "init" "b" z0 b)
        (p "" z1 1) (p "" "x" z1 n1)  ; Listener
        (non (privk a)) (non (privk b))
        (uniq n1))
      (false))))
```

Figure 7.3: Needham-Schroeder Secrecy Goal

```
(defgoal ns
  (forall ((a b name) (n text) (z0 strd))
    (implies
      (and (p "init" z0 2) (p "init" "n1" z0 n)
        (p "init" "a" z0 a) (p "init" "b" z0 b)
        (non (privk a)) (non (privk b)) (uniq n))
      (exists ((z1 strd))
        (and (p "resp" z1 2) (p "resp" "b" z1 b)))))
  (forall ((a b name) (n text) (z0 strd))
    (implies
      (and (p "init" z0 2) (p "init" "n1" z0 n)
        (p "init" "a" z0 a) (p "init" "b" z0 b)
        (non (privk a)) (non (privk b)) (uniq n))
      (exists ((z1 strd))
        (and (p "resp" z1 2) (p "resp" "a" z1 a))))))
```

Figure 7.4: Two Initiator Authentication Goals

satisfied. If the point of view skeleton is not the characteristic skeleton of the antecedent of Φ, the fact that all skeletons satisfy Φ cannot be used to conclude that Φ is true in all executions of the protocol.

CPSA performs satisfaction-checking when an input skeleton in annotated with one or more security goals. The annotation uses the `goals` key.

```
(defskeleton
   ...
   (goals SENT+))
```

The program `cpsa4sas`, discussed in the next section, can be used to generate a formula with an antecedent such that the input skeleton is the characteristic skeleton of the

antecedent.

7.5 Syntactic Abbreviations

Section 7.2 described how CPSA extracts

```
(defstrand init 3 (a a) (b b) (n1 n1) (n2 n2))
```

from the following conjunction of facts

```
(and (p "init" z0 3)
  (p "init" "n1" z0 n1) (p "init" "b" z0 b)).
```

Using a conjunction of facts to represent a single `defstrand` is convenient for defining and implementing a clear semantics, however it can be quite cumbersome and error prone to specify these conjunctions. We therefore also provide support for syntactic sugar in the goal language to allow users to represent these conjunctions in a manner much closer to the `defstrand` form found in a skeleton. In particular, the user can write

```
(strand init z0 3 (n1 n1) (b b))
```

as a single conjunct in a formula, and CPSA will ingest it with the same interpretation as if the user had entered

```
(and (p "init" z0 3)
  (p "init" "n1" z0 n1) (p "init" "b" z0 b)).
```

Thus the goal in Fig. 7.1 could therefore be written as follows instead:

```
(defgoal ns
  (forall ((b name) (n1 text) (z0 strd))
    (implies
     (and (strand init z0 3 (n1 n1) (b b))
      (non (privk b)) (uniq n1))
     (exists ((z1 strd))
      (strand resp z1 2 (b b))))))).
```

This representation has the benefit of being syntactically quite close to the skeleton representation, and therefore the connection is more visually obvious to the user. Notice the crucial difference that the `(strand ...)` form includes a strand variable after the name of the role. While such a variable is not needed for skeletons, it is required in formulas to allow other facts to reference the strand it represents.

We also provide a similar abbreviation for listener strands. Instead of writing

```
(defgoal ns
  (forall ((n1 n2 text) (b a name) (z strd))
    (implies
      (and (p "init" z 3) (p "init" "n1" z n1)
        (p "init" "n2" z n2) (p "init" "a" z a)
        (p "init" "b" z b) (non (privk b)) (uniq-at n1 z 0))
      (exists ((n2-0 text) (z-0 strd))
        (and (p "resp" z-0 2) (p "resp" "n2" z-0 n2-0)
          (p "resp" "n1" z-0 n1) (p "resp" "b" z-0 b)
          (p "resp" "a" z-0 a) (prec z 0 z-0 0)
          (prec z-0 1 z 1))))))
```

Figure 7.5: Initiator Shape Analysis Sentence

```
(and (p "" z0 1) (p "" "x" z0 n1))
```

we can write the following instead

```
(listener z0 n1).
```

These abbreviations can be used both in goals and in rules (described below in Section 7.7).

7.6 Shape Analysis Sentences

A shape analysis sentence expresses all that can be learned from a run of CPSA as a security goal. If a security goal can be derived from a shape analysis sentence, then the protocol achieves the security goal, that is, the goal is true in all executions of the protocol.

The program `cpsa4sas` extracts shape analysis sentences from the output of CPSA. Consider the first example in this paper (Figure 7.1), which is in the sample file `goals.scm`. To generate a maximally informative security goal from the initiator point of view with `ghci` and `Make.hs`, type

```
$ ghci Make.hs
*Make> sas "goals"
```

When using GNU make, type "`make goals_sas.text`". The resulting shape analysis sentence is displayed in Figure 7.5.

A shape analysis sentences asserts that either a realized skeleton does not satisfy its antecedent or it satisfies one or more of the disjuncts in its conclusion. CPSA has been designed so that this assertion is true. Therefore, every shape analysis sentence is true in all executions.

A security goal is true in all executions if it can be derived from a shape analysis sentence [13]. In practice, theorem-proving using shape analysis sentences is rarely employed. It's clumsy and it requires too much expertise. The main use of `cpsa4sas` is for generating a formula that is edited to become a desired security goal.

In general, CPSA will not use the syntactic abbreviations of the previous section in its output, preferring instead the fully elaborated conjunction of facts. So the shape analysis sentences will not contain these abbreviations.

7.7 Rules

Support for rules was introduced in version 3.6.0 of CPSA.

Each protocol includes a small collection of rules. A rule is a sentence in the goal language presented in Table 7.1. Rules are defined after the roles of a protocol are defined. The syntax of a rule follows.

RULE ← (**defrule** NAME SENTENCE COMMENTS)

A rule is an axiom added to a protocol. CPSA uses the axiom as a rewrite rule to derive zero or more new skeletons from a skeleton produced during a step. An example of a protocol with a rule is in Figure 7.8 on Page 66.

The trust rule states that when CPSA finds a person strand of length at least one, and the inverse of it's `p` parameter is non-originating, CPSA should assume the inverse of it's `d` parameter is non-originating.

7.7.1 Facts

Each skeleton includes a small database of facts. A fact is a named relation among fact terms. A fact term is either a strand of the skeleton or an algebra term. A set of facts is defined anywhere after strands are defined using the `facts` form. The syntax of facts follows.

FACTS ← (**facts** FACT∗)
FACT ← (SYMBOL FTERM∗)
FTERM ← MESG | NAT

For example, in a skeleton, a user may want to note that strand 0 owns the private key for `a` by assuming.

```
(facts (owns 0 (privk a)))
```

Facts are most useful when combined with rules. Here is an example of their combination. Suppose a point of view skeleton has two names, `a` and `b`, and the problem is modeling a situation in which the two names are known to differ. To enforce this constraint, add

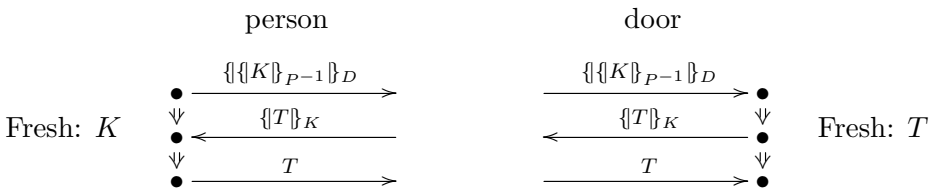


Figure 7.6: DoorSEP Protocol

```
(facts (neq a b))
```

to the point of view skeleton. CPSA contains a rule governing `neq` rule stating essentially:

```
(defrule neq
  (forall ((a mesg))
    (implies
      (fact neq a a)
      (false))))
```

7.7.2 DoorSEP

As a motivating scenario consider the Door Simple Example Protocol (DoorSEP), derived from an expository protocol that was designed to have a weakness. Despite this, the protocol achieves the needs of an application, given a trust assumption.

Imagine a door D which is equipped with a badge reader, and a person P equipped with a badge. When the person swipes the badge, the protocol executes. Principals such as doors or persons are identified by the public parts of their key pairs, with D^{-1} and P^{-1} being the corresponding private keys. We write $\{|M|\}_K$ for the encryption of message M with key K. We represent digital signatures $\{|M|\}_{P^{-1}}$ as if they were the result of encrypting with P's private key.

P initiates the exchange by creating a fresh symmetric key K, signing it, and sending it to the door D encrypted with the door's public key. D extracts the symmetric key after checking the signature, freshly generates a token T, and sends it—encrypted with the symmetric key—back to P. P demonstrates they are authorized to enter by decrypting the token and sending it as plaintext to the door. The two roles of DoorSEP are shown in Fig. 7.6, where each vertical column displays the behavior of one of the roles. The CPSA encoding of the roles is in Figure 7.8.

CPSA finds an undesirable execution of DoorSEP. Assume the person's private key P^{-1} is uncompromised and the door has received the token it sent out. Then CPSA finds that P freshly created the symmetric key K. However, nothing ensures that the person meant to open door D. If P ever initiates a run with a compromised door C, the adversary can perform a man-in-the-middle attack, decrypting using the compromised key C^{-1} and

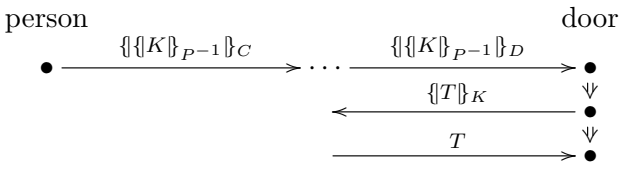


Figure 7.7: DoorSEP Weakness

re-encrypting with D's public key, as elided in the $\cdots$ in Fig. 7.7. To verify this result with CPSA, remove the trust axiom in the doorsep protocol in `examples/rules.scm` and run CPSA. Thus, without additional assumptions, the door cannot authenticate the person requesting entry.

But possibly we can trust the person to swipe her badge only in front of doors our organization controls. And we can we ensure that our doors have uncompromised private keys. If so, then the adversary cannot exercise the flaw.

We regard this as a *trust assumption*, and we can express it as an axiom:

Trust Assumption 1. If an uncompromised signing key P^{-1} is used to prepare an instance of the first DoorSEP message, then its owning principal has ensured that the selected door D has an uncompromised private key.

The responsibility for ensuring the truth of this axiom may be split between P and the organization controlling D. P makes sure to swipe her badge only at legitimate doors of the organization's buildings. The organization maintains a security posture that protects the corresponding private keys.

Is DoorSEP good enough, assuming the trust axiom? Add the trust axiom back to the doorsep protocol in `doc/rules.scm` and see. You should find that the protocol does its job; namely, ensuring that the door opens only when an authorized person requests it to open.

A trick to control rules. Sometimes, one would like to explore two versions of a protocol: One that contains a particular rule, and another that doesn't. Some CPSA queries should be executed with the rule in force, and others without the rule. For instance, one may want to know which security properties hold even without the assumption made in the rule.

To do this, one can add a sort of flag to the rule as an additional premise. For instance, in the DoorSEP case, we could have written the rule as shown in Fig. 7.9, with the additional premise `(fact use-trust-rule)`.

```
(defprotocol doorsep basic
  (defrole person
    (vars (d p akey) (k skey) (t text))
    (trace
     (send (enc (enc k (invk p)) d))
     (recv (enc t k))
     (send t)))
  (defrole door
    (vars (d p akey) (k skey) (t text))
    (trace
     (recv (enc (enc k (invk p)) d))
     (send (enc t k))
     (recv t)))
  (defrule trust
    (forall ((z strd) (p d akey))
            (implies
             (and (p "person" z 1)
                  (p "person" "p" z p)
                  (p "person" "d" z d)
                  (non (invk p)))
             (non (invk d))))
    (comment "The trust rule"))
  (comment "Doorsep protocol using unnamed asymmetric keys"))

(defskeleton doorsep
  (vars (p akey))
  (defstrand door 3 (p p))
  (non-orig (invk p))
  (comment "Analyze from the doors's perspective"))
```

Figure 7.8: Door Simple Example Protocol

```
  (defrule trust-sometimes
    (forall ((z strd) (p d akey))
            (implies
             (and (fact use-trust-rule)
                  (p "person" z 1)
                  (p "person" "p" z p)
                  (p "person" "d" z d)
                  (non (invk p)))
             (non (invk d))))
    (comment "The trust rule.  Assume (fact use-trust-rule) to enable it"))

. . .

(defskeleton doorsep
  (vars (p akey))
  (defstrand door 3 (p p))
  (non-orig (invk p))
  (facts (use-trust-rule))
  (comment "Analyze from the doors perspective, with trust rule"))
```

Figure 7.9: Rule with a flag to control its use

Now, a query that should not use the `trust-sometimes` rule can be stated as usual, as in Fig. 7.8. A query that should use the rule can add the `use-trust-rule` fact to the `defskeleton` form, as shown in Fig. 7.9. As a consequence, the additional premise will be satisfied any time that CPSA would apply the rule.

This also illustrates the way that a fact that is written `(fact use-trust-rule)` in a goal or rule is declared `(facts (use-trust-rule))` in a `defskeleton` form.

7.7.3 Generated rules

CPSA generates certain rules either automatically in all cases or else as a consequence of user inputs. The rules generated for a given protocol input may be found in the printed output for that protocol; these are the formulas appearing in `defgenrule` forms in the `.txt` and `.xhtml` files.

Negated equalities. Since negated equalities are often needed to express assumptions users are interested in, CPSA generates three rules with the body:

```
(implies (fact neq x x)
         (false))
```

This rule has the effect of terminating a branch of analysis if two expressions that the user has stipulated should not be equal become unified to the same value. Three rules are generated since the variable `x` may be of sort `mesg`, or of sort `strd` for a strand in the skeleton, or of sort `indx` meaning the index of a node along a strand. To stipulate a negated equality between two expressions e_1, e_2, the user may write in the input file:

```
(defskeleton protocol
   ... strand declarations, etc. ...
   (facts (neq e1 e2)))
```

Negated equality assertions may also appear in the conclusions of user-defined rules, or anywhere else that an atomic formula may appear.

A good number of additional rules are generated to implement CPSA 4's model of state (see Section 8.2, esp. Figs. 8.4–8.5 and pp. 88–89).

"Assume" declarations. A formula may be provided after the `trace` declaration in a `defprotocol` declaration in an `assume` clause. For instance:[1]

```
(defrole init
    (vars ...)
    (trace ...)
    ...
    (assume (fact neq (exp (gen) upsilon) (gen))))
```

[1] See the file `dhcr_um_expt_assume.scm` in the manual examples directory.

In this case, the CPSA loader checks that all of the free variables (just `upsilon` here) appear in the trace. If i is the first height at which all of the variables have occurred, CPSA generates a rule stating that any instance of the rule of height $\geq i$ satisfies this property of the instances of the role parameters. In this case, $i = 4$, so the generated formula states:

```
(defgenrule assume-init-0
    (forall ((z strd) (upsilon expt))
      (implies (and (p "init" z 4) (p "init" "upsilon" z upsilon))
        (fact neq (exp (gen) upsilon) (gen)))))
```

Here, the assumption is the uninterpreted predicate symbol `neq` applied to two expressions. However, the assumption may also involve conjunction and existential quantifiers. CPSA also permits disjunctions, but these typically cause unreasonable branching and bad performance.

In case the assumption is a fact, such as the `neq` assertion above, one can write

```
(defrole init
    (vars ...)
    (trace ...)
    ...
    (facts (neq (gen) (exp (gen) upsilon))))
```

instead of writing the assume annotation above. Many facts may be listed as clauses here, with the effect of a conjunction. The keyword here is the plural `facts`, and the singular `fact` does not appear in the individual clauses.

The `facts` annotation may also appear in a `defskeleton` query.

```
(defskeleton dhcr-um
  (vars (a b name) (beta expt))
  (defstrand init 5 (a a) (b b) (beta beta))
  (non-orig (privk "sig" b))
  (facts (neq a b) (undisclosed beta)))
```

Currently, CPSA does not recognize the `assume` keyword in `defskeleton` queries.

7.7.4 Rules generated for trace annotations

CPSA now allows a few types of annotations to be interspersed within the trace declaration within a role. These are:

rely, used after a reception to declare a formula containing only free variables appearing up to and including this reception. Every instance of the role long enough to include a successful instance of this reception must satisfy the formula;

guar, used before a transmission to declare a formula containing only free variables appearing up to and including the following transmission. Every instance of the role long enough to include an instance of the following transmission must satisfy the formula.

All of these trace annotations will cause similar rules to be generated. However, when Zappa compiles role definitions to Rust code, they are compiled to have different operational effects. Trace annotations do not add to the height of the trace. They simply express constraints on what happens in the "real" events around them.

Expressing rely and guarantee statements. In work on trust management, and on compiling cryptographic protocols [9, 7, 11], we introduced the idea of a *rely formula* that expresses what a principal learns when a message is received, possibly assuming that certain peers are uncompromised. We also introduced the idea of a (dual) *guarantee formula* that must be true in order for a principal to transmit a message. In particular, if a message sent by a principal A has a particular guarantee formula ϕ, and a peer B receives it and authenticates its origin with A in this step of the protocol, then B may rely on a formula with the meaning, A *says* ϕ.

This example will be *sound* in the sense that the authentication property ensures that A really did say (i.e. guarantee) ϕ in any execution in which B relies on the claim. The soundness of rely statements may be checked by analyzing the protocol with the rely statements are omitted, and querying them as goal statements under the relevant non-compromise assumptions. We will now assume that this soundness checking has been done, and that any relevant non-compromise assumptions are made in the protocol definition.

Thus, a *rely* formula definitely holds true whenever the principal successfully completes the preceding receive event. Indeed, the operational interpretation of a *rely* formula is that the principal will add an assertion to a database of its knowledge. CPSA transforms a *rely* formula after a reception which is at height i in a trace into a rule stating: "long-enough" instances of the role will satisfy the formula, for that instance's values of the role parameters.

This transformation is similar to what CPSA does for a `cheq` annotation, except that instead of an equation, the conclusion may involve other atomic formulas, conjunctions, and existential quantifiers. Disjunctions are permitted, but will usually lead to intractable CPSA analysis.

A *guarantee* formula has a dual relation with a subsequent transmission. Just as the received message may determine the instances of parameters appearing in a rely formula, a guarantee formula may dually select instances of parameters that will appear in the subsequent transmission. As a rely formula is interpreted operationally as an assert to a database held by the principal, a guarantee formula is interpreted operationally as a query against that database. Hence, assertions caused by previous rely formulas may influence the parameter instances selected by subsequent guarantee formulas.

A rely-guarantee example. Consider a protocol in which a customer will interact with a merchant and a bank to purchase an item at a cost; the merchant will be paid by a debit from the customer's account at the bank. Each participant should obtain evidence that the expected peers are participating in a corresponding session. Moreover, the merchant does not need to know the customer's account number, and therefore should not see this information. The bank does not need to know what item the customer is buying, and therefore should not see that. Outsiders should not see the details at all. How best to achieve this goal?

Each party will need to generate a digital signature, since in case of dispute a third party must be able to reconstruct who committed to paying whom, and to sending what merchandise. Moreover, public key encryption will be needed, at least for a handshake, since the parties must protect secrets from the outsiders and from the third party. Thus, there is no way to avoid having three digital signatures generated, along with a similar number of public key encryptions, with corresponding verification and decryption actions. A protocol that achieves this goal is found in `examples/minipay.scm`.

Each party is making assertions, in effect. The customer asserts his desire to buy the item for the cost. The bank asserts that it will transfer the funds to the merchant. The merchant asserts that it will ship the item. Moreover, the protocol should ensure that each party knows that the others have made these commitments. For instance, the merchant is willing to ship the goods because it knows that the bank will transfer the payment. Thus, each party makes a *guarantee*, and each party learns that it may *rely* on its peers having made their guarantee. A fine point is the bank's rely formula; it learns that the customer has requested *some* item in exchange for the cost, but not which one. Thus, the item is existentially bound in the bank's rely formula.

This yields the rely and guarantee formulas displayed in Fig. 7.10; the protocol with these trust annotations may be found in `examples/minipay-rely-guar.scm`. The principals in a particular session are required not to proceed past a guarantee formula unless the instances of the variables are known to satisfy the property claimed. The rely formulas summarize what they have learned in the preceding reception.

But: How do we know these formulas are in fact true in every execution that includes the preceding reception, or at least every execution satisfying some reasonable assumptions about uncompromised keys?

We can use goal formulas to check this. In a separate file, `examples/minipay-guar.scm`, we define the protocol with the same message structure and guarantee formulas, but with no rely formulas. We now pose goal statements for CPSA to resolve; these are shown in Fig. 7.11–7.12. In these formulas, the conclusions are the rely statements that must be justified. The hypotheses say that there is a strand z instantiating the relevant role long enough to reach the rely statement position. They also connect the variables in the rely formula with the parameters of z. Moreover, each customer goal assumes only the relevant peer's signature key to be non-compromised. The merchant assumes only the bank's signing key non-compromised, which is desirable since it may have no long-term association

```
(defrole cust (vars ...)
  (trace
    (guar (fact buy-via c m b item cost n))
    (send ...)
    (recv ...)           ; from bank
    (rely (fact will-transfer c m b cost n mtr btr))
    (recv ...)           ; from merchant
    (rely (fact will-ship c m b item mtr))))

(defrole merc (vars ...)
  (trace
    (recv ...)           ; from customer
    (send ...)
    (recv ...)           ; from bank
    (rely (and (fact buy-via c m b item cost n)
               (fact will-transfer c m b cost n mtr btr)))
    (guar (fact will-ship c m b item mtr))
    (send ...)))

(defrole bank (vars ...)
  (trace
    (recv ...)
    (rely (exists ((item merchandise))
                  (fact buy-via c m b item cost n)))
    ;; Do not proceed unless customer signature trustworthy.
    ;; Given that, now commit to transferring the funds
    (guar (and (non (privk "sig" c))
               (fact will-transfer c m b cost n mtr btr)))
    (send ...)))
```

Figure 7.10: Rely and gauantee formulas in `minipay.scm`

```
(defgoal minipay-guar
  (forall
   ((z strd) (c m b name) (cost amount) (n data) (mtr btr text))
   (implies
    (and (p "cust" z 2)
         (p "cust" "c" z c)
         (p "cust" "m" z m)
         (p "cust" "b" z b)
         (p "cust" "cost" z cost)
         (p "cust" "n" z n)
         (p "cust" "mtr" z mtr)
         (p "cust" "btr" z btr)
         (non (privk "sig" b)))
    (fact will-transfer c m b cost n mtr btr))))

(defgoal minipay-guar
  (forall
   ((z strd) (c m b name) (item merchandise) (n data) (mtr btr text))
   (implies
    (and (p "cust" z 3)
         (p "cust" "c" z c)
         (p "cust" "m" z m)
         (p "cust" "b" z b)
         (p "cust" "item" z item)
         (p "cust" "n" z n)
         (p "cust" "mtr" z mtr)
         (p "cust" "btr" z btr)
         (non (privk "sig" m)))
    (fact will-ship c m b item mtr))))
```

Figure 7.11: Goal statements to validate customer's rely formulas

with the customer. The bank presumably has an association with the customer, established when the customer opened the account; the customer's signing key could be established at that time.

The goals may be constructed from the generated rules CPSA provides when processing `examples/minipay-rely-guar.scm`, essentially adding the non-compromise assumptions.

Since CPSA validates these goal formulas in the `minipay-guar` protocol that contains the guarantee formulas but not the rely formulas, the generated rules introduced by the rely formulas are also sound subject to the stated non-compromise assumptions.

We regard this rely-guarantee method as a systematic way to formalize the trust assumptions, and the effects, of protocols that accomplish real-world tasks, in this case

```
(defgoal minipay-guar
  (forall
   ((z strd) (c m b name) (item merchandise) (cost amount) (n data) (mtr btr text))
   (implies
    (and (p "merc" z 3)
         (p "merc" "c" z c)
         (p "merc" "m" z m)
         (p "merc" "b" z b)
         (p "merc" "item" z item)
         (p "merc" "cost" z cost)
         (p "merc" "n" z n)
         (p "merc" "mtr" z mtr)
         (p "merc" "btr" z btr)
         (non (privk "sig" b)))
    (and (fact will-transfer c m b cost n mtr btr)
         (fact buy-via c m b item cost n)))))

(defgoal minipay-guar
  (forall
   ((z strd) (c m b name) (cost amount) (n data))
   (implies
    (and (p "bank" z 1)
         (p "bank" "c" z c)
         (p "bank" "m" z m)
         (p "bank" "b" z b)
         (p "bank" "cost" z cost)
         (p "bank" "n" z n)
         (non (privk "sig" c)))
    (exists ((item merchandise))
            (fact buy-via c m b item cost n)))))
```

Figure 7.12: Goal statements to validate other rely formulas

purchase and payment.

Chapter 8

Channels and State

Suppose that we are developing a protocol. We may well want to work out the overall message flow, and understand the consequences of different choices, before we figure out how—concretely—to protect particular messages. For instance, we may know that the sender of a particular message will want to ensure that its contents will remain confidential, without selecting a particular mechanism for doing so. Having worked out the overall structure of the protocol, it may later be clear how to discharge the assumption that the contents remain confidential. Or in other cases, we may know that we will use the same main message flow in several contexts, and in each of those contexts there may be different mechanisms that would make sense.

In those kinds of development, CPSA allows us to assume that messages are transmitted over *channels* that are assumed to offer security guarantees to the messages they carry. The treatment of channels also helped determine an approach to representing long-term, cross-session mutable state in CPSA. The goal of this chapter is to explain how to use the CPSA channel mechanism, and to build on that understanding to work with long-term mutable state.

Channel security guarantees. The most important guarantees are *authenticated* channels and *confidential* channels. They stipulate actions that must be taken by some instance of a role of the protocol, rather than by the adversary.

- An authenticated channel guarantees to the recipient of a message that this message was sent by some instance of a role of the protocol.
- A confidential channel guarantees to the sender that this transmission will not be received by the adversary, i.e. that any recipient is an instance of a role of the protocol. Hence, any disclosure of not-yet-public parts of the message occur only if the message is received and acted on by one or more instances of protocol roles.

A channel that is both *authenticated* and *confidential* is sometimes called a *secure* channel.

Later, when we have developed an adequate protocol using the channel properties, we can refine it to one or more protocols that use specific cryptographic mechanisms to achieve the properties we need. For instance, digital signatures or Message Authentication Codes can achieve authenticity, public-key encryption can achieve confidentiality, and symmetric encryption (e.g. an authenticated encryption mode) can implement a secure channel.

Likewise, specific TLS modes can be used to implement authentic, confidential, and secure channels. For instance, a typical unilaterally authenticated TLS connection provides the client an authenticated channel for messages from the server, while it provides a confidential channel for messages to the server. In itself, the TLS unilateral handshake provides no guarantee to the server.

Other properties of channels are sometimes of interest, but are not directly implemented by CPSA. For instance, a *fair* channel ensures that a message transmitted onto it will eventually be delivered to some instance of a role of the protocol. A *deliver-once* channel ensures that a message transmitted onto it will be delivered at most once to any instance of a role of the protocol. *In-order delivery* is another possible channel property. Some of these may be represented indirectly in CPSA specifications [5]. However, authenticated and confidential channels are often useful in protocol design, and fit smoothly into the CPSA authentication test method.

In Section 8.1, we describe how to use the CPSA channel mechanism.

Channels and long-term state. In Section 8.2, we also describe a model of protocols that rely on long-term, cross-session state, which is based on the channel mechanism. Here we consider a regular principal that interacts privately with a device containing some registers that maintain state.[1] The principal may read values out of the state registers and write back other values, often related values; these actions determine the state transitions the device can undergo when used in this protocol.

The connection with channels is this. When a regular protocol participant writes a value into a state register, this is akin to sending a message to oneself at a future moment. What one relies on, when using this mechanism, is that the recipient of this message—one's future self—is still a regular participant, so that any future disclosure to the adversary will be due to some other action. That is, writing to a state register is sending a message on a confidential channel.

When reading from a state register, there are two possibilities. One is that one's past self wrote this value into the state register previously. In this case, reading from the state register is like receiving a message on an authenticated channel. However, there is another case also, in which the value read may simply have been an initial state, available from the state register at power-up, so to speak. We call a state value a *generated state value* when the second case cannot apply to it, i.e. it is a value that may have been written

[1] By a *regular* principal for a protocol Π, we mean a non-adversarial principal, i.e. one that acts only in accordance with the protocol Π.

into the register but was definitely not available simply as an initial, power-up value. We summarize the idea by saying that reading a generated state value from a state register is receiving a authenticated value from a channel.

Thus, state registers offer a confidentiality property for written values, and an authenticity property for read values, if they are generated values. Thus, for state registers, the duality of confidentiality and authenticity holds in a limited form, i.e. for generated values. This is the reason why CPSA implements state via the same ideas as confidential and authenticated channels, although state requires some additional properties that will be described below in Section 8.2.

8.1 Channels

The CPSA model of authenticated or confidential channels requires only a small amount of syntax. First, variables of sort `chan` may be declared to range over channels, for instance:

```
(vars ... (ch-name-1 ch-name-2 chan) ...)
```

Second, a transmission event or reception event may have an additional component that is a variable of sort `chan`, taking the forms:

```
(send ch-name-1 sent-msg)
(recv ch-name-2 recvd-msg)
```

Finally, we may assert the *authenticated* or *confidential* property of a channel, using the annotations:

```
(auth ch-name-1 ch-name-2)
(conf ch-name-2)
```

As with annotations such as `uniq-orig` and `non-orig`, these can appear either within a `defskeleton` query form or within a `defrole` role declaration. As in the previous cases, the semantics for an annotation in a particular query is that the annotation holds only for that specific instance. An annotation in a role declaration is a much stronger assumption. It instructs CPSA to add the instance of the assumption *every time* it adds an instance of that role. Since it is a much stronger assumption, it yields less precise analyses.

8.1.1 Yahalom with Channels

We provide two examples of the channel mechanism, both based on the Yahalom protocol. They both use channels with annotations to explain the security properties needed for interaction with the key server. An explicit encryption is present only for the last message, in which the initiator encrypts and returns the responder's nonce, establishing that the

session key has been received and accepted, and was indeed requested by the initiator when starting the session.

In the first version (Section 8.1.2), we "take the easy way out," and incorporate the channel annotations into the role definitions, meaning that CPSA will introduce a new instance of the assumption for every instance of a role that it considers.

In the second version, we use the slower mechanism, and introduce each individual channel property in a declaration in the `defskeleton` query forms. From the latter we learn a kind of *session-independence*: The security properties of a particular session of the protocol depend only in the channels that should be used in that protocol session, and the messages cannot leak from one session to undermine the security goals of another session.

It is perfectly reasonable in a development process to explore the protocol design choices with the channel annotations as assumptions at the role definition level first. When analysis shows that a version is acceptable, then one can remove the role definition annotations, and add assumptions strand-by-strand during a stepwise analysis.

If this analysis yields an acceptable result, for instance the session-independence property, one can then further refine the protocol by incorporating cryptographic mechanisms to ensure the channel properties are met. In the original Yahalom protocol (see e.g. [12]), the channels are implemented via symmetric cryptography. If the initiator and responder each share an uncompromised long term key with the server, then an authenticated encryption mode certainly satisfies the channel properties. However, other mechanisms are also possible, e.g. asymmetric primitives or, indeed, TLS.

Indeed, a choice of a particular primitive early in the design process obscures what actual properties are needed for the protocol to achieve its goals.

Yahalom: Protocol idea. In the Yahalom protocol, the initiator A provides a nonce N_A to the responder B, who passes that along with its own nonce N_B and both names A, B to a key server. The server then creates a fresh session key K for A and B, packaging it separately for A and for B with their names; the package for A also contains N_A, N_B. A completes the exchange by encrypting B's nonce N_B with the session key K.

Yahalom: Channel assumptions. In any session, the server needs a confidentiality guarantee for the packages it sends in order to protect the session key K in that session. So that requires the channel confidentiality property for the outbound channels from the server. Moreover, the initiator and responder each need an authentication guarantee for the packages it receives, so as to know that the key is safe to use.

8.1.2 Yahalom with Channels, 1: Role-level assumptions

We first codify the confidentiality and authenticity channel properties using annotations in the role declarations. This corresponds to the stronger assumption that compliant principals *always* use channels that successfully achieve those security properties. Since

```
(defprotocol yahalom basic
  (defrole init
    (vars (a b c name) (n-a n-b text) (k skey) (ch3 chan))
    (trace (send (cat a n-a))
           (recv ch3 (cat a b k n-a n-b))
           (send (enc n-b k)))
    (auth ch3))

  (defrole resp
    (vars (b a c name) (n-a n-b text) (k skey) (ch1 ch2 chan))
    (trace (recv (cat a n-a))
           (send ch1 (cat a b n-a n-b))
           (recv ch2 (cat a b k))
           (recv (enc n-b k)))
    (auth ch2))

  (defrole serv
    (vars (c a b name) (n-a n-b text) (k skey) (ch1 ch2 ch3 chan))
    (trace (recv ch1 (cat a b n-a n-b))
           (send ch3 (cat a b k n-a n-b))
           (send ch2 (cat a b k)))
    (conf ch3 ch2)
    (uniq-orig k)))
```

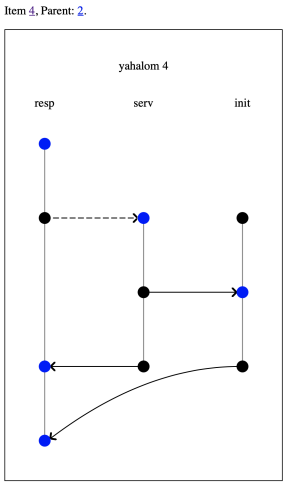
Figure 8.1: Yahalom protocol with channel assumptions in role declarations

the assumption is stronger, any conclusions we reach will be *weaker* conclusions: they are implications that depend on a stronger hypothesis. In Section 8.1.3, we will show how, with a bit of extra work, we can find the tightest relevant hypotheses.

In Fig. 8.1 we show the three roles of the protocol with the behavior and channel assumptions mentioned at the end of Section 8.1.1. CPSA will automatically instantiate the channel assumptions for each role every time it incorporates an instance of the role.

We can now ask CPSA a succession of queries about this protocol. First, if the responder has a complete, four-step local session, then there are corresponding server and initiator runs (Fig. 8.2). All participants agree on the identities A, B, the responder's nonce N_b, and the session key K. Moreover, the server and initiator agree on the initiator's nonce N_A, but the responder does not need to. CPSA also validates that the session key K cannot be disclosed.

When we ask what follows if the initiator has a full local session, the result is more limited. The initiator authenticates a server session of at least length two, agreeing on the identities A, B, the two nonces, and the session key K. Again, CPSA validates that K cannot be disclosed.




```
(defstrand resp 4 (k k) (n-a n-a)
          (n-b n-b) (b b) (a a) ...)

(defstrand serv 3 (k k) (n-a n-a-0)
          (n-b n-b) (a a) (b b) ...)

(defstrand init 3 (k k) (n-a n-a-0)
          (n-b n-b) (a a) (b b) ...)
```

Figure 8.2: The responder's point of view

Observe that if any initiator and responder have local sessions involving a shared key K, then at least the result of Fig. 8.2 must hold, and they agree on each other's identities A, B.

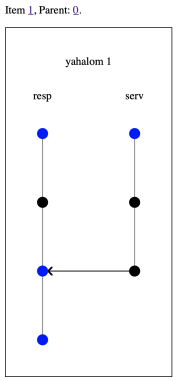
The server receives no authentication guarantees from the protocol as defined here, but is guaranteed that the shared key K will remain undisclosed. This is undesirable from one point of view, as it may cost resources for the key server to generate a new key and package it. Thus, an adversary can force this to occur as desired. This is (presumably) the reason why the original Yahalom protocol encrypts the message from the responder to the server (the arrow that appears dashed in Fig. 8.2). We may achieve the desired effect by stipulating that the server's channel `ch1` is authenticated. Our analysis shows that there is no benefit to confidentiality on that channel.

One might, however, doubt whether there is much benefit to the `(auth ch1)` stipulation, since the adversary can still arrange for messages from legitimate responders to be delivered repeatedly.

We can infer from this analysis that symmetric authenticated encryption long term key shared between the key server and each client is a good way to implement the channels `ch2` and `ch3`, since the key server relies on their confidentiality property and the clients rely on their authentication property.

8.1.3 Yahalom with Channels, 2: Per-strand assumptions

We now remove the role definition annotations, i.e. we omit the `(conf ch3 ch2)` annotation from the declaration of the server, and we omit the `(auth ch2)` and `(auth ch3)` annotations from the responder and initiator declarations, resp. We will now ask a succession of queries, each eliciting some more information, and allowing us to add small




```
(defskeleton yahalom
  (vars (k skey) (n-b n-a n-a-0 n-b-0 text)
                 (a b name) ...)
  (defstrand resp 4 (k k) (n-a n-a) (n-b n-b)
                    (b b) (a a) ...)
  (defstrand serv 3 (k k) (n-a n-a-0) (n-b n-b-0)
                    (a a) (b b) ...)
  (precedes ((1 2) (0 2)))
  (uniq-orig k n-b)
  (auth ch2) ...)
```

Figure 8.3: Responder's first guarantee

assumptions at each stage to identify the individual channel properties that matter. First, we ask:

```
(defskeleton yahalom
  (vars (a b c name) (n-b text) (ch1 ch2 chan))
  (defstrand resp 4 (a a) (b b) (n-b n-b) (ch1 ch1) (ch2 ch2))
  (auth ch2)
  (uniq-orig n-b))
```

This yields the output in Fig. 8.3.

Now that we have ascertained a server run occurred with the same identities A, B and session key K, we may resubmit this output to CPSA with the added annotation `(conf ch2 ch3)`, saying that we will assume that *this* server session successfully used channels with the confidential property. At this point, CPSA completes the analysis with the same shape shown in Fig. 8.2.

This establishes that no other channel assumptions could be relevant to the conclusion. This was not evident from Fig. 8.2, since other channel assumptions may have led CPSA to discard some branches of analysis as impossible, which might then never be displayed.

The analysis from the initiator's point of view can then follow the same pattern.

8.2 State

CPSA models situations in which regular principals control devices with state registers or storage locations—we will henceforth use the term *location* for all such pieces of memory—that allow long term storage and coordination among different local protocol sessions.[2]

[2] Allowing the Dolev-Yao adversary to have devices does not strengthen its powers, as it can already remember unbounded numbers of messages and perform all the reasonable operations on them. This is why our model highlights devices under the control of the regular principals.

Each protocol role determines when a regular principal reads from a device location; what it does with the value read from it, and the parts of the value; and when it writes a new value into a location.

Locations and channels. As we mentioned at the start of the chapter, CPSA relies on the analogy between reading from locations and receiving messages from channels, and between writing to locations and sending messages onto channels. In particular, writing to a location is like sending to a confidential channel, as we are modeling regular principals in control of the devices. And reading a state value is partly like receiving from an authenticated channel: Unless the value read might be the result of device initialization, it must have been written by a previous write action by a regular principal. We say that a value is a *generated state value* when we model it as not a possible result of device initialization. When CPSA knows of a value read from a location that it is a generated state value, it treats this as an authenticated channel reception. It then investigates the possible ways that regular principals may previously have written this state value to the location.

Contrast: Locations are mutable. Channels can deliver the same message more than once, and, since redelivery is never ruled out, effectively always remember their previous contents. However, locations are mutable. After a transition event observes the value in a location and writes a new (possibly related) value into the location, subsequent reads are guaranteed to deliver the new value, not its predecessor. Although any number of read events may observe the same value, a transition event ensures that the new value is the only value available from this location. This fact—that transitions erase the past—is the essential characteristic of mutable state.

"Leads to." When a state transition event n_0 writes a value v into a state location ℓ, and a later observation or transition event n_1 obtains this value v from ℓ, without any other transitions having occurred to ℓ in between, we say that n_0 *leads to* n_1, and we write $n_0 \rightsquigarrow n_1$.

Thus, when $n_0 \rightsquigarrow n_1$ the following conclusions hold:

1. n_0 and n_1 are events on the same location ℓ;
2. n_0 is a transition event;
3. n_0 precedes n_1, also written $n_0 \prec n_1$; and
4. n_1 receives the value that n_0 has written into ℓ.

The event n_1 may just be a state observation rather than a state transition. In the former case, it is a pure read against ℓ. In the latter case, it is the beginning of a transition event, which we think of as consisting of a read of the old value in ℓ followed by a write of the new value into ℓ.

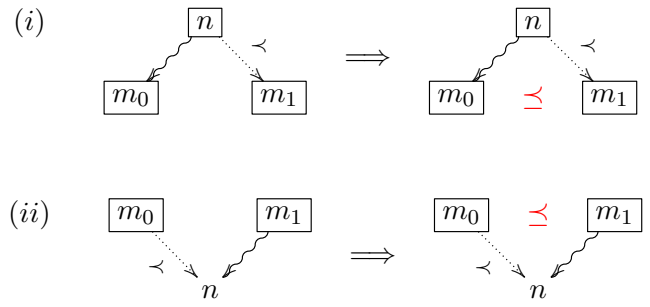


Figure 8.4: Two axioms about state events on the same location ℓ

Internally, CPSA represents a state event as transmitting or receiving a pair $t = (p, v)$. The second component is the state value v being read or written. The first component is a special value which is thought of as a spacetime point p. If the event is a write event, then CPSA assumes p to be uniquely originating. CPSA defines $n_0 \rightsquigarrow n_1$ to hold iff (i) n_0 is a state node; (ii) $n_0 \prec n_1$, and (iii) for some message t, n_0 transmits t and n_1 receives t.

This CPSA representation lets us add to (1)–(4) another conclusion: When $n_0 \rightsquigarrow n_1$,

5. if also $n_2 \rightsquigarrow n_1$, then $n_0 = n_2$.

8.2.1 The axioms of state

The mutating, past-erasing property of transitions is expressed in two basic properties of the leads-to relation. They say in essence that leads-to is *discrete* for transitions events on any one location.

We write them diagrammatically in Fig. 8.4, using a boxed variable $\boxed{m}$ to represent a transition event, and a plain variable m to represent an event that may be either a transition or a pure observation. We use a wavy arrow to represent leads-to; a dotted arrow to represent the precedence ordering $\prec$; and $\preceq$ to express the weak ordering, i.e. $n_0 \preceq n_1$ holds if and only if $n_0 \prec n_1$ or $n_0 = n_1$.

In each case, the events n, m_0, m_1 are all state events on the same location ℓ.

Axiom (i) states that when $n \rightsquigarrow m_0$ and $n \prec m_1$, all of these being transition events on ℓ, then $m_0 \preceq m_1$. Thus, the leads-to relation is a discrete step, and if $m_0 \neq m_1$, then the latter comes definitely later.

We call axiom (i) the **discrete-after** axiom, since it forces m_1 to come after a transition that is the target of the $\rightsquigarrow$ edge.

Axiom (ii) states that when $m_0 \prec n$ and $m_1 \rightsquigarrow n$, each m_i being a transition on ℓ, then $m_0 \preceq m_1$. Again, the leads-to relation is a discrete step, and if $m_0 \neq m_1$, then the former comes definitely earlier.

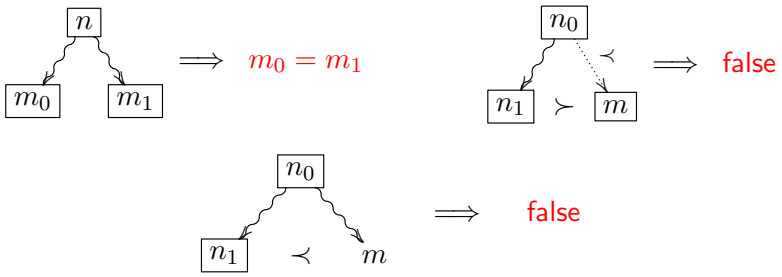


Figure 8.5: Three derived rules for state: Scissors, no interruption, and cake

We call axiom (ii) the **discrete-before** axiom, since it forces m_0 to come before a transition that is the source of the $\rightsquigarrow$ edge.

The symmetry between these two principles is better brought out if we omit to state, in (i), that n is a transition. The source of a leads-to edge is always a transition. The common idea is that a weak ordering holds between m_0 and m_1, whenever one is a connected to n by $\rightsquigarrow$ and the other by $\prec$. They jointly say: a transition ordered with respect to one end of a leads-to arrow has the corresponding weak ordering with respect to the other end.

8.2.2 Consequences of the axioms

What is distinctive of state—over and above channel confidentiality and channel authenticity for generated state values—is captured in the *discrete-after* and *discrete-before* axioms. However, each of them has a disjunction as its conclusion, namely either $m_0 = m_1$ or $m_0 \prec m_1$. Disjunctions are logically less convenient than non-disjunctive conclusions, since separate possibilities must each be explored. Hence, CPSA also formalizes three consequences of the discreteness axioms. One of them has the conclusion $m_0 = m_1$, and the other two have the conclusion *false*. Again assuming that the state events concern the same location ℓ, the rules are:

Scissors rule, which says that when a given node n leads to transition nodes m_0, m_1, the latter two are equal.

Equivalently, a single node leads to at most one transition node.

No interruption, which says that a transition node m is never properly between transition nodes n_0 and n_1 when $n_0 \rightsquigarrow n_1$.

Thus, a transition node m cannot "interrupt" the leads-to edge.

Cake rule, which says that if a node n_0 leads to two other nodes, and one of them is a transition n_1, then the other one m does not follow the transition.

Thus, it says that a transition n_1 caps off the period when other events m (which are pure observations by the scissors rule) may result from n_0.

We present these diagrammatically in Fig. 8.5. The two leads-to edges of the *scissors rule* are like the blades, which the equation $m_0 = m_1$ snaps shut. The cake rule formalizes the fact that you can't eat your cake at n_1 and have it too at m, at least not if m follows n_1.

Again, their concrete form may be found in the output of CPSA specifications using state, such as `examples/open-closed.tst`.

Why these rules are consequences. Each of these rules is easily justified from the discrete-after and discrete-before rules, using the facts that $n \rightsquigarrow m$ implies $n \prec m$ and that $\preceq$ is a (weak) partial order.

Scissors rule: The scissors rule follows by two applications of the discrete-after axiom. Specifically, since $n \rightsquigarrow m_1$ implies $n \prec m_1$, we can apply the discrete-after axiom to infer that $m_0 \preceq m_1$. Symmetrically, observing thay $n \rightsquigarrow m_0$ implies $n \prec m_0$, we can again apply the discrete-after axiom to infer that $m_1 \preceq m_0$.

The antisymmetry of $\preceq$ implies that $m_0 = m_1$.

No interruptions rule: For the no interruptions rule, apply discrete-after to infer that $n_1 \preceq m$. But since $m \prec n_1$ is also assumed, this would yield a cycle.

Cake rule: For the cake rule, apply discrete-before, instantiating its variable m_0 as m, its m_1 as n_0, and its n as n_1. Thus, it follows that $m \preceq n_0$, but the leads-to edge $n_0 \rightsquigarrow m$ implies $n_0 \prec m$, yielding a contradiction.

8.2.3 Syntax for state

In defining the syntax for modeling state, we first describe state events and the state segments that they can build up. We then describe how to specify generated state values—which were mentioned at the beginning of the chapter—and critical sections—which are atomicity assumptions about state segments.

State events and state segments. In addition to the `chan` sort discussed in Section 8.1, there is a sort `locn`. A variable `lv` declared of sort `locn` via a declaration such as `(lv locn)` may be used as the target of `load` and `stor` events; these events indicate that a value is to be read from a location or written to it. Thus, Fig. 8.6, which uses a number of operators from a `lang` field,[3] shows a role that manipulates two variables `lk, ls` of sort `locn`. Its intent is to open a door in response to a suitable encrypted command, conforming its action. The form `(lk ls locn)` declares the variables of sort `locn`. The form `(load lk`

[3]See Section 4.4. In this example, `dev-key-state`, `open-req`, `door-state`, and `opened` are tuple operators, and `hash-dk` is a hash operator, in essence a key derivation function. This example is found in `examples/open-closed.scm`.

```
(defrole dev-open
    (vars (k skey) (n nb text) (any mesg) (d o b name) (lk ls locn))
    (trace
     (load lk (dev-key-state d o k))
     (recv (cat b n (enc (open-req b d o nb n) (hash-dk (cat b nb n k)))))
     (load ls (door-state d any))
     (load lk (dev-key-state d o k))         ; check k unchanged
     (stor ls (door-state d (opened b nb n)))
     (send (hash (open-req b d o nb n))))
    (gen-st (dev-key-state d o k))
    (facts (same-dev ls lk)))
```

Figure 8.6: Device-open role causing door to open

`(dev-key-state d o k))` loads a tuple including a key `k` from the `locn` `lk`, which must contain a tuple of this kind for the event to occur. After receiving a message that contains a request to open the door, it loads the current door state from the other location `ls`, after which it stores back the "open" value. These events might be implemented by a sensor and an actuator in a real device. Finally, it confirms success by sending the hash of the open request.

A `load` or `stor` event always contains a location variable followed by a message term, just like a channel `send` or `recv` (Section 8.1).

Observe that the `stor` event immediately follows a `load` event, and this is in fact always the case. Since together they represent a transition that consumes an old value that will no longer be available after the the transition, the `stor` event always follows a paired `load` event. Together, they form a *state segment*. However, state segments may be longer than a single pair of events.

In general, a state segment consists of one or more `load` events followed by zero or more `stor` events, subject to the requirements that:

1. no two `load` events in this state segment load from the same location;
2. no two `stor` events in this state segment store into the same location;
3. each `stor` event stores into a location from which a value was `load`ed in this state segment.

A particular state segment is a *transition for* a location ℓ iff there is a `stor` into ℓ in this state segment. It is a *(pure) observation for* ℓ iff there is a `load` from ℓ but no `stor` into ℓ in this state segment.

We use state segments of length greater than two to represent events that involve multiple locations. If a transition for location ℓ_1 depends on the value present in ℓ_2, then a state segment can load v_1 and v_2 from ℓ_1 and ℓ_2 (resp.) and then store a new value

dependent on v_1, v_2 into ℓ_1. In this case, the state segment is a transition for ℓ_1 but a pure observation for ℓ_2. If instead new values are stored into both ℓ_1 and ℓ_2, then the state segment is a transition for both. In other cases, a state segment loads from both ℓ_1 and ℓ_2, but instead of storing back to either of them, it simply uses those values to control future message transmissions and receptions. In the latter cases, the state segment is a pure observation for both ℓ_1 and ℓ_2.

Rules for protocols with state. CPSA generates three types of rules for protocols that manipulate state, in addition to the two axioms of Section 8.2.1 and the three consequences of Section 8.2.2.[4]

The first kind are the *transition rules*, and these are generated entirely automatically by CPSA. The axioms and rules of Sections 8.2.1–8.2.2 depend on which events are transition events, the boxed events in Figs 8.4–8.5. Moreover, the syntax of the state segments determines which these are. Thus, CPSA generates rules that imply that an event is a transition event if:

1. it is a `stor` event; or
2. it is a `load` event, and the strand is long enough also to have a `stor` event from this state segment for the same location ℓ.

These are the transition rules.

The second kind are the *generated state value rules*. As we mentioned at the start, a generated state value is a value v that is definitely non-initial. Thus, if a `load` event reads a generated state value v from a location ℓ, then ℓ is acting like an authenticated channel for v. In particular CPSA will consider which instances of the roles could previously have stored v into ℓ; one of those must be present.

A generated state assumption may be introduced as a per-strand assumption, akin to the declarations discussed in Section 8.1.3. In that case it is syntactically present in a CPSA `defskeleton` query such as:

```
(defskeleton open-closed
  (vars (k skey) (d o name))
  (defstrand dev-pass 4 (k k) (d d) (o o))
  (gen-st (dev-key-state d o k)))
```

This `gen-st` declaration says that the particular tuple `(dev-key-state d o k)` using the values of the variables that appear in this strand forms a generated state value.

A generated state assumption may be introduced as a role-level assumption, akin to the declarations discussed in Section 8.1.2. In this case, the specifier wants to tell CPSA that the declared value is a generated state value whenever an instance of the role is long enough to furnish values for the variables that appear in it. Fig. 8.6 contains a role-level generated

[4] See Section 7.7 for rules in CPSA.

```
(defrole dev-up
    (vars (k skey) (d o name) (old old1 mesg) (ch chan) (lk ls locn))
    (trace
     (recv ch (cat "power-up" d o k))
     (load lk old)
     (load ls old1)
     (stor lk (dev-key-state d o k))
     (stor ls (door-state d (closed o)))
     (send (enc "up" k)))
    (auth ch)
    (facts (same-dev ls lk)))
```

Figure 8.7: State segments are critical sections

state assumption, namely `(dev-key-state d o k)`. Since the trace of the `dev-open` rule loads a value of this form as its first action, these variables have values in every instance of height ≥ 1.

CPSA enforces this role-level assumption by generating a rule stating:

```
(defgenrule gen-st-dev-open-0
    (forall ((z strd) (k skey) (o d name))
      (implies
        (and (p "dev-open" z 1) (p "dev-open" "k" z k)
             (p "dev-open" "o" z o) (p "dev-open" "d" z d))
        (gen-st (dev-key-state d o k)))))
```

CPSA uses the `defgenrule` form to distinguish rules it has generated automatically from rules that were explicitly written by the specifier, as described in Section 7.7.

Finally, the third kind of rule concerns *critical sections*. These are generated automatically for each state segment. These generated rules imply that any node that precedes one of the events in a state segments must precede all of them. Any node that follows one of the store events must follow all of them. Thus for instance in Fig. 8.7, the load events at indices 1, 2 and the store events at indices 3, 4 occur with transaction-like atomicity. CPSA generates the rules in Fig. 8.8. They are of value only when the state segment has multiple loads or multiple stores. In this particular example, the critical section declaration does not in fact make a difference to the analysis.

8.2.4 Channels and state: Summary

A core idea is that receptions may have *authenticity*, meaning that the adversary has not delivered the message; thus, CPSA explores what regular strands may have transmitted it. Dually, a transmission may have *confidentiality*, meaning that the adversary will not receive the message directly. Thus, any parts of the transmitted message that appear in new forms

```
(defgenrule cau-dev-up-2
  (forall ((z z1 strd) (i indx))
    (implies
      (and (p "dev-up" z 3) (prec z1 i z 2))
      (or (= z z1) (prec z1 i z 1)))))

(defgenrule eff-dev-up-3
  (forall ((z z1 strd) (i indx))
    (implies
      (and (p "dev-up" z 4) (prec z 3 z1 i))
      (or (= z z1) (and (p "dev-up" z 5) (prec z 4 z1 i))))))
```

Figure 8.8: Critical section: Generated rules

subsequently will require some regular strand to have received it and transformed these parts.

In the channel mechanism, CPSA allows these properties to be directly associated with channels, providing a useful technique for protocol design. In the state mechanism, CPSA models the *confidentiality* of storing a value into a device controlled locally by the regular principal(s). Moreover, *stored* state values with the `gen-st` property satisfy the *authenticity* condition also.

Besides these properties, mutable state also requires modeling the fact that an old state value can no longer be observed in a location after that location has undergone a transition. We express these in the two axioms of Fig. 8.4; CPSA also uses their three consequences as shown in Fig. 8.5 for efficiency.

The modeling distinguishes between *transitions* on a location, which are expressed in state segments that contain a `stor` to it, and *observations*, expressed in state segments that contain a `load` but no `stor` to it. CPSA determines which events are transitions on a location automatically, and generates rules to keep track of this information. Rules are also available to determine which state values are *stored* state values, and these are generated from `gen-st` declarations.

CPSA's job is to enumerate shapes. When a specification distinguishes a kind of value that can only appear in an initial `load`, and transitions that do not produce these values, but always produce *stored* state values with the `gen-st` property, then a run can typically have 0, 1, 2, ..., n transitions, and these will have no homomorphisms between them. Hence there will be infinitely many distinct shapes to enumerate, and CPSA will not terminate, although its partial outputs are often informative. The craft of using CPSA to understand state in protocols is often to restrict the `gen-st` declarations so that CPSA does not distinguish among state histories of (irrelevantly) different lengths.

Part IV

Reference material

Chapter 9

Troubleshooting

The CPSA tool is a complicated one and many errors are possible in its use. In this chapter we discuss these errors, from the simplest to the most complex, and offer suggestions as to how to resolve them.

9.1 Non-termination

The CPSA tool is not guaranteed to complete its search on all well-formed inputs. The problem space CPSA attempts to perform includes some Turing-undecidable problems.

Because of this, the tool has two bail-out conditions that users should be aware of:

- The **strand bound** causes the tool to abort its analysis if any skeleton it analyzes has more strands than the bound. By default, the strand bound is 12.
- The **step limit** causes the tool to abort its analysis if during the analysis of a single input `defskeleton` or `defgoal`, the number of skeletons it processes exceeds the limit. By default, the step limit is 2000.
- The **depth limit** causes the tool to not analyze skeletons more steps away from the initial point of view than the bound. There is no depth limit by default. Skeletons that are unrealized and not analyzed due to the depth limit are marked with "`(fringe)`".

If you execute an analysis and the tool says "Strand bound exceeded" or "Step limit reached," then that bail-out condition has come into play. This may indicate an analysis that would never terminate, but it may also be the case that the strand bound or step limit is too small, and a larger one will enable the analysis to complete.

Unlike the strand bound and the step limit, the depth limit never triggers an error condition, and can thus be useful for multi-skeleton analyses in which one of the earlier skeletons would otherwise have a non-terminating analysis.

The step limit, depth limit, and strand bound can be adjusted through the `limit`, `depth`, and `bound` options, respectively. See Section 10.5.

Note that sometimes, a user may become impatient waiting for an analysis to either complete or bail out. When this happens, the user should not hesitate to interrupt the tool; the tool will output a partial result that can be graphed so the user can examing the analysis done so far.

An analysis that doesn't terminate does not necessarily represent an insecure protocol, it may just indicate a protocol where a more clever analysis is required than CPSA's automated one.

9.1.1 Tweaking the search

There are cases in which the default CPSA analysis does not terminate, but a non-default analysis would terminate. The tool has several settings that influence the search but can be tweaked:

- **Node precedence.** In a skeleton with multiple unrealized receptions, the tool will, by default, focus on the topmost unrealized node of the rightmost strand that contains an unrealized node. If you find that an analysis gets into a large search space due to exploring those unrealized receptions first, you could alter this order with the `reverse-nodes` or `try-old-strands` options. The latter will prioritize the leftmost strands over the rightmost, while the former will prioritize the bottom-most unrealized node in the strand rather than the topmost.

- **Critical term precedence.** Occasionally, a reception will arise that is unrealized and multiple critical terms are available. In particular, there are cases where a term contains both a hard-to-explain encryption and a restricted nonce. For instance, in the Kerberos protocol, the ticket $\{|k, a, b|\}_{SK(b,s)}$ can serve as both a critical encryption (because $SK(b, s)$ may be declared non-originating) and a critical term (because k is uniquely originating). By default, CPSA will treat the encryption as the critical term because this tends to lead to learning more in fewer steps, but this choice can be reversed by using the `check-nonces` option.

- **Priority.** The tool contains an ability to declare a priority for certain receptions that differs from the default. Priority takes precedence over all other search orderings. See Section 10.4 for the format requirements for declaring priorities, and the `priority_test.scm` example in the examples directory.

  Note that the default priority is 5, and priority 0 indicates that the tool should never bother solving tests at those nodes. This may be of use, for instance, if solving one particular node leads to infinite analysis, but other nodes would result in a quick determination that a skeleton is dead.

9.2 Error messages

In this section, we provide an alphabetical listing of error messages / failures that may arise during CPSA execution. If you get an error message not included here, it likely represents a bug and should be reported to the tool maintainers.

- **"[ASSERT FAILED] [...]"**. This kind of error should not occur. If you see this happen, please contact the tool maintainers and make a bug report!
- **"Aborting after applying 500 rules and more are applicable"**. This most likely indicates a circular use of rules.
- **"Algebra.absenceSubst: bad absence assertion"** or **"Algebra.nullifyOne: unexpected pattern"** The `absent` declaration must be declared on a pair where the first element is an RNDX variable and the second element is an exponent. These errors should not occur if you did not give CPSA an input with an absent declaration.
- **"Algebra.inv: Cannot invert a variable of sort mesg"**. Variables of the `mesg` sort should never be used as the key in an encryption. CPSA uses a single function symbol to represent both symmetric and asymmetric encryption, and when the key is a variable of sort `mesg`, it is ambiguous which is meant. As a result, it is unclear what the decryption key would be for such a message. When CPSA tries to calculate the decryption key when the encryption key is a variable of sort `mesg`, this error is produced.
- **"Atom not unique at node"**. This occurs when a formula has been specified including a `uniq-at` predicate in the antecedent that is untrue.
- **"Bad char [...]"**. This error message comes from a low-level parser trying to understand S-expressions. When parsing an S-expression, any non-whitespace that isn't a parenthesis is an "atom" but we expect atoms to be symbols, numbers, or quoted strings, and only certain characters are allowed in these. An atom that starts with a digit is expected to be a number, for instance, so subsequent non-digits will cause an error of this kind. The characters allowed in symbols include all alphanumeric characters and the following punctuation marks: `+`, `-`, `*`, `/`, `<`, `=`, `>`, `!`, `?`, `:`, `$`, `%`, `_`, `&`, `~`, `^`.
- **"Bad height" / "Bad position in role" / "Negative position in role"**. A `defstrand` includes a specification of a height (the length of the instance) but that height must be positive and must not exceed the length of the role.
- **"Bad str-prec"**. Your goal included a `str-prec` predicate among node variables associated with different roles. In other words, your formula has attempted to make a single strand that includes events from distinct roles.

- **"Close of unopened list"**. Your input has an erroneous close-paren.
- **"Disallowed bare exponent"**. See Section 4.2. The tool requires that within roles and skeletons, exponents occur only inside an exponentiation function.
- **"Domain does not match range"**. This error message occurs when CPSA is trying to understand the variable assignment you have specified in a `defstrand`. You may have defined the value of a parameter more than once, or your definition may have a type mismatch. For instance if a is a parameter role expected to be of the name type, and you declare t to be a text variable, then including `(a t)` in a defstrand will produce this error.
- **"Duplicate role [...] in protocol [...]"**. Fairly self-explanatory: the roles in a protocol must have distinct names. This error occurs if you have two protocols with the same name.
- **"Duplicate variable declaration for [...]"**. Fairly self-explanatory: within any `vars` statement, any symbol may be used for a variable name, but each variable name can be declared only once.
- **"End of input in string"**. You included a quote-delimited string but didn't close it before the end of the input file.
- **"Equals not allowed in antecedent"**. The `equals` predicate may only be used on the conclusion side of a `defgoal`.
- **"Expansion limit exceeded"**. This most likely indicates a circular use of macros. The limit of expansion of a macro within a macro is hard-coded in the tool as depth 1000.
- **"Expecting [...] to be [a/an ...]"**. You have a type error in your use of a function symbol. For instance if `(pubk a)` is to be loaded within a particular variable declaration scope, the a variable should be of the NAME sort.
- **"Expecting a node variable" / "Expecting an algebra term"**. Certain predicates within a `defgoal` expect one of their inputs to be a declared node variable (or to be a non-node variable). If a variable used in such an input is declared otherwise, this error message is produced.
- **"Expecting an atom"**. Certain declarations, in particular the `uniq-orig`, `uniq-gen`, and `non-orig` ones, are expected to be used on atomic terms rather than compound ones.
- **"Expecting terms in algebra [...]"**. The tool actually expects to know the message algebra to use up front, before it begins parsing. The algebra is the basic one

by default, or you may specify through a command-line argument or a herald to use the Diffie-Hellman algebra. Each `defprotocol` in the input specifies an algebra to use, and this error occurs when that algebra doesn't match the one CPSA is prepared to parse. To resolve: check that you aren't requesting the wrong algebra, and check that you have properly spelled the name of the algebra in your `defprotocol`.

- **"Identifier [...] unknown"**. This is a relatively common user-caused error that occurs when you try to use a variable not declared in your `vars` declaration.

- **"Include depth exceeded with file [...]"**. Most likely, this indicates a circular use of the `include` command. The limit of inclusion within an included file is depth 16.

- **"Keyword [...] unknown"**. The tool was expecting the symbol to specify an algebra function symbol, but it didn't match any of the available ones. This most commonly indicates that the user forgot to include a function symbol name at the beginning of a list when describing a term. One of the most common forms of this mistake is to include `(send (a b))` in a trace of a role, when the user intended to model the sending of the pair (a, b). The proper input would be `(send (cat a b))`.

  Because of this type of mistake, it is recommended to avoid using variables in your model that are the same as function symbol names such as "ltk" or "pubk".

- **"In a rule equality check, cannot find a binding for some variable"**. An equality in a rule is receiving a variable that has not been bound by a length or parameter predicate. Try moving the equality to the end of the conjunction in which it occurs.

- **"In rule [...], parameter predicate for [...] did not get a strand"**. This message occurs when a strand variable is not bound by a length predicate.

- **"In rule [...], [...] did not get a strand"**. This message occurs when a strand variable is not bound by a length predicate.

- **"In rule [...], [...] did not get a term**. This message occurs when an algebra variable is not bound by a parameter predicate.

- **"Malformed [...]"**. Generally speaking, this indicates a syntax error. Consult the grammar in Chapter 10 for the syntax requirements for the type of object the tool claims was malformed. Double-check that you have spelled required keywords correctly, and that your parentheses are matched.

- **"Malformed association list"**. This refers to one of the "-alist" symbols in the grammar; these may occur in skeletons, goals, protocols, or roles.

Association lists are lists of S-expressions, each of which is a list that starts with a symbol. This error would occur if you had, for instance, a symbol or a number, or an S-expression starting with a number as an input to a `defrole` or `defskeleton`.

- **"Malformed input"**. Top level S-expressions in your input file must be one of the following: `defprotocol, defskeleton, defgoal, comment,` or `herald`. The tool also recognizes `defpreskeleton` as a synonym for `defskeleton`. If you have an S-expression at the top level that is other than one of these, this is the error message you will see.

- **"Malformed pair – nodes in same strand"**. In a `defskeleton` you are prohibited from specifying orderings between nodes in the same strand.

  This is not the case for `leadsto` relationships.

- **"No strands"**. Your `defskeleton` did not include any strands at all; it must include at least one.

- **"Node occurs in more than one role predicate"**. Node variables in a goal must occur within a role position predicate, but should not occur within more than one within their defined scope.

- **"Priority declaration disallowed on [...]"**. Prioritization has no effect except on events that need an explanation. If you try to change the default priority of a send or state initialization event, this is assumed to be a mistake and the tool produces this error.

- **"Protocol [...] unknown"**. This error occurs when you have a `defskeleton` or `defgoal` with a protocol name not matching any `defprotocol` so far present in the file.

- **"Role [...] not found in [...]"**. You included a `defstrand` referencing a role that does not exist in the protocol definition.

- **"Role in parameter pred differs from role position pred"**. A node variable in a formula should occur in a role position predicate but may also occur in node parameter predicates. However, node parameter predicates for a given node variable should match the role of the role position predicate the variable occurs in.

- **"Role not well formed: role trace is a prefix of a listener"**. CPSA disallows the use of roles that begin with the reception of some message followed by the transmission of that same message, because there is an ambiguity as to whether an instance is a listener or an instance of a protocol role. This should not be a problem because beginning a role in this manner is quite unusual, but if it is necessary to you

to do so we recommend the reception be paired with a tag constant such as `(cat "regular role" [...])`.

- **"Role not well formed: non-orig [...] carried"**. The `non-orig` declaration specifies that a certain atomic value not be carried (see Section 6.2). You have made such a declaration but a plain (full-height) instance of your role violates the rule.

- **"Role not well formed: uniq-orig [...] doesn't originate"**. The `uniq-orig` declaration states not only that the declared value originates on a regular strand uniquely (see Section 6.2), but also states that the apparent origination point is the unique origination point of that value. As such, you may only use the `uniq-orig` declaration on a value that does originate somewhere. If you declare a value `uniq-orig` on a role but it does not originate on that role, you get this error.

- **"Role not well formed: variable [...] not acquired"**. Variables of the MESG sort must be "acquired" when used in roles. This means that the first occurrence of the variable must be a *carried* occurrence in a reception event. See Section 6.2 fr an explanation of "carried."

- **"Role not well formed: variable [...] not obtained"**. Variables of the EXPR sort must be "obtained" when used in roles, meaning that the first occurrence must be in a reception.

- **"[Role / Skeleton] not well formed: inequality conditions violated"**. A `neq` declaration is false where it is first declared: in a role or skeleton definition.

- **"[Role / Skeleton] not well formed: lt declarations form a cycle"**. The `lt` declarations present in a role or in a skeleton are already violated in the role or skeleton definition.

- **"[Role / Skeleton] not well formed: subsort requirements violated"**. The `subsort` declarations present in the role or skeleton being defined are already violated.

- **"Skeleton not well formed: a variable in [...] is not in some trace"**. A `defskeleton` causes this error when a variable used in a declaration does not appear in any of the traces.

- **"Skeleton not well formed: cycle found in ordered pairs"**. The ordering edges (strand succession plus ordered pairs) of a skeleton should form an acyclic graph. A cycle represents circular causality which should not be possible in any real execution.

- **"Skeleton not well formed: non-orig [...] carried"**. The `non-orig` declaration specifies that a certain atomic value not be carried (see Section 6.2). You have made such a declaration but your `defskeleton` violates the rule.

- **"Skeleton not well formed: ordered pairs not well formed"**. This error occurs when an ordering is specified between the wrong types of events. In CPSA, an ordering must be such that the earlier node has an outgoing type, so for instance an ordering directly between two reception events is disallowed.

- **"Skeleton not well formed: uniq-orig [...] doesn't originate"**. The `uniq-orig` declaration states not only that the declared value originates on a regular strand uniquely (see Section 6.2), but also states that the apparent origination point is the unique origination point of that value. As such, you may only use the `uniq-orig` declaration on a value that does originate somewhere. If you declare a value `uniq-orig` on a skeleton but it does not originate in the skeleton, you get this error.

  Similarly, **"...: uniq-gen [...] doesn't generate"** represents a detected error in that `uniq-gen` states that not only does the given value generate (see Section 6.2) uniquely, but that its apparent generation point is that generation point. As such, a generation point is expected.

- **"Sort [...] not recognized"**. You attempted to declare a variable to be of a sort not present in the algebra. Check to ensure that if you are using Diffie-Hellman-related sorts that you are using the `diffie-hellman` algebra.

- **"Terms in [role/skeleton] not well formed"**. This error occurs when you have constructed a term using a function symbol that expects inputs of a certain sort, but your inputs are not of that sort. For instance, in `(ltk a b)`, `a` and `b` must be variables of the `name` sort, or they are not well-formed. To resolve: double-check your variable declarations and your use of function symbols.

  This may also occur if you use the `node` sort in a role or skeleton; that sort should only be used in a goal declaration.

- **"Too many locations in declaration"**. You have a native declaration that appears to include two or more locations in it. All native declarations allow at most one location.

- **"Type mismatch in equals"**. The `equals` predicate in a `defgoal` can be used to compare node variables or to compare algebra variables, but cannot be used to compare node variables to algebra variables.

- **"Unbound variable in [...]"**. In a `defgoal`, variables must meet specific binding requirements. See Chapter 7 for details. This error indicates that you have provided a formula that the tool rejects for this reason.

- **"Unexpected end of input in list"**. One of the most frequent user errors - you didn't include close parens for all your S-expressions.

Chapter 10

CPSA Program

10.1 CPSA pre-processing

The CPSA tool performs a pre-processing step before it interprets its input. There are two important features that take place during pre-processing: macros and file inclusion.

File inclusion. CPSA input files that become large enough become unweildy, and the user may wish to break them down into logical components and include one file in another. For instance, one might wish to separate protocol definitions out so they can be reused, or a user might wish to put together a file of macros they find useful. To include one file in another, add `(include "filename")` as a top-level S-expression where you wish that file to be included. Inclusion recognizes only relative paths.

Macros. The CPSA pre-processor interprets macros.

To define a macro in a CPSA input file, use the `defmacro` keyword in an S-expression. The `envelope.scm` file in the examples directory uses the following macro:

```
;; This is the refusal token
(defmacro (refuse n pcr v k aik)
  (enc "quote" (extend "refuse" (extend n pcr)) (enc v k) aik))
```

The first input to the defmacro defines the format that will trigger the macro. In this case, the first macros are defined for an S-expression with keyword `refuse` and five additional inputs. The second input to the defmacro describes what the macro should be replaced with. Symbols that exactly match the subsequent symbols in the first input are interpreted as standing for the inputs when the macro is used. So for instance `(refuse a b c d e)` would be replaced by

```
(enc "quote" (extend "refuse" (extend a b)) (env c d) e)
```

wherever it appears in what follows. Macros can call on other macros, but there is a depth limit to the amount of recursion that this can entail. In the example, `extend` is actually another macro.

Normally a `defmacro` will replace a symbol with a single S-expression, but the `^` (splice) keyword can be used to indicate that a macro should be replaced with more than one S-expression. This may be of use, for instance, to describe a portion of a role's trace, when defining multiple roles with some behavior in common. For instance:

```
(defmacro (handshake n a b)
  (^ (send (enc "hello" a b n (pubk b)))
     (recv (enc "hello-received" a n (pubk a)))))
```

Note that the pre-processor actually handles the splice keyword as a separate pre-processing step after macro expansion. For this reason, use of `^` outside of macros can produce unanticipated behavior.

10.2 CPSA input syntax

The complete syntax for the analyzer using the Basic Crypto Algebra is shown in Table 10.1. The start grammar symbol is FILE, and the terminal grammar symbols are: (,), SYMBOL, STRING, INTEGER, and the constants set in typewriter font.

The ALIST, PROT-ALIST, ROLE-ALIST, and SKEL-ALIST productions are Lisp style association lists, that is, lists of key-value pairs, where every key is a symbol. Key-value pairs with unrecognized keys are ignored, and are available for use by other tools. On output, unrecognized key-value pairs are preserved when printing protocols, but elided when printing skeletons.

The contents of a file can be interpreted as a sequence of S-expressions. The S-expressions used are restricted so that most dialects of Lisp can read them, and characters within symbols and strings never need quoting. Every list is proper. An S-expression atom is either a SYMBOL, an INTEGER, or a STRING. The characters that make up a symbol are the letters, the digits, and the special characters in "`-*/<=>!?:$%_&~^+`". A symbol may not begin with a digit or a sign followed by a digit. The characters that make up a string are the printing characters omitting double quote and backslash, except when double quote and backslash are escaped using the backslash character. Double quotes delimit a string. A comment begins with a semicolon, or is an S-expression list at top-level that starts with the `comment` symbol.

FILE	←	HERALD? FORM+
HERALD	←	(**herald** [SYMBOL | STRING] ALIST)
FORM	←	(**comment** ...) | PROTOCOL | SKELETON | GOAL
PROTOCOL	←	(**defprotocol** ID ALG ROLE+ RULE∗ PROT-ALIST)
ID	←	SYMBOL
ALG	←	**basic** | **diffie-hellman**
ROLE	←	(**defrole** ID VARS TRACE ROLE-ALIST)
VARS	←	(**vars** VDECL∗)
VDECL	←	(ID+ SORT)
TRACE	←	(**trace** {EVENT | ANNOT}+)
EVENT	←	(DIR ID? TERM)
DIR	←	**send** | **recv** | **load** | **stor**
ANNOT	←	(**cheq** ID TERM) | (**rely** CONCLUSION) | (**guarantee** CONCLUSION)
RULE	←	(**defrule** ID SENTENCE ALIST)
ROLE-ALIST	←	ROLE-DECL ROLE-ALIST | ALIST ROLE-ALIST
ALIST	←	(SYMBOL ...)? ALIST?
SKELETON	←	(**defskeleton** ID VARS STRAND+ SKEL-ALIST)
STRAND	←	(**defstrand** ID INTEGER MAPLET∗)
	|	(**defstrandmax** ID INTEGER? MAPLET∗)
	|	(**deflistener** TERM)
MAPLET	←	(TERM TERM)
SKEL-ALIST	←	SKEL-DECL SKEL-ALIST | ALIST SKEL-ALIST
	|	(**precedes** NODE-PAIR∗) SKEL-ALIST
	|	(**leadsto** NODE-PAIR∗) SKEL-ALIST
	|	(**goal** SENTENCE+)| (**facts**(ID TERM*))
NODE-PAIR	←	(NODE NODE)
NODE	←	(INTEGER INTEGER)
GOAL	←	(**defgoal** ID SENTENCE+ ALIST)

Table 10.1: CPSA Input Syntax. See Tables 10.3 and 10.5 for algebra syntax (for the TERM and SORT symbols), Table 10.6 for declaration syntax (for the ROLE-DECL and SKEL-DECL symbols), and Table 7.1 for goal syntax (for the SENTENCE and CONCLUSION symbols).

Sorts

Sorts:	NAME, TEXT, DATA, TAG, SKEY, AKEY $<$ MESG CHAN, LOCN $<$ MESG

Operations

$\{\!\lvert \cdot \rvert\!\}_{(\cdot)}$	MESG $\times$ MESG $\to$ MESG	Encryption
$(\cdot, \cdot)$	MESG $\times$ MESG $\to$ MESG	Pairing
$\#(\cdot)$	MESG $\to$ MESG	Hashing
$K_{(\cdot)}$	NAME $\to$ AKEY	Public key of name
$K^s_{(\cdot)}$	NAME $\to$ AKEY	s-Public key of name
$(\cdot)^{-1}$	AKEY $\to$ AKEY	Inverse of key
$\mathsf{ltk}(\cdot, \cdot)$	NAME $\times$ NAME $\to$ SKEY	Long-term key

Constants

$Tags$	TAG	Tag constants

Equations

$(a^{-1})^{-1} = a$	a: AKEY

Derivations

$m_0, m_1 \vDash \{\!\lvert m_0 \rvert\!\}_{m_1}$	m_0, m_1: MESG	Encryption
$m_0, m_1 \vDash (m_0, m_1)$	m_0, m_1: MESG	Pairing
$(m_0, m_1) \vDash \{m_0, m_1\}$	m_0, m_1: MESG	Destructuring
$\{\!\lvert m \rvert\!\}_k, \mathrm{inv}(k) \vDash m$	m, k: MESG	Decryption
$m \vDash \#(m)$	m: MESG	Hashing

Table 10.2: The Basic Cryptoalgebra

ALG	←	`basic`
SORT	←	`text` | `data` | `name` | `tag` | `skey` | `akey`
	|	`chan` | `locn` | `mesg` | ID
ID	←	SYMBOL
TERM	←	ID | (`pubk` ID) | (`privk` ID) | (`invk` ID)
	|	(`pubk` ID STRING) | (`privk` ID STRING)
	|	(`ltk` ID ID) | STRING | (`cat` TERM+)
	|	(`enc` TERM+ TERM) | (`hash` TERM+)
	|	(ID TERM+)

Table 10.3: CPSA Basic Algebra Syntax; sorts, operators ID from LANG field

10.3 Algebra reference

10.3.1 Basic crypto algebra

The basic crypto algebra is an order-sorted algebra with signature described in Table 10.2. The algebra is the free order-sorted algebra generated by the function symbols, sorts, and constants given, modulo the one equation.

Additionally, CPSA reasons about derivability of values from other values, and the basic derivation rules are given in the table, where inv is defined as follows (with $\perp$ indicating "undefined"):

$$\mathrm{inv}(a) = \begin{cases} a^{-1} & \text{if } a : \text{AKEY} \\ a & \text{if } a \text{ is not a variable of sort MESG} \\ \perp & \text{otherwise.} \end{cases} \tag{10.1}$$

Because inv is undefined on variables of sort MESG, CPSA cannot handle protocols or skeletons in which a value is encrypted with such a variable. This is a consequence of the choice we made in the design of CPSA to use only one encryption function symbol, despite there being two forms of encryption (symmetric and asymmetric). When encrypting with a variable of sort MESG, the type of encryption is ambiguous.

Table 10.3 describes the grammar for basic crypto algbera messages in the input syntax.

Each of these function symbols has a specific interpretation in the algebra signature; for instance, `enc` refers to the $\{\!|\cdot|\!\}_{\cdot}$ function symbol, and `pubk` refers to the $K_{(\cdot)}$ function symbol when it has one input, but to $K^{s}_{(\cdot)}$ when it has two inputs, where the second is s. `privk` refers to the composition of either $K_{(\cdot)}$ or $K^{s}_{(\cdot)}$ with $(\cdot)^{-1}$.

10.3.2 The Diffie-Hellman crypto algebra

The Diffie-Hellman crypto algebra is an order-sorted algebra with signature described in Table 10.4. The algebra is the free order-sorted algebra generated by the function symbols,

sorts, and constants given, modulo the equations.

In Section 4.2 we discussed modeling Diffie-Hellman in CPSA, but we said almost nothing about the EXPT sort that features prominently in the algebra signature. Our message model for Diffie-Hellman has two sorts of exponents. The RNDX sort is for exponents that are to be thought of as atomic values. Randomly chosen exponents should always be of this sort, which is why `rndx` appears exclusively in our examples, while `expt` does not. The EXPT sort is for compound expressions involving exponents, or values `(exp (gen) alpha)` that are received with unconstrained exponent. There may also be variables of this sort, but they should not first occur in transmissions in protocol roles.

Table 10.5 describes the grammar for Diffie-Hellman crypto algbera messages in the input syntax. Symbols in the grammar appearing in the grammar for the basic crypto algebra retain their same interpretation. Additionally, the `gen` and `one` symbols, used as 0-ary function symbols, specify the g and 1 constants, respectively.

10.4 Declaration syntax

Table 10.6 gives a grammar for the syntax of native and user-defined declarations. For the purposes of this table, there are two top-level symbols: ROLE-DECL and SKEL-DECL.

10.5 Command-line options

The following command-line options are defined for the CPSA program:

- 'o' ("output") – specifies a file to which the output will be written. Usage: `-o FILE` or `--output=FILE`.
- 'l' ("limit") – specifies a step limit (see Section 9.1). By default, the step limit is 2000. Usage: `-l INT` or `--limit=INT`.
- 'd' ("depth") – specifies a depth limit (see Section 9.1). The depth limit is infinite by default. Usage: `-d INT` or `--depth=INT`.
- 'b' ("bound") – specifies a strand bound (see Section 9.1). By default, the strand bound is 12. Usage: `-b INT` or `--bound=INT`.
- 'm' ("margin") – specifies the number of characters allowed per line in the output. By default, the margin is 72. Usage: `-m INT` or `--margin=INT`.
- 'e' ("expand") – expands macros only. For use in debugging macros; when run with this option, the tool does not analyze its input.
- 'z' ("noanalyze") – loads the input but does not analyze. This could be used for checking whether any errors are present in the input.

Sorts

Sorts:	NAME, TEXT, DATA, TAG, SKEY, AKEY$<$ MESG
	RNDX $<$ EXPT $<$ MESG
	CHAN, LOCN $<$ MESG

Operations

$\{\!\lvert \cdot \rvert\!\}_{(\cdot)}$	MESG $\times$ MESG $\to$ MESG	Encryption
$(\cdot,\cdot)$	MESG $\times$ MESG $\to$ MESG	Pairing
$\#(\cdot)$	MESG $\to$ MESG	Hashing
$K_{(\cdot)}$	NAME $\to$ AKEY	Public key of name
$K^s_{(\cdot)}$	NAME $\to$ AKEY	s-Public key of name
$(\cdot)^{-1}$	AKEY $\to$ AKEY	Inverse of key
$\mathsf{ltk}(\cdot,\cdot)$	NAME $\times$ NAME $\to$ SKEY	Long-term key
$\mathsf{bltk}(\cdot,\cdot)$	NAME $\times$ NAME $\to$ SKEY	Bi-directional LTK
$(\cdot)^{(\cdot)}$	BASE $\times$ EXPT $\to$ BASE	Exponentiation
$(\cdot\cdot)$	EXPT $\times$ EXPT $\to$ EXPT	Multiplication
$i(\cdot)$	EXPT $\to$ EXPT	Mult. Inverse

Constants

$Tags$	MESG	Tag constants
g	BASE	Generator
1	EXPT	Mult. Identity

Equations

$(a^{-1})^{-1} = a$		a: AKEY
$\mathsf{bltk}(a,b) = \mathsf{bltk}(b,a)$		a, b: NAME
$xy = yx$	$x(yz) = (xy)z$	x, y: EXPT
$x1 = x$	$x(i(x)) = 1$	x: EXPT
$h^1 = h$	$(h^x)^y = h^{(xy)}$	h: BASE, x, y: EXPT

Derivations

$m_0, m_1 \vDash \{\!\lvert m_0 \rvert\!\}_{m_1}$	m_0, m_1: MESG	Encryption
$m_0, m_1 \vDash (m_0, m_1)$	m_0, m_1: MESG	Pairing
$(m_0, m_1) \vDash \{m_0, m_1\}$	m_0, m_1: MESG	Destructuring
$\{\!\lvert m \rvert\!\}_k, \mathrm{inv}(k) \vDash m$	m, k: MESG	Decryption
$m \vDash \#(m)$	m: MESG	Hashing
$x, y \vDash xy$	x, y: EXPT	Multiplication
$x \vDash i(x)$	x: EXPT	Inversion
$h, x \vDash h^x$	h: BASE, x: EXPT	Exponentiation

Table 10.4: The Diffie-Hellman Cryptoalgebra

ALG	←	`diffie-hellman`
SORT	←	`text` | `data` | `name` | `tag` | `skey` | `akey`
	|	`rndx` | `expt` | `chan` | `locn` | `mesg` | ID
ID	←	SYMBOL
TERM	←	ID | (`pubk` ID) | (`privk` ID) | (`invk` ID)
	|	(`pubk` ID STRING) | (`privk` ID STRING)
	|	(`ltk` ID ID) | STRING | (`cat` TERM+)
	|	(`enc` TERM+ TERM) | (`hash` TERM+)
	|	(`exp` TERM TERM) | (`gen`) | (`mul` TERM TERM)
	|	(`rec` TERM) | (ID TERM+)

Table 10.5: CPSA Diffie-Hellman Algebra Syntax

ROLE-DECL	←	(ONE-TERM-DECL HT-TERM+)
	|	(`priority` (HEIGHT INTEGER)+)
	|	(`facts` (ID TERM*)*) | (`assume` CONCLUSION*)
ONE-TERM-DECL	←	`non-orig` | `pen-non-orig` | `uniq-orig` | `uniq-gen`
	|	`conf` | `auth` | `gen-st`
HEIGHT	←	INTEGER
HT-TERM	←	TERM | (TERM HEIGHT)
SKEL-DECL	←	(ONE-TERM-DECL TERM+)
	|	(TWO-TERM-DECL (TERM TERM)+)
	|	(`facts` (ID TERM*)*)
	|	(`priority` (NODE INTEGER)+)
NODE	←	(INTEGER INTEGER)

Table 10.6: Declaration syntax. See Tables 10.3 and 10.5 for the algebra syntax, which defines the TERM symbols. See Table 7.1 for formulas CONCLUSION.

- 'n' ("noisochk") – disable isomorphism checks. **Not recommended except in rare circumstances.** When the tool produces a new skeleton, it only gets a number in the output if no isomorphic skeleton has already been found. With isomorphism checking disabled, many analyses that otherwise terminate will not terminate. However, there is one advantage. Normally, when a skeleton is found more than once in an analysis, its operation field (see Section 5.2) is displayed only for the first time that skeleton is found. Other instances are not annotated with an operation field but instead simply link to the isomorphic skeleton.

- 'c' ("check-nonces") – check nonces first. Normally, the tool will consider the largest possible critical term when attempting to solve an unrealized node. This option flips the default behavior. Some analyses terminate only with this option, while others perform better with this option.

- 't' ("try-old-strands") – try old strands first. Normally, when there are unrealized nodes on multiple strands, the tool will focus on the highest-numbered (rightmost in the diagram) strand with unrealized nodes. This option flips the default behavior. Some analyses terminate only with this option, while others perform better with this option.

- 'r' ("reverse-nodes") – try younger nodes first. Normally, when a strand containing unrealized nodes is chosen for analysis, if there are multiple unrealized nodes, the earliest (oldest) node is analyzed. This option flips the default behavior. Some analyses terminate only with this option, while others perform better with this option.

- 'g' ("goals-sat") – stop when goals are satisfied. Normally, a `defgoal` is converted to a skeleton and analyzed, and only shapes are compared to the goals. With this option enabled, the goals are checked against every intermediate skeleton, and branches of the analysis in which the goals are satisfied are not further pursued. The output analysis will show the skeletons in which the goals were first satisfied, and any shapes in which the goals were not satisfied. In a sense, running CPSA with this option asks the question: what are all the ways in which this goal may not be satisfied?

- 'a' ("algebra") – sets the algebra to use for the input. The analysis tool expects to run with a single algebra for an entire session. By default the tool uses the `basic` algebra; an input file using the Diffie-Hellman algebra will cause an error unless you specify the `diffie-hellman` algebra via this option. Usage: `-a STRING` or `--algebra=STRING`.

- 's' ("show-algebras") – causes the tool to print a list of the allowed algebras, instead of performing any analysis.

- 'h' ("help") – causes the tool to print a list of command-line options, and also to print the directory in which the package's documentation exists. When run with this option, the tool will not perform any analysis.
- 'v' ("version") – causes the tool to simply print its version number rather than performing any analysis.

Additionally,

- To protect CPSA against memory exhaustion, run CPSA with the command-line options `+RTS -M512m -RTS`.
- To make use use of parallelism on an n-processor machine, start CPSA with the command-line options `+RTS -N`n `-RTS`.

See the GHC User's Guide [15] for documentation on these and other options.

10.5.1 Heralds

At the top of each CPSA input file is, optionally, a `herald` that is an alternate way to specify command-line options. All of the examples in the examples directory contain heralds. The herald, in addition to setting certain options for the analyzer, is also included in the output file, which may be useful to the analyst in distinguishing one analysis from a similar, related one.

Options are included in the herald by constructing an S-expression with first element equal to the long name of an option, and the second element (if any) set to be the parameter for that option. For instance, `(algebra diffie-hellman)` is included for all the examples that use the `diffie-hellman` algebra.

Command-line options override the herald. Also, certain command-line options are not allowed in the herald, specifically, "output", "help", "version", and "show-algebras".

Chapter 11

CPSA Tools

This chapter describes programs that complement the CPSA program. All the tools accept a `--help` option that describes the tool's usage and a `--version` option that prints the version of the tool.

`cpsa4init`: Populates a directory with files used to create a CPSA project. See Section 2.3.1.

`cpsa4shape`: Cuts down CPSA's output to only show final results—the shapes.

`cpsa4graph`: Constructs an XHTML document that visualizes the output of a CPSA run or the output of `cpsa4shape`.

`cpsa4goalsat`: Prints the labels of skeletons that do not satisfy a `defgoal`. See Section 7.1. For each point of view skeleton specified by a goal, the program prints a list. The first element of the list is the label of the point of view skeleton. The remaining elements of the list are labels of the skeletons that do not satisfy the goal.

`cpsa4prot`: Translates protocols expressed in Alice and Bob notation into CPSA's `defprotocol` syntax. It translates `defprot` forms, and passes all else through unchanged. The syntax of `defprot` forms is presented in Table 11.1.

The basic layout of a `defprot` is a chronological description of things that happen in the intended execution of the protocol. The s-expression `(msg A B M)` is used to indicate that role `A` sends `M`, while role `B` receives it. Other features of `cpsa4prot` are given in Table 11.2.

See Figure 11.1 for an example.

`cpsa4sas`: Generates Shape Analysis Sentences from the output of CPSA. See Section 7.6.

`cpsa4pp`: Pretty prints CPSA input. It can also be used to translate S-expression syntax into JSON for use with tools written in other languages such as Python.

PROT	←	(**defprot** ID ALG VARS FORM∗ RULE∗ PROT-ALIST)
VARS	←	(**vars** VDECL∗)
FORM	←	(**msg** ROLE ROLE CHMSG)
	|	(**msg** ROLE ROLE CHMSG CHMSG)
	|	(**from** ROLE CHMSG)
	|	(**to** ROLE CHMSG)
	|	(**assume** ROLE ROLE-ALIST)
	|	(**clone** ROLE ROLE)
CHMMSG	←	(**chmsg** ID TERM) | (**chmsg** TERM) | TERM

Table 11.1: `defprot` Grammar

Form	Meaning
(`msg A B M`)	`A` sends `M` to `B`
(`msg A B M N`)	`A` sends `M` and `B` receives it as `N`
(`from A M`)	`A` sends `M`
(`to A M`)	`A` receives `M`
(`assume A M`)	Add the assumption `M` to the role `A`
(`clone A B`)	Create a role `B` which includes all behavior described for `A` to this point.

Table 11.2: `defprot` commands

`cpsa42latex`: Translates CPSA macros specifying message formats into equivalent LaTeX macros. Algebra primitives such as `enc` and `pubk` are mapped to unspecified LaTeX macros with the same name and arity (sometimes with an optional tag), allowing the user to easily customize how they would like these primitives to be typeset.

`cpsa4json`: Translates CPSA input in JSON syntax into S-expression syntax.

`cpsa4diff`: Compares the skeletons in two files. If a difference is detected, it prints the first difference and exits.

```
$ cat blanchet.prot
(defprot blanchet basic
  (vars (a b akey) (s skey) (d data))
  (msg init resp (enc (enc s (invk a)) b))
  (msg resp init (enc d s))
  (assume init (uniq-orig s))
  (assume resp (uniq-orig d))
  (comment "Blanchet's protocol"))
$ cpsa4prot blanchet.prot
(defprotocol blanchet basic
  (defrole init
    (vars (a b akey) (s skey) (d data))
    (trace (send (enc (enc s (invk a)) b)) (recv (enc d s)))
    (uniq-orig s))
  (defrole resp
    (vars (a b akey) (s skey) (d data))
    (trace (recv (enc (enc s (invk a)) b)) (send (enc d s)))
    (uniq-orig d))
  (comment "Blanchet's protocol"))
$
```

Figure 11.1: `defprot` Example

Bibliography

[1] W. Diffie and M. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, November 1976.

[2] Shaddin F. Doghmi, Joshua D. Guttman, and F. Javier Thayer. Searching for shapes in cryptographic protocols. In *Tools and Algorithms for Construction and Analysis of Systems (TACAS)*, number 4424 in LNCS, pages 523–538, 2007.

[3] Daniel Dolev and Andrew Yao. On the security of public-key protocols. *IEEE Transactions on Information Theory*, 29:198–208, 1983.

[4] Joseph A. Goguen and José Meseguer. Order-sorted algebra I: equational deduction for multiple inheritance, overloading, exceptions and partial operations. *Theoretical Computer Science*, 105(2):217–273, 1992.

[5] Joshua D. Guttman. State and progress in strand spaces: Proving fair exchange. *Journal of Automated Reasoning*, 48(2):159–195, 2012.

[6] Joshua D. Guttman. Establishing and preserving protocol security goals. *Journal of Computer Security*, 22(2):201–267, 2014.

[7] Joshua D. Guttman, Jonathan C. Herzog, John D. Ramsdell, and Brian T. Sniffen. Programming cryptographic protocols. In Rocco De Nicola and Davide Sangiorgi, editors, *Trust in Global Computing*, number 3705 in LNCS, pages 116–145. Springer, 2005.

[8] Joshua D Guttman, Moses D Liskov, and Paul D Rowe. Security goals and evolving standards. In *Security Standardisation Research*, pages 93–110. Springer, 2014.

[9] Joshua D. Guttman, F. Javier Thayer, Jay A. Carlson, Jonathan C. Herzog, John D. Ramsdell, and Brian T. Sniffen. Trust management in strand spaces: A rely-guarantee method. In David Schmidt, editor, *Programming Languages and Systems: 13th European Symposium on Programming*, number 2986 in LNCS, pages 325–339. Springer, 2004.

[10] Gavin Lowe. Breaking and fixing the Needham-Schroeder public-key protocol using FDR. In *Proceeedings of* TACAS, volume 1055 of *LNCS*, pages 147–166, 1996.

[11] Jay A. McCarthy, Shriram Krishnamurthi, Joshua D. Guttman, and John D. Ramsdell. Compiling cryptographic protocols for deployment on the web. In *World Wide Web Conference*, pages 687–696. ACM, 2007.

[12] Lawrence C. Paulson. Relations between secrets: Two formal analyses of the Yahalom protocol. *Journal of Computer Security*, 2001.

[13] John D. Ramsdell. Deducing security goals from shape analysis sentences. The MITRE Corporation, April 2012. `http://arxiv.org/abs/1204.0480`.

[14] Paul D. Rowe, Joshua D. Guttman, and Moses D. Liskov. Measuring protocol strength with security goals. *International Journal of Information Security*, 15(6):575–596, November 2016. DOI 10.1007/s10207-016-0319-z, `http://web.cs.wpi.edu/~guttman/pubs/ijis_measuring-security.pdf`.

[15] The GHC Team. GHC user's guide documentation, release 8.2.1, July 2017. `https://downloads.haskell.org/~ghc/latest/docs/users_guide.pdf`.

Index

^(macro splicing), 101

absent, 48
added strand, 42
akey, 20
Algebra, 11
algebra, 108
and, 52
assume declarations, 68
augmentation
  listener, 41
  regular, 41
auth, 78

Blanchet protocol, 18
bltk, 36

carried subterm, 41, 47
cat, 12
channel, 76
  annotations, 78
  authenticated, 76
  confidential, 76
check-nonces option, 93, 108
cohort, 13
comments, 101
conf, 78
constants, 35
contraction, 41

data, 20
defgoal, 52
deflistener, 21, 49
defmacro, 100
defprotocol, 11
defrole, 12
defskeleton, 11, 49
defstrand, 49
defstrandmax, 27, 49
deletion, 33
depth limit, 92, 105
Diffie-Hellman, 31
  algebra, 25, 108
  plain, 32
  unauthenticated, 32
displacement, 41

enc, 12
encryption
  function symbol, 10
event, 46
examples
  Blanchet, 18
  Diffie-Hellman, 32
  Envelope protocol
    macros in, 100
  Kerberos, 26, 42
    with bi-directional keys, 36
  Needham-Schroeder, 9
    logical goals in, 52
  Otway-Rees, 31
    with bi-directional keys, 36
  Station to Station, 34
  Unified Method, 34
  Yahalom, 31
    with bi-directional keys, 36
exists, 52

`expt`, 33, 105

fact term, 63
`false`, 52
file extensions
  .scm, 11
`forall`, 52
forward secrecy, 34

`gen-st`, 88
generalization, 33
generated state value, 88
generation, 47
generics, 27
goals-sat option, 108
graphing, 13
guarantee declarations, 70

`hash`, 35
hash functions, 35
herald, 109

`implies`, 52
`include`, 100
instance, 11
interrupting, 93

Kerberos protocol, 26, 42
key-of function symbol, 10

label, 13
lang field, 37
leads-to relation, 83
listener, 21
listener augmentation, 41
`load`, 87
`locn`, 86
long-term key
  bidirectional, 36
Lowe attack, 18
`ltk`, 26, 31

macros, 100
  nesting, 101
  splicing, 101
maplet, 13
`mesg`, 27
message algebra, 10
minimality of CPSA output, 34
model, 10
mutable state, 83
  axioms, 84

Needham-Schroeder, 9
  logical goals in, 52
Needham-Schroeder-Lowe protocol, 18
non-orig, 13
`non-orig`, 47

operation field, 42
options
  algebra, 108
  bound, 105
  check-nonces, 93, 108
  depth, 105
  expand, 105
  goals-sat, 108
  help, 108
  limit, 105
  margin, 105
  noanalyze, 105
  noisochk, 105
  output, 105
  reverse-nodes, 93, 108
  show-algebras, 108
  try-old-strands, 93, 108
  version, 109
`or`, 52
origination, 47
  non-origination, 47
  penetrator non-origination, 47
  unique, 47
Otway-Rees protocol, 31
  with bi-directional keys, 36

pairing function symbol, 10
pen-non-orig, 47
priority, 93
protocol, 10
pubk, 12

recv, 12
regular augmentation, 41
rely declarations, 69
reverse-nodes option, 93, 108
rndx, 33
role, 10
role declarations, 20

search tree, 14
    colors, 14
seen child, 108
send, 12
shape, 14
signatures, 20
skeleton, 11
    dead, 14, 21
    realized, 14, 41
skey, 20
state, 83
Station to Station protocol, 34
step limit, 92, 105
stor, 87
strand bound, 92, 105
strand spaces, 46

tag, 25, 35
tags, 35
tooltip
    instance, 16
    skeleton node, 14
trace, 12
trans, 88
try-old-strands option, 93, 108
tuple, 36

Unified Method, 34
uniq-gen, 33, 47
uniq-orig, 13
uniq-orig, 47

vars, 12

Yahalom protocol, 31